

复杂系统架构模型正确性检查方法研究

2023 年 6 月

中图分类号：TP18
UDC 分类号：621.3

复杂系统架构模型正确性检查方法研究

作者姓名	<u>丁洁</u>
学院名称	<u>机械与车辆学院</u>
指导教师	<u>姚丽亚副教授</u> <u>王国新教授</u>
答辩委员会主席	<u>王武宏教授</u>
申请学位	<u>专业学位硕士</u>
学科专业	<u>机械</u>
学位授予单位	<u>北京理工大学</u>
论文答辩日期	<u>2023 年 6 月</u>

Research on the Correctness Checking Method of Complex System Architecture Model

Candidate Name:	<u>Ding Jie</u>
School Or Department:	<u>School of Mechanical Engineering</u>
Faculty Mentor:	<u>Associate Prof. Yao Liya</u>
	<u>Prof. Wang Guoxin</u>
Chair, Thesis Committee:	<u>Prof. Wang Wuhong</u>
Degree Applied:	<u>Professional Degree of Master</u>
Major:	<u>Mechanical Engineering</u>
Degree By:	<u>Beijing Institute of Technology</u>
The Date Of Defence:	<u>June, 2023</u>

复杂系统架构模型正确性检查方法研究

丁洁

北京理工大学

研究成果声明

本人郑重声明：所提交的学位论文是我本人在指导教师的指导下进行的研究工作获得的研究成果。尽我所知，文中除特别标注和致谢的地方外，学位论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京理工大学或其它教育机构的学位或证书所使用过的材料。与我一同工作的合作者对此研究工作所做的任何贡献均已在学位论文中作了明确的说明并表示了谢意。

特此申明。

签 名： 日期：

关于学位论文使用权的说明

本人完全了解北京理工大学有关保管、使用学位论文的规定，其中包括：①学校有权保管、并向有关部门送交学位论文的原件与复印件；②学校可以采用影印、缩印或其它复制手段复制并保存学位论文；③学校可允许学位论文被查阅或借阅；④学校可以学术交流为目的，复制赠送和交换学位论文；⑤学校可以公布学位论文的全部或部分内容（保密学位论文在解密后遵守此规定）。

签 名： 日期：

导师签名：

摘要

在采用基于模型的系统工程（Model-based Systems Engineering, MBSE）方法进行复杂系统设计时，是以系统架构模型为核心驱动后续研发过程。采用约束规则对系统需要满足的设计规范进行描述，并基于约束规则对系统架构模型进行正确性检查能够尽早发现不满足设计规范的、有缺陷的模型，从而避免错误的模型传播到设计后期引起更大的问题和隐患。然而，在多领域交叉的复杂系统架构设计场景下，目前的模型正确性检查方法具有一定的局限性，具体表现为：**（1）缺乏面向模型正确性检查的约束规则体系**，导致建模工程师难以完整地定义用于正确性检查的所有约束规则。**（2）缺乏同时支持模型构建和正确性检查的建模语言**，导致工程师需要耗费额外的精力将异构的模型信息转换和传递至正确性检查工具。**（3）模型正确性检查方法不具备通用性**，导致对多领域的异构系统架构模型进行正确性检查时，需要应用多种检查方法，增加学习成本。针对上述问题，开展面向多领域系统架构模型的正确性检查方法研究，具体内容如下：

（1）面向模型正确性检查的约束规则体系构建。首先界定模型正确性的内涵，并给出模型正确性检查的约束规则类型，用以支持模型正确性内涵的描述。然后结合 MOF（Meta Object Facility）元建模框架和 RFLP（需求-功能-逻辑-物理）建模方法，将正确性检查的约束规则定义过程分为两个阶段（元模型和模型构建阶段）以及三个级别（范式级别-建模领域级别-目标建模对象级别），确定每个级别所关注的模型正确性内涵，形成完整的约束规则体系。

（2）面向多领域系统架构模型的正确性检查方法。首先，以 GOPRR-E（Graph-Object-Point-Property-Relationship-Role-Extension）元建模方法为基础实现多领域系统架构模型的统一表达。其次，提出面向 GOPRR-E 方法所构建模型的正确性检查方法，将检查流程分为：约束规则定义，模型查询和约束可满足性检查三个步骤。在此基础上，采用图模式匹配和一阶逻辑方法分别实现检查流程中的模型查询过程和约束可满足性检查过程，从而实现多领域系统架构模型的正确性检查。

（3）模型正确性检查语言设计与实现。在上述方法基础上，以支持 GOPRR-E 元建模方法的多架构建模语言 KAMRA（Kombination of Architecture Model Specification）为基础，设计模型正确性检查语言。首先在 KARMA 语言已有语法规义的基础上进行

扩展，形成模型正确性检查部分的抽象语法和具体语法语义，实现基于 KARMA 的多领域架构模型描述以及正确性检查的一体化表达。然后构建面向 KARMA 模型的正确性检查插件，实现模型构建以及正确性检查的平台一体化。

（4）**面向供配电系统的模型正确性检查案例**。该案例以运载火箭供配电系统为对象，通过 KARMA 语言和 RFLP 方法构建供配电系统架构模型。依据所构建的约束规则体系，梳理规则体系每个级别中与所关注正确性内涵相关的约束规则。最后，使用所提出的模型正确性检查方法和语言，自动地识别出供配电系统架构模型中不满足约束规则的错误模型，以验证所提方法和语言的有效性。

关键词：MBSE；复杂系统设计；系统架构模型；模型正确性检查；KARMA 语言

Abstract

When Model-based Systems Engineering (MBSE) is adopted for complex system design, the system architecture models are taken as the core to drive the subsequent research and development process. Constraint rules are used to describe the design specifications that the system needs to meet. The correctness checking of system architecture models based on constraint rules can find wrong models that do not meet the design specifications as early as possible, thus avoiding the propagation of wrong models to the later design stage and cause greater problems. However, in complex system architecture design scenarios where multiple domains intersect, current model correctness checking methods have certain limitations. **(1)Lack of constraint rule hierarchy for model correctness checking**, which makes it difficult for modeling engineers to completely define all constraint rules for correctness checking. **(2)Lack of modeling language and tools to support both model construction and correctness checking**, resulting in extra effort to transform and transfer heterogeneous model information to correctness checking tools. **(3)The method of model correctness checking is not universal**, which leads to apply several checking methods to the correctness checking of multi-domain heterogeneous architecture models, increasing the learning cost. To solve the above problems, this paper researchs on the correctness checking method of the multi-domain system architecture model. The specific content is as follows:

(1)Research on the construction of constraint rule hierarchy for model correctness checking. Firstly, the connotation of model correctness is defined, and the constraint rule type of model correctness checking is given to support the description of model correctness connotation. Then, based on the MOF (Meta Object Facility) framework and the Requirement-Function-Logic-Physical (RFLP) modeling method, The process of defining constraint rules for model correctness checking is divided into two stages (metamodel and model construction stage) and three levels (formalisim level, modeling domain level and target modeling object level), Then, the model correctness connotation concerned by each level is determined to form a complete constraint rule hierarchy.

(2)Research on the correctness checking method of multi-domain system

architecture models. Firstly, we use the GOPRR-E (Graph-Object-Point-Property-Relationship-Role-Extension) meta-modeling method to realize the unified representation of the multi-domain system architecture models. Secondly, a correctness checking method for the model constructed by GOPRR-E method is proposed. The checking process is divided into three steps: constraint rule definition, model query and constraint satisfaction checking. On this basis, the model query process and the constraint satisfiability checking process are realized respectively based on the graph pattern matching and the first-order logic, so as to realize the correctness checking of the multi-domain system architecture models.

(3)Research on the design and implementation of model correctness checking language. Based on the above method and the multi architecture modeling language KAMRA which supports the GOPRR-E method, the model correctness checking language is designed. Firstly, extend the existing syntax and semantics of KARMA to form abstract and concrete syntax for model correctness checking. Thus achieving integrated expression of model description and correctness checking based on KARMA. Then, a plug-in for KARMA model correctness checking is constructed to realize the platform integration of model construction and model correctness checking.

(4) Case Study of model correctness checking for power supply and distribution system. This case takes the power supply and distribution system of a launch vehicle as the target, and constructs the power supply and distribution system architecture models using KARMA language and RFLP method. Then, based on the proposed constraint rule hierarchy, sort out constraint rules related to the concerned correctness connotation in each level of the rule hierarchy. Finally, the proposed model correctness checking method and language are used to automatically find out erroneous models in the power supply and distribution system architecture models that do not meet constraint rules. Therefore, the effectiveness of the proposed method and language is verified.

Key Words: MBSE; Complex system design; System architecture models; Model correctness checking; KARMA language

目录

摘要	6
Abstract	8
图目录	14
表目录	17
第 1 章 绪论	18
1.1 研究背景	18
1.2 国内外研究现状	19
1.2.1 复杂系统架构模型构建	19
1.2.2 模型正确性的内涵及约束规则体系	22
1.2.3 模型正确性检查方法	24
1.2.4 研究现状存在的问题	26
1.3 研究意义	26
1.4 研究内容与组织结构	27
1.5 本章小结	28
第 2 章 系统架构模型正确性检查理论与研究框架	29
2.1 引言	29
2.2 多领域系统架构模型统一构建方法	29
2.2.1 元建模技术	29
2.2.2 GOPPRR-E 元建模方法	34
2.2.3 基于 GOPPRR-E 元建模方法的系统架构模型构建过程	36
2.3 系统架构模型正确性检查相关理论	37
2.3.1 模型正确性检查的定义	37
2.3.2 模型正确性检查方式	38
2.3.3 模型正确性检查语言	39
2.4 多领域系统架构模型正确性检查研究框架	39
2.5 本章小结	40
第 3 章 面向模型正确性检查的约束规则体系构建	41

3.1 引言	41
3.2 模型正确性的内涵界定	41
3.3 模型正确性检查的约束规则类型	42
3.3.1 模型结构约束:	43
3.3.2 属性约束	45
3.3.3 混合约束	46
3.4 面向模型正确性检查的约束规则体系	46
3.4.1 RFLP 建模方法	46
3.4.2 约束规则体系构建	50
3.5 本章小结	52
第 4 章 系统架构模型正确性检查方法	53
4.1 引言	53
4.2 面向 GOPPRR-E 模型的正确性检查流程	53
4.3 基于图模式匹配的约束上下文模型查询	54
4.3.1 图模式匹配相关理论与概念	54
4.3.2 GOPPRR 模型元素至图的映射	56
4.3.3 基于图模式匹配的模型查询流程	59
4.4 基于一阶逻辑的 GOPPRR-E 模型约束可满足性检查	60
4.4.1 一阶逻辑理论	60
4.4.2 基于一阶逻辑的模型约束形式化表达	63
4.4.3 基于一阶逻辑的约束可满足性检查过程	68
4.5 模型正确性检查方法应用案例	69
4.6 本章小结	71
第 5 章 模型正确性检查语言设计与实现	72
5.1 引言	72
5.2 基于 KARMA 的模型检查语言抽象语法	72
5.2.1 KARMA 语言介绍	72
5.2.2 KARMA 语言模型正确性检查部分抽象语法	74
5.3 基于 KARMA 的模型正确性检查语言具体语法	76

5.3.1 模型正确性检查语言关键字	76
5.3.2 语言具体语法设计	77
5.4 模型正确性检查插件开发	85
5.4.1 ANTLR 技术介绍	86
5.4.2 Z3 约束求解器介绍	86
5.4.3 AGG 图匹配引擎介绍	87
5.4.4 模型正确性检查解析器和执行器构建	87
5.5 模型正确性检查语言应用案例	89
5.6 本章小结	91
第 6 章 供配电系统的模型正确性检查案例	92
6.1 引言	92
6.2 运载火箭供配电系统设计背景	92
6.3 供配电系统架构模型约束规则的定义	93
6.3.1 建模工具和建模语言	93
6.3.2 元模型构建阶段约束规则梳理	94
6.3.3 基于 RFLP 方法构建供配电系统架构模型	97
6.3.4 供配电系统架构模型构建阶段的约束规则梳理	106
6.4 模型正确性检查的实施	107
6.5 案例总结与分析	125
6.6 本章小结	127
总结与展望	128
工作总结	128
工作展望	129
参考文献	131
攻读学位期间发表论文与研究成果清单	137
参与科研项目:	137
发表论文:	137
软件著作权:	137
相关专利:	137

相关竞赛:	137
致谢	139

图目录

图 1.1 论文组织结构	28
图 2.1 MOF 元建模框架.....	30
图 2.2 六种方法的元元模型类图	32
图 2.3 GOPPRR-E 元元模型及其之间的关系	34
图 2.4 基于 GOPPRR-E 元建模方法的模型构建流程	36
图 2.5 子系统 A 和 B 的内部模块图	38
图 2.6 系统架构模型正确性检查研究框架	40
图 3.1 RFLP 方法应用于 V 模型的需求分析与架构设计阶段	47
图 3.2 RFLP 方法建模具体流程.....	48
图 3.3 模型正确性检查约束规则体系	50
图 4.1 面向 GOPPRR-E 方法所构建模型的正确性检查流程	53
图 4.2 基于图表达 GOPPRR-E 元模型	57
图 4.3 基于图模式匹配的模型查询	60
图 4.4 基于一阶逻辑的约束可满足性检查过程	68
图 4.5 UML 活动图中的元模型（部分）	69
图 4.6 案例模式图、源实例图及查询结果	70
图 4.7 温度控制系统活动图约束可满足性检查示例	71
图 5.1 KARMA 语言模型描述部分抽象语法结构	73
图 5.2 MetaModelClass 类和 ModelClass 类的具体语法规则示例	74
图 5.3 基于 KARMA 扩展的模型正确性检查部分抽象语法	75
图 5.4 模型结构实例	80
图 5.5 含有基数的模型结构示例	81
图 5.6 模型正确性检查插件开发框架	85
图 5.7 基于 ANTLR 的语法解析过程	86
图 5.8 MetaGraph 集成模型正确性检查插件	88
图 5.9 基于 MetaGraph 的模型正确性检查流程	89
图 5.10 在 MetaGraph 中执行模型正确性检查示例	90

图 5.11 模型正确性检查语言实现约束规则的构建和执行示例	91
图 6.1 运载火箭供配电系统（局部）	92
图 6.2 基于 KARMA 描述的图形式和文本式需求图元模型	94
图 6.3 供配电系统建模流程	97
图 6.4 供配电系统任务需求图	98
图 6.5 供配电系统完整需求图	98
图 6.6 供配电系统上下文和供配电系统用例图	99
图 6.7 描述所有用例场景的活动分析图	99
图 6.8 紧急断电功能细化与分配	100
图 6.9 状态监控功能细化与分配	100
图 6.10 供配电系统顶层视图	101
图 6.11 外部接口交互内部模块图	101
图 6.12 活动至结构分配表单（部分）	101
图 6.13 供配电系统逻辑组成模块定义图	102
图 6.14 供配电系统逻辑连接内部模块图	102
图 6.15 活动至结构分配表单（部分）	103
图 6.16 DC0 电池的物理组成模块定义图	103
图 6.17 组件之间活动分析图（部分）	104
图 6.18 一级和二级自主配电器物理组成	104
图 6.19 自主配电器物理连接内部模块图	104
图 6.20 供配电系统物理组成模型定义图	105
图 6.21 供配电系统物理组成内部接口交互图	105
图 6.22 rule1A 和 rule1B 的约束上下文模式图	109
图 6.23 rule1A 和 rule1B 执行结果（错误信息仅展示部分）	110
图 6.24 rule2A~rule2C 的约束上下文模式图	111
图 6.25 基于 rule2A~rule2C 执行模型正确性检查	112
图 6.26 rule3A~rule3E 约束上下文模式图	115
图 6.27 执行 rule3A~rule3E 后输出的违反约束的错误信息	116
图 6.28 不满足规则 rule3A~3D 的模型元素示例	117

图 6.29 fix 自动修复功能示例	118
图 6.30 Rule4 约束上下文模式图	119
图 6.31 rule4A 和 rule4B 执行结果	120
图 6.32 rule5 约束上下文模式图	122
图 6.33 执行 rule5A 和 rule5B 后的结果示例	122
图 6.34 rule6 约束上下文模式图	124
图 6.35 执行 rule6 之后输出的错误信息	125

表目录

表 1.1 主流建模语言	19
表 1.2 主流六种元建模语言	20
表 1.3 主流建模工具	22
表 1.4 模型质量类型	23
表 1.5 模型正确性检查方法	25
表 2.1 六种元元模型能够表达的概念分析	33
表 3.1 模型结构约束	43
表 3.2 属性约束	45
表 3.3 RFLP 方法与 Harmony SE、OOSEM 和 MagicGrid 方法的过程映射	47
表 4.1 有关个体词、谓词的相关概念和符号	61
表 4.2 一阶逻辑公式相关概念	62
表 5.1 模型正确性检查语言关键字	76
表 5.2 二元关系和三元关系	79
表 6.1 范式级别和建模领域级别约束规则	95
表 6.2 目标建模对象级别约束规则	106
表 6.3 应用的约束规则类型、描述的正确性内涵及执行后的效果	126

第1章 绪论

1.1 研究背景

系统工程（System Engineering, SE）是一种跨学科的方法和手段，旨在成功地研发和实现系统，它关注全生命周期流程中所有利益相关方与研发参与者之间的交互活动^[1]。传统的系统工程采用基于文档的研制模式支持系统设计，设计数据存储于大量的研制文件和过程文件中，并以文档的方式与生命周期所有参与方进行交互和追溯，涉及了大量的手工活动^[2]。随着系统复杂程度的日益增加，系统研制过程呈现涉及多学科多领域、接口数量庞大、协同设计单位众多的特点，以文档为中心的工作方式越来越难以应对多学科协同设计模式下的数据一致性追溯、交互变更、验证确认等挑战。

为了应对上述挑战，系统工程开始从基于文档的系统工程过渡至基于模型的系统工程（Model-based Systems Engineering, MBSE）。MBSE 通过形式化建模方法，将复杂系统全生命周期中所涉及到的需求、设计、分析、验证和确认等活动都使用计算机可解析的模型来表达^[3]。通过模型承载设计信息，有利于采用自动化的手段进行模型间信息的追溯以及设计自动变更，减少人为手段更新过程中带来的不一致^[4]。基于模型开展系统设计，有利于借助规范良好的建模语言描述准确、无歧义的设计信息。同时支持在早期设计阶段开展基于模型的集成验证，通过自动化的手段对系统的功能、性能以及接口等方面进行验证与确认，从而提前识别早期设计过程中的错误，减少由于返工而增加的成本、缩短研制周期。

在 MBSE 中，以早期设计阶段的系统架构模型为唯一真相源指导后续的设计研发过程，不符合设计规范和需求的系统架构模型通过传播将引起设计后期更大的错误和隐患，从而降低研发效率，增加研制成本^[5]。因此，系统架构模型的质量至关重要。模型的质量指的是模型及其含义的正确性和完整性^[6]，例如所构建的模型是否对所建模领域具有正确的表述，是否拥有所有必要的相关信息以充分地表达利益相关者的建模目的。模型正确性检查是一种通过半自动/自动化的手段检查给定架构模型是否满足给定规范的手段，通过模型正确性检查能有效提升系统架构模型质量。例如，通过定义若干针对模型元素的约束规则，并根据自动化的检查引擎得出违反约束规则的模型元素，建模工程师即可在设计早期及时发现和改正有缺陷的模型，从而提升模型的质量。然而，多领域交叉的复杂系统设计背景下，目前 MBSE 领域内的模型正确性检

查方法具有一定的局限性。其一，缺乏面向模型正确性检查的约束规则体系，导致建模工程师难以完整地定义用于正确性检查的所有约束规则。其二，缺乏同时支持模型构建和正确性检查的建模语言，导致需要耗费额外的精力将异构的模型信息转换至正确性检查工具，增加资源消耗。其三，模型正确性检查方法不具备通用性，导致涉及多领域的复杂系统架构模型的正确性检查需要应用多种检查方法，增加学习成本。

1.2 国内外研究现状

1.2.1 复杂系统架构模型构建

系统架构模型是表示如何实现系统所有类型需求的第一个方案，是早期设计决策的体现^[7]。模型正确性检查用于识别系统架构模型中不满足设计规范的错误模型，系统架构模型的构建是模型正确性检查的基础，而系统架构模型的构建离不开建模语言、建模方法和建模工具。建模语言通过其语法和语义来定义模型中能够出现的模型元素的种类以及元素之间的关系，建模语言能否支持涉及多种领域的系统架构模型的描述以及模型的正确性检查，取决于其语法规义的完整性和精确性。建模方法是指导整个建模过程的一整套理论和方法，它规定了系统开发过程所包含的模型构建内容以及基于模型的分析、验证等活动^[8]。建模工具是遵循一种或多种建模语言，为工程师提供使用建模语言构建模型的平台^[9]。建模工具能否实现模型构建、模型正确性检查的平台一体化取决于其功能的丰富性。下文将从建模语言、建模方法和建模工具三个方面阐述复杂系统架构模型构建的研究现状。

1.2.1.1 建模语言

建模语言是基于元模型的，元模型陈述了语言中的概念以及用这些概念指定的模型需要满足的语义规则^[10]，而建模语言的元模型可以使用元建模语言来定义。此外，从表现形式来看，建模语言具有文本式和图形式两种形式。

(1) 建模语言

目前国内外在建模语言和元建模语言方面已经开展了大量的研究工作，针对不同的设计领域，提出了不同的建模语言。表 1.1 总结了主流的图形化和文本式建模语言。

表 1.1 主流建模语言

语言	名称	简介
----	----	----

图形化建模语言	UML	UML (Unified Modeling Language) 是软件工程的标准建模语言, 可以支持模型化和软件系统开发的图形化表示 ^[11] 。
	SysML	SysML (Systems Modeling Language) 是 OMG 推出的一种标准化系统建模语言, 是 UML 在系统工程领域的延伸 ^[12] 。
	BPMN	BPMN (Business Process Modeling Notation) 是一套标准化的业务流程建模语言 ^[13] , 可以用于设计业务流程操作的图形化模型。
	UPDM	UPDM ^[14] (Unified Profile for DoDAF and MODAF) 是一个建模标准, 支持美国国防部的防务架构框架和英国国防部的防务架构框架。
	AADL	AADL ^[15] (架构分析和设计语言) 是一种标准化架构描述语言, 常用于嵌入式实时系统的软硬件建模。
文本式建模语言	Modelica	Modelica ^[16] 是一种开放、面向对象、基于方程的建模仿真语言, 可以跨越不同领域, 方便地实现复杂物理系统的建模仿真。
	SysML v2	SysML v2 是在 SysML 基础上形成的文本式建模语言, 提高了其语言概念的准确性、互操作性、一致性和集成性。
	KARMA	KARMA 采用元建模方法支持构建 UML、SysML、BPMN 等语言的模型以及针对特定域开发的模型, 实现对建模语言的统一描述 ^[17] 。

图形化建模语言使用带有特定符号的图表技术来表示概念和概念之间关系, 是半形式化的语言^[18]。图形化建模语言所构建的模型较为直观, 但所表达的内容不够精确。而文本式建模语言使用计算机可解释的表达式来表达模型, 形式化程度更高, 能对模型信息做更为准确的表达, 但对于用户来说更难理解。目前主流使用的建模语言 (SysML、UML 等) 多为图形化建模语言, 支持所在领域系统架构模型的构建, 但缺乏更精确和完整的语法规则以支持描述模型的正确性检查。

(2) 元建模语言

元建模语言通过元元模型来支持元模型的构建。现有研究中主要包括六种元建模语言, 如表 1.2 所示。此六种元建模语言的应用范围涵盖集成信息系统领域、电气工程领域、软件工程领域, 而对于涉及多领域的复杂系统架构模型的支持程度有待评估。

表 1.2 主流六种元建模语言

名称	介绍
----	----

ARIS	ARIS(Architecture of Integrated Information Systems)是一种集成信息系统领域的元建模结构，其元元模型包括模型、对象定义类型、关联定义类型，属性等 ^[19] 。
ECore	ECore 是 EMF (Eclipse Modeling Framework) 中使用的元建模语言 ^[20] ，核心元素包括 E 类、E 属性、E 引用以及 E 数据类型。
GME	GME (The Generic Modeling Environment) 是一个用于电气工程领域建模的工具 ^[21] 。其元元模型为：模型、原子、集、关联引用、关联角色、属性、范式。
GOPRR	GOPRR 是 MetaEdit+内嵌的一种元建模语言 ^[22] ，由六种元素构成整个建模框架的元元模型，分别为图、对象、点、属性、角色、关系。
MS DSL Tools	MS DSL Tools 采用了领域类、领域关系、领域角色、命名领域元素、领域属性和语言六种元元模型来支持建模语言的定制 ^[23] 。
Microsoft Visio	Microsoft Visio 是一个通用的建模和数据可视化工具，支持通过模板的方式定义建模语言，其元元模型包括：模板、母板和属性 ^[24] 。

1.2.1.2 建模方法

国外对于 MBSE 建模方法的研究起步较早，目前主流使用的建模方法包括：Harmony-SE 方法、面向对象的系统工程方法 OOSEM 和 MagicGrid 方法。

Harmony-SE 以 SysML 为建模语言，其设计流程分为：需求分析、系统功能分析、结构综合设计^[25]。需求分析用于收集利益相关人的需要并将其转化为系统需求；功能分析用于将需求转化为一致的系统操作；设计综合用于开发满足系统需求的体系架构。

OOSEM 使用 SysML 来支持系统的分析、设计和验证^[26]。OOSEM 的流程包括：分析利益相关者的需求、定义系统需求、定义逻辑体系结构、合成候选分配架构、优化和评估可选方案、验证和确认系统。

MagicGrid 以 SysML 为建模语言，将建模流程分为问题域和解决域^[27]。问题域分为黑盒分析和白盒分析。黑盒分析用于描述顶层系统功能，系统与外部环境的交互。白盒分析则描述系统内部的子系统和组件。解决域用于定义系统逻辑架构的精确模型。

总的来说，主流的建模方法通常面向某种特定的建模语言（如 SysML），但一种建模语言难以解决复杂系统设计域中的所有问题。此外，建模方法更加关注模型的构建，而缺乏对模型进行正确性检查的理论指导。

1.2.1.3 建模工具

建模工具是实现 MBSE 落地的直接环节，表 1.3 总结了目前国内外所使用的主流建模工具，可以发现多数建模工具能够支持一种或几种建模语言（如 SysML、UML）模型的构建，但对模型的正确性检查功能支持有限，难以实现涉及多种领域系统架构模型的正确性检查。

表 1.3 主流建模工具

提出者	建模工具	功能
No magic 公司	MagicDraw	支持 UML、SysML、BPMN 和 UPDM 的可视化建模，支持工程师进行需求验证、决策权衡以及需求追溯分析 ^[28] 。在模型正确性检查方面，仅支持 SysML/UML 的接口不兼容性检查。
Thales 公司	Capella	Capella ^[29] 为系统、软件和硬件架构师提供方法论指导，支持对系统架构进行早期分析和验证。
IBM 公司	Rational Rhapsody	Rhapsody 是基于 UML/SysML 的模型驱动开发集成环境,专注于嵌入式和实时系统 ^[30] 。
Sparx Systems 公司	Enterprise Architect	是一个支持 UML/SysML/BPMN 语言的系统设计和架构分析工具，也支持基于模型的分析、实现、仿真等功能。在模型正确性检查方法，支持通过 OCL 构建 UML 和 SysML 的约束规则 ^[31] 。
浙江大学	M-Design	支持 SysML 建模，支持系统设计过程中的概念设计和详细设计等过程中的跨地域协同设计 ^[32] 。
杉石公司	Modelook	支持电子系统设计的模型化描述，实现系统架构的设计与优选 ^[33] 。

1.2.2 模型正确性的内涵及约束规则体系

模型正确性检查与所要发现的模型缺陷相关，模型缺陷即模型中的要素违反了某一或多个模型质量特性。因此，模型正确性的内涵与模型的质量类型息息相关。目前针对模型质量类型的分类已开展了深入的研究，表 1.3 总结了若干个关于模型质量分类的研究，可以发现多数研究聚焦于软件工程领域，以 UML 模型为主，关于复杂系统工程领域的研究较少。

表 1.4 模型质量类型

提出者	面向模型领域	模型质量分类
Lindland ^[34] 1994	需求模型	(1) 合适的：模型中的概念必须适合领域中的系统角色。 (2) 简洁明了：模型中的概念简单清晰、易于理解。(3) 完整的：包含所有需要的信息。(4) 表达能力强：每个需求无需过多的陈述。(5) 无歧义的：每个需求只有一种解释。 (6) 一致的：需求之间彼此没有矛盾。
Krogstie ^[35] 2000	UML 模型	在 Lindland 基础上增加了：(1) 符合建模目标且所有目标都可通过模型实现。(2) 技术可行性：即模型可被工具解释。
Unhelkar ^[36] 2006	UML 模型	(1) 语法：指模型与建模语言的对应程度。如果模型包含语言中未定义的符号，或者模型缺乏符合语言语法的结构或信息，就会发生语法错误。(2) 语义：是指模型与领域内知识的对应程度以及描述的完整性、一致性。(3) 符合美学：注重模型的外观，以使工程师更好理解。
Mohagheghi ^[37] 2009	软件工程领域	(1) 正确性：包括正确的要素及其之间的正确关系、关于领域的正确表述以及不违反规则和惯例。(2) 完整性：拥有所有必要的相关信息。(3) 一致性：模型中没有矛盾。涵盖了水平一致性、垂直一致性和模型之间的语义一致性。(4) 可理解性：即对于使用者是可理解的。(5) 受限性：与建模的目的致。(6) 可变性：支持模型可以快速持续地演进。
Kahraman ^[38] 2015	特定域建模	面向特定域建模语言提出了十种模型质量分类，包括：功能稳定性、可用性、可靠性、生产力、可扩展性、兼容性、表达能力、重用性、可集成性。

模型的正确性内涵中蕴含系统需要满足的给定规范，此规范可以通过约束规则进行描述。面向模型正确性检查的约束规则体系用于指导工程师根据建模每一阶段需要关注的模型正确性内涵，完整地构建所有的约束规则，以根据约束规则执行正确性检查。Berenbach^[39]从模型的组织、用例定义、用例关系、业务流程、具体模型设计等方面提炼了若干条规则来作为建模的指导法则和约束。此指导法则比较具体，具有一定的实践价值，但是彼此之间较为松散，没有形成完整的约束规则体系。Sourrouill 等人

^[40]将 UML 模型的约束检查过程分为：（1）范式级别；（2）范式拓展级别；（3）建模领域级别；（4）目标领域级别；（5）实现级别。他们阐述了五个级别的具体含义和支持定义的约束种类，为模型检查约束的定义过程提供了参考。但是在目标领域级别，缺乏在模型的详细构建过程中定义约束规则的研究。

1.2.3 模型正确性检查方法

模型正确性检查方法主要用于验证系统是否满足给定的规范（给定的规范与模型正确性的内涵相关）。在目前的研究中，有关架构模型正确性检查的研究以形式化方法为主。形式系统（或理论）由公理和推理规则组成，一个形式化的系统允许人们通过语法转换和语义可满足性测试来证明给定的表达式是否正确^[41]。主流的形式化方法包括^[42]：（1）命题演算：BDD（Binary Decision Diagram）技术和 SAT（Satisfiability Problem）等；（2）结合谓词的命题演算：如可满足性模理论（Satisfiability Modulo Theories, SMT）；（3）一阶逻辑：如 B 方法、AVL2 等；（4）时态逻辑方法。

BDD 是 Bryant 提出的一种形式化验证手段，常被用于组合电路的等价性证明和有限状态系统^[43]。布尔可满足性问题（SAT 问题）是为给定的布尔函数 F 确定一个令人满意的赋值，或者证明不存在这样的赋值的问题。Mathias 等人^[44]使用 SAT 来验证具有 OCL（Object Constraint Language）表达式的 UML 模型。他们使用 SAT 理论为 UML 模型的系统状态、OCL 约束以及验证任务进行编码，并输入可满足性求解器 MiniSAT 进行求解，从而验证 UML 模型。在使用 SAT 对 UML 模型进行验证的基础上，Mathias 等人^[45]又使用 SMT 理论来验证 UML 的动态行为，他们增加了基于 SMT 的 UML 操作调用序列的编码，从而实现多视图动态行为的验证。

B 方法（B-Method）是 Abrial^[46]开发的一个形式化描述语言，Erwan 等人^[47]使用 B 方法来验证系统的安全属性。他们建立了 SysML 模型与 B 方法语义的映射关系，从而支持 SysML 模型向 B 方法模型的转换，并采用求解工具 Atelier-B 自动验证安全性属性。时态逻辑方法研究状态随时间变化系统的逻辑特性，常应用于程序验证和硬件验证。UPPAAL^[48]是一个基于时间自动机的模型检测工具，使用多个并行自动机表示并行系统，并且在自动机中引入时间概念。Abbas 等人^[49]利用 UPPAAL 实现对 SysML 状态机模型的验证。他们首先采用模型转换工具将状态机模型转换生产 SystemC 模型，然后将 SystemC 模型转换为 UPPAAL 自动机，然后输入模型检查器得到验证结果。

除了形式化方法，还有基于规则或者图模式的方法。Liu 等人^[50]提出了 UML 模型中生产系统和生产规则的概念，他们将基于规则的生产系统模型检查过程分为：（1）识别生产规则；（2）解决规则的冲突，选择一条规则执行；（3）应用已执行规则的结果中给出的操作，以支持动态 UML 模型的不一致性检查。Herzig 等人^[51]提出了一种基于图模式的检查框架。将 UML 模型转换为三元组表示的图，将一致性规则使用图模式进行描述，通过查询图模式来识别模型中的不一致。Wei 等人^[52]利用知识工程方法来识别模型缺陷。他们将 UML 类图、序列图 and 状态图转换为 OWL 本体模型，并使用语义查询语言对本体模型进行查询，从而实现 UML 模型的一致性检查。Shaikh 等人^[53]提出了基于图的系统静态结构检查方法，通过将复杂 UML 类图切片成若干子模型，检查子模型的 OCL 约束可满足性，从而达到系统整体静态结构正确性检查的目的。EVL（Epsilon Validation Language）语言^[54]是一种基于约束规则的验证语言。EVL 支持在模型定义完成后，基于元模型指定的模型约束完成对模型的评估和快速修复。

将上述方法所阐述的内容总结为表 1.5 所示。可以发现，目前的检查方法通常面向软件工程领域的 UML 模型，能有效支持系统动态行为和静态特性的验证。但现有方法面向某种或某几种建模语言所构建模型，难以支持复杂系统设计过程中、多种领域建模语言所构建系统架构模型的检查。

表 1.5 模型正确性检查方法

提出者	理论和工具	面向的模型	检查内容
Bryant 1998	BDD 方法	组合电路模型	等价性
Liu 2002	基于规则	UML 模型	一致性
Kolovos	基于规则	UML、SysML 模型	格式规范、一致性
Mathias 2010	SAT、MiniSAT 工具	UML 模型	系统状态
Mathias 2011	SMT	UML 模型	系统动态行为
Erwan 2012	B 方法、Atelier-B 工具	SysML 模型	系统安全属性
Abbas 2014	时态逻辑、UPPAAL 工具	SysML 状态机	系统状态
Herzig 2014	基于图模式	UML 模型	一致性
Shaikh 2014	基于图、OCL	UML 类图	系统静态结构
Wei 2021	基于图模式	UML 类图、序列图 and 状态图	一致性

1.2.4 研究现状存在的问题

从上述对国内外研究现状的分析可知，MBSE 以系统架构模型为复杂系统研发过程的蓝图，为应对复杂系统设计中的挑战提供了解决方案。模型正确性检查作为一种自动化的验证手段，能有效提高 MBSE 设计过程中系统架构模型的质量。目前国内外在模型正确性检查领域已经开展了深入的研究，大多数研究聚焦于软件工程领域，在面向 UML 的模型正确性检查方法和工具领域取得了大量研究成果。但目前系统工程领域内、针对多领域系统架构模型的正确性检查方面存在以下问题：

（1）缺乏面向模型正确性检查的约束规则体系：已有研究对模型正确性内涵的分类做了大量工作，但缺乏面向模型正确性检查的约束规则体系的研究，即没有明确建模每个阶段中需要关注的模型正确性内涵以及正确性内涵与约束规则之间的关系。缺乏规则体系的指导将使工程师难以根据建模阶段中需要关注的正确性内涵，完整地定义所有的约束规则，使模型正确性检查的实施难以落地。

（2）缺乏同时支持模型构建和正确性检查的建模语言：UML、SysML 等图形化建模语言能有效地支持模型的构建，但是缺乏描述模型正确性检查的语法语义，通常需要集成另一种约束语言或者形式化检查工具来支持模型的正确性检查。建模工程师需要耗费额外的精力实现 UML 等模型至形式化检查工具的转换以实现检查，增加资源消耗。

（3）模型正确性检查方法不具备通用性：复杂系统具有跨学科、跨领域的特点，在应用 MBSE 方法进行复杂系统设计时，不同领域的架构模型采用各自领域异构的建模语言构建。现有模型正确性检查方法通常面向某种（或某几种）建模语言所构建模型的正确性检查，导致涉及多领域的复杂系统架构模型的正确性检查需要应用多种检查方法，增加学习成本。

1.3 研究意义

针对缺乏面向模型正确性检查的约束规则体系，现有建模语言难以统一实现多领域系统架构模型的构建和正确性检查的问题，从系统架构模型的构建出发，基于 GOPPRR-E 元建模方法实现多领域系统架构模型的统一描述。然后构建面向模型正确性检查的约束规则体系，明确建模每一阶段需要定义的约束规则。然后结合图模式匹

配和一阶逻辑理论，形成模型正确性检查方法。基于检查方法，构建模型正确性检查语言及其插件，实现面向多领域系统架构模型的正确性检查功能。其研究意义在于：

(1) **构建面向多领域系统架构模型的模型正确性检查理论方法。**本文提出了一种结合 GOPRR-E 元建模方法、图模式匹配和一阶逻辑理论的模型正确性检查方法。该方法采用 GOPRR-E 元建模方法，支持多领域架构模型的形式化表达；采用图模式匹配方法和一阶逻辑实现面向 GOPRR-E 的模型正确性检查。通过统一的模型正确性检查方法支持多领域异构模型的检查，能有效提升系统设计人员的集成设计能力，提高复杂系统设计效率。

(2) **解决工程领域内采用模型正确性检查语言和插件自动识别模型错误的问题。**工程领域内模型正确性检查的实施需要工程师完整地定义约束规则，并基于规则识别模型错误。针对上述需求，构建面向模型正确性检查的约束规则体系，指导工程师完整地定义约束规则。然后构建模型正确性检查语言及其插件，支持对约束规则进行描述并执行，从而自动地识别模型的错误，便于模型正确性检查实施的落地。

1.4 研究内容与组织结构

本文拟对多领域系统架构模型的正确性检查方法和语言进行研究。首先明确系统架构模型正确性检查理论与研究框架，围绕研究框架开展面向模型正确性检查的约束规则体系构建、模型正确性检查方法及语言的研究。研究的组织结构如图 1.1 所示，共分为 7 个章节，各章节的内容安排如下：

第一章：绪论。从 MBSE 领域中系统架构模型正确性检查的背景出发，阐述国内外关于系统架构模型构建、模型正确性内涵及约束规则体系、正确性检查方法等方面的研究现状，分析目前研究现状存在的问题，引出研究内容和论文结构。

第二章：系统架构模型正确性检查理论与研究框架。采用 GOPRR-E 元建模方法实现多种领域建模语言所构建系统架构模型的统一描述。然后分析模型正确性检查的定义、检查方式、检查语言等理论，提出本文的研究框架，作为后续研究的整体指导。

第三章：面向模型正确性检查的约束规则体系构建。首先界定模型正确性的内涵，然后给出模型正确性检查的约束规则类型，以描述正确性内涵。最后，结合 MOF 元建模框架和 RFLP 方法形成完整的约束规则体系。

第四章：系统架构模型正确性检查方法。首先分析面向 GOPRR-E 方法所构建模

型的正确性检查流程。然后研究基于图模式匹配的方法实现检查流程中的约束上下文查询，最后基于一阶逻辑实现检查流程中的约束可满足性检查。

第五章：模型正确性检查语言设计与实现。在采用 GOPRR-E 元建模方法的 KARMA 语言的基础上，扩展 KARMA 语言，设计用于描述模型正确性检查的抽象语法和具体语法语义。最后开发模型正确性检查插件，实现语言的模型正确性检查功能。

第六章：供配电系统的模型正确性检查案例。以运载火箭供配电系统为例，基于 KARMA 构建供配电系统架构模型，根据所提约束规则体系梳理建模过程中的所有约束规则，使用所提正确性检查方法和语言实现供配电系统架构模型的正确性检查。

最后，在总结和展望部分，对本文的研究工作和创新点进行总结，并分析研究所存在的不足和未来的工作展望。

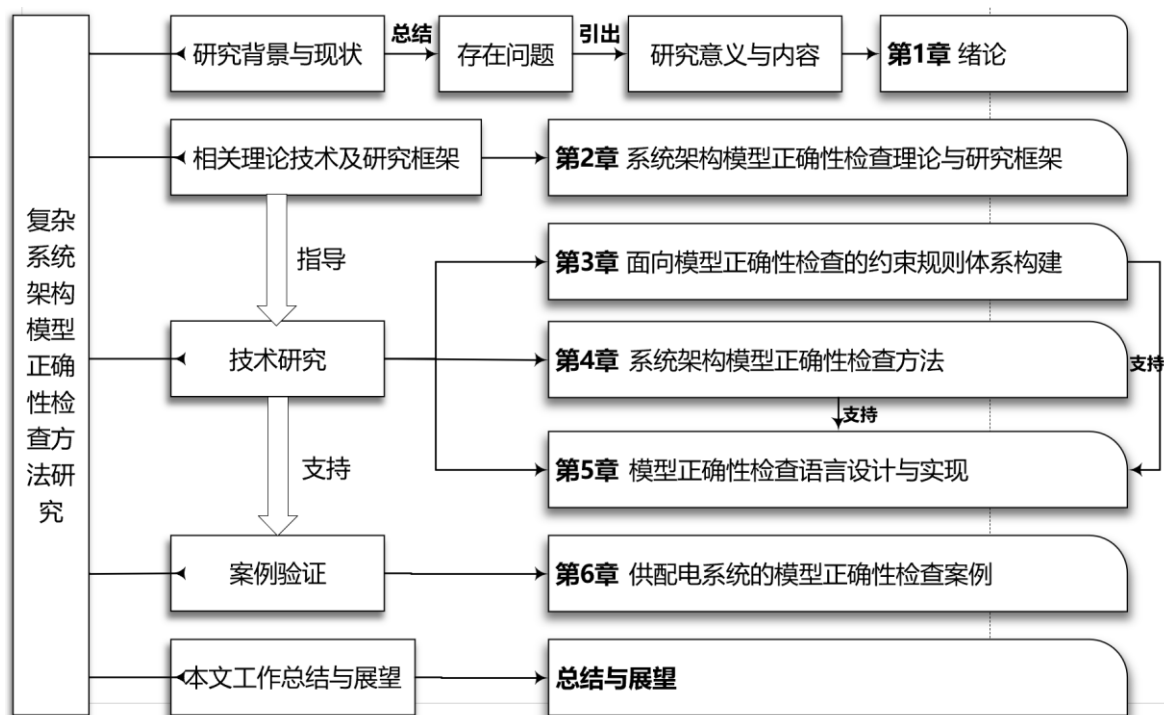


图 1.1 论文组织结构

1.5 本章小结

本章从系统工程领域的复杂系统设计出发，阐述了国内外在复杂系统架构模型构建以及模型正确性检查领域的研究现状，分析并总结了现有研究在多领域系统架构模型正确性检查方面存在的问题，从而确定本文的研究内容和组织结构。

第2章 系统架构模型正确性检查理论与研究框架

2.1 引言

采用 MBSE 方法进行复杂系统设计过程中，早期设计阶段的系统架构模型是指导后续设计研发阶段的核心。其中，系统架构模型的构建和正确性检查是其中两个关键步骤，系统架构模型的构建是正确性检查的基础。目前复杂系统设计过程呈现多层级、多领域的特点，不同领域的工程师通常采用各自领域的建模语言构建系统架构模型，这些由不同的、异构的建模语言形成的模型可简称多领域系统架构模型。这些不同领域的模型底层数据异构，难以进行统一管理、交互与正确性检查。针对以上问题，本章提出多领域系统架构模型统一构建方法，实现采用统一的底层数据描述多领域架构模型。此外，明确系统架构模型正确性检查的相关理论，然后提出本文的整体研究框架，为下文的研究提供思路指导。

2.2 多领域系统架构模型统一构建方法

系统架构模型的构建需要建模语言的支持，而元建模是一种广泛应用于定义建模语言的技术^[55]。采用元建模技术可以使用统一的底层数据构建不同领域的建模语言，从而实现多领域系统架构模型的统一表达。MOF 元建模框架是一种规范的元建模技术^[56]，它以底层的元元模型为基础构建领域建模语言。本节以 MOF 框架为基础，介绍该框架内涵，并对现有研究中存在的元元模型进行分析。最后提出 GOPPRR-E 元建模方法，并阐述基于 GOPPRR-E 元建模方法的系统架构模型构建过程。

2.2.1 元建模技术

2.2.1.1 MOF 元建模框架

OMG 为了规范的使用元建模技术，制定了元建模框架 MOF，被广泛地应用于模型驱动工程领域。如图 2.1 所示，MOF 使用四层（M3-M0）结构来指定构建元模型和模型的方法，其中四层架构的具体含义如下：

- 元元模型层（M3）：元元模型是描述模型的最底层基本元素，它是领域无关的。例如 visio 图中的一个最简单的矩形块。
- 元模型层（M2）：元模型是元元模型添加目标领域语义信息后进行实例化得

到的元素。元模型定义了目标领域的主要概念元素、元素之间的关系和元素及其关系拥有的属性，以指定模型的基本结构。构建目标领域元模型的过程实际上就是在定义目标领域建模语言。例如 SysML 中的“需求图”就是 SysML 语言的一种元模型。

- **模型层 (M1):** 模型是元模型的实例, 用于抽象的描述真实物理世界的系统视角。例如, 使用 SysML 中的需求图构建的一个运载火箭分离系统需求模型即可认为是对系统需求视角的抽象描述。
- **真实世界视角:** 描述真实世界物体/系统的某一视角。例如运载火箭系统的需求视角、行为视角等。



图 2.1 MOF 元建模框架

2.2.1.2 元元模型分析

元元模型是 MOF 元建模框架的基础，基于元元模型可以定义多种领域元模型，不同的元模型及其之间关系的组合可以形成不同的建模语言，从而实现以同一底层数据描述多种领域架构模型的目的。目前现有研究和应用中存在多种元元模型，不同的元元模型具有不同的表达能力。本节以 1.2.1 节部分六种主流的元建模方法为对象，分析它们的元元模型在描述模型概念方面的表达能力，从而确定所使用的元元模型。

ARIS 方法中具有六个元元模型：模型、对象定义类型、对象出现类型、关联定义类型、关联出现类型以及属性。模型由对象出现类型和关联出现类型元素的集合组成。对象定义类型代表一组具有相似特征的实体，用于抽象描述对象出现类型。对象出现类型和模型类型之间可以创建链接关系，表示使用模型对对象特定方面进行细化描述。

关联定义类型用于定义两个对象定义类型之间的关系，用于抽象描述关联出现类型。关联定义类型具有两个对对象定义类型的无界引用，表示可能的源对象和目标对象类型。此外，对象定义类型和关联定义类型可以具有属性。

Ecore 方法中具有四种元元模型：E 类、E 属性、E 引用以及 E 数据类型。E 类用于描述一组类似的模型实体，不同的 E 类之间可以具有继承关系。E 引用表示两个 E 类之间的单向关系。E 类可以具有 E 属性，用于表示 E 类的属性。此外 E 类在某个 E 引用上下文中可以具有多重性。

GME 方法的元元模型包括：第一类对象（模型、原子、引用、关联、集）和关联角色、属性和范式。原子是最基本的实体，不能包含内部结构。模型是由原子复合而成的对象，可以拥有内部结构。关联通过关联角色来表达两个对象之间的关系，它可以是有向的也可以是无向的。同时关联可以描述其两端所连接对象的多重性。引用用于描述不同模型之间元素的关系。建模范式决定了模型中允许哪些元素扮演哪些角色。此外，所有的第一类对象都能具有一组属性。

MS DSL Tools 方法的元元模型包括：语言、领域类、领域属性、领域角色、领域关系。语言由若干个领域类组成，用于描述特定域建模语言。领域类、领域属性、领域角色都是一种命名领域元素。领域类用于描述一类抽象的实体，它允许继承且具有领域属性。领域关系两端通过领域角色与领域类相连，并确定领域类在该连接中充当的角色（源或目标）。

MS Visio 方法的元元模型包括：模板、母版和属性。一个模板由一组母版组成。母版描述了一类可以独立存在的模型元素（在 **Visio** 中表示为形状），它可以以两种形式出现：简单形状和连接形状。在图的上下文中，简单形状和连接形状可以分别被视为节点和边。属性具有类型和默认值，每个属性均需要指派给母版。

GOPRR 方法的元元模型包括图、对象、关系、点、角色和属性，其内涵将在 2.2.2 节详细叙述，此处不赘述。基于上文的描述，分析六种方法的元元模型以及元元模型之间的关系，并使用 **UML** 中的类图绘制为图 2.2 所示。

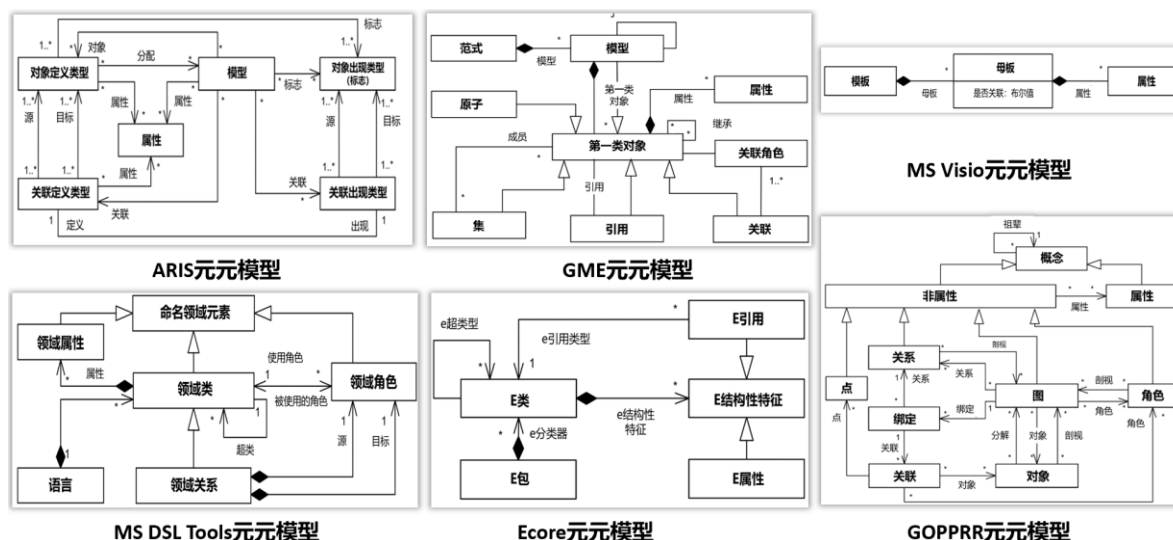


图 2.2 六种方法的元元模型类图

为了比较和分析六种方法中元元模型的表达能力，首先对基础的模型概念要素进行梳理，以 Kern 等人^[57]的研究为基础，得出基础的模型概念要素如下：

- 模型类型：描述一类具有相同属性的模型。
- 对象类型：具有相等特征的对象集或类。通常，对象类型表示建模域的概念或实体类型。
- 关系类型：关系类型用于连接对象类型。
- 对象多重性：定义了对对象类型在关系类型上下文中可以与另一个对象类型相连的对象的数量。
- 角色类型：角色类型定义对象类型如何参与关系类型。如果关系类型支持角色概念，则关系将间接地通过角色连接对象。
- 端口类型：端口类型定义了一组或一类端口，端口允许对对象之间如何通过关系连接添加附加语义。
- 组成：即对象类型之间的包含与被包含关系。组合影响容器对象与被包含对象之间的生命周期依赖关系。
- 继承：继承是模型概念之间的一种特殊关系，继承允许重用元素的属性、约束或行为等。
- 属性类型：描述模型/对象/关系类型的一类属性。
- 属性多重性：该特性允许定义多值属性（当多重性大于 1 时）。

- 实例：表示具体的对象/关系/模型/属性，其类型是对象/关系/模型/属性类型。
- 链接到模型类型：对象类型、关系类型等概念链接到模型类型，以描述模型元素的细化。

基于上述的模型概念要素，分析六种元元模型能够表达的概念，如表 2.1 所示。

表 2.1 六种元元模型能够表达的概念分析

	ARIS	Ecore	GOPRR	GME	MS DSL Tools	MS Visio
模型类型	模型	不支持	图	模型	不支持	不支持
对象类型	对象定义 对象出现	E 类	对象	原子、集	领域类	母版
关系类型	关联定义 关联出现	E 引用	关系	关联、引用	领域关系	母版
端口类型	不支持	不支持	端口	不支持	不支持	不支持
角色类型	不支持	不支持	角色	关联角色	领域角色	不支持
属性类型	属性	E 属性	属性	属性	领域属性	属性
对象多重性	未提及	通过属性 和 E 引用	通过角色	通过角色	通过角色	不支持
属性多重性	未提及	未提及	未提及	未提及	未提及	未提及
组成	不支持	支持	支持	支持	支持	不支持
继承	不支持	支持	支持	支持	支持	不支持
链接到模型 类型	支持	未提及	支持	支持	未提及	未提及

从表中可以分析得出，对象类型、关系类型和属性是基本的建模概念，六种方法中的元元模型都支持这些概念。除此之外，也都支持“依赖”概念。部分方法的元元模型支持模型类型、角色类型和端口类型的概念。模型类型允许实现视图概念，而角色类型允许实现复杂关系。Ecore、GOPRR、GME 支持对象多重性概念，Ecore、GOPRR、GME 以及 MS DSL Tools 支持继承和组成概念。链接到模型类型允许使用模型对元素的结构或者其他方面进行更细化的描述，六类元元模型中能支持此概念的为 ARIS、GME、GOPRR。通过上述比较可以发现 **GOPRR** 元元模型能够支持绝大

多数建模概念的表达，在六类元元模型中具有最强的表达能力。因此以 GOPPRR 元元模型作为多领域系统架构模型构建的基础。

2.2.2 GOPPRR-E 元建模方法

本节在 GOPPRR 元元模型的基础上添加了若干扩展(Extension)以描述 GOPPRR 元元模型之间的关联规则。然后基于 2.2.1.1 节中的 MOF 元建模框架，形成 GOPPRR-E 元建模方法。在该方法中，规定 GOPPRR 元元模型的实例称为 GOPPRR 元模型（位于 MOF 框架的 M2 元模型层），例如图元模型。GOPPRR 元模型的实例被称 GOPPRR 实例（位于 MOF 框架的 M1 模型层），例如对象实例。此外，GOPPRR 实例的类型即相应的 GOPPRR 元模型。

如图 2.3 所示，GOPPRR 元元模型包括图(Graph)、对象(Object)、点(Point)、属性(Property)、关系(Relationship)、角色(Role)。其中图、对象、关系、点、角色是非属性概念，用于描述模型中的元素及其组成的结构。属性元元模型是属性概念，用于描述非属性概念具有的参数特性。六个元元模型的具体含义如下所示。

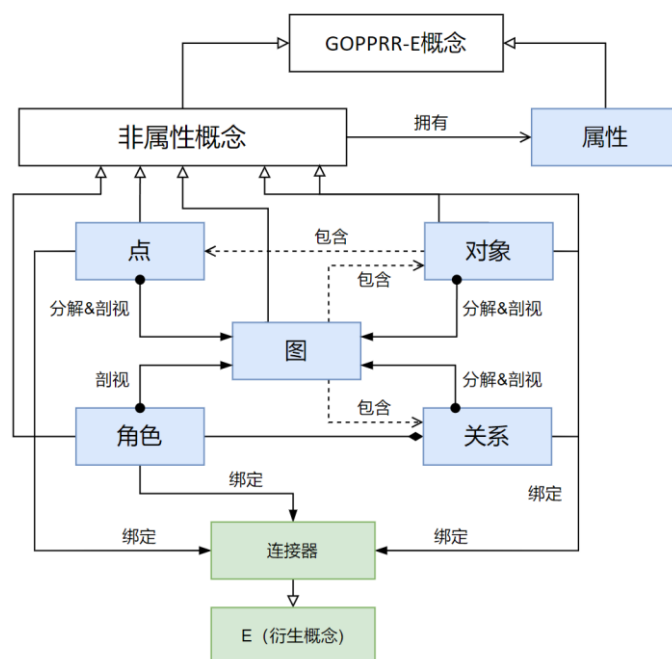


图 2.3 GOPPRR-E 元元模型及其之间的关系

- 图：表示模型的某一类视图，是对象、关系的集合。图可以拥有属性。
- 对象：对象是对图中一类实体（例如，使用 SysML 建模语言描述的一个电池

模块即为一个实体)的抽象描述,可以单独存在。对象可以包含点,且可以拥有若干个属性。

- 点:描述对象中与关系(或角色)连接的端口,点不能够单独存在,必须通过依附在对象的上形式存在。
- 关系:是对象之间的关联关系的抽象描述,通过角色与对象或者对象上拥有的点相连。关系不能单独存在,其两端必须与对象或者对象上的点相连才有意义。
- 角色:描述对象或对象上的点如何与关系相连接。角色依附在关系的两端,不能脱离关系单独存在。角色可以用于表示方向,例如始端和终端。
- 属性:用于描述其他五个元元模型(图、对象、角色、关系和点)的特性。属性不能单独存在,此外,属性具有数据类型,例如字符串型、布尔型、整数型、浮点型等,不同的数据类型用于描述不同方面的特性。

GOPRR 元元模型作为构建领域建模语言的基础,也遵循一定的语法以描述六种元素之间的关联规则。在 GOPRR-E 元建模框架中,这些关联规则是通过扩充的衍生概念描述的,E(Extension)即扩充的衍生概念,主要包括:

(1) 包含:描述元元模型之间的存在的包含关系,图可以包含对象、关系,对象可以包含点。

(2) 属性所属:描述图、对象、点、关系、角色五种元素和属性之间的拥有关系。

(3) 剖视:描述一个对象(或关系、点、角色)元模型的实例可以从不同视角映射为多个图实例,表示图实例是对对象(或关系、点、角色)实例某一视角剖面的详细描述。

(4) 分解:描述一个对象(或关系、点)元模型的实例可以分解为一个图实例。表示图实例是对对象(或关系、点)实例内部组成的详细化描述。

(5) 绑定:描述对象、点、角色、关系元元模型之间存在的约束方式,通过绑定可以定义元建模语言中元素之间的连接规则。例如,描述何种类型对象实例或点实例可以与何种类型角色实例进行绑定,何种关系实例两端能连接何种类型的角色实例。

(6) 跨模型关联:描述属于两个不同图实例的元素之间的关联关系,元素可以是对象、点、角色实例。

2.2.3 基于 GOPPRR-E 元建模方法的系统架构模型构建过程

GOPPRR-E 元建模方法以统一的底层数据描述多领域建模语言的元模型和模型，实现了多领域架构模型的统一表达。使用 GOPPRR-E 元建模方法构建系统架构模型的过程可以分为元模型构建阶段和模型构建阶段。如图 2.4 所示，元模型构建阶段的步骤包括：（1）确定元模型：建模工程师根据所建模目标对象所涉及的领域，总结领域相关的知识、标准规范以及约束，然后抽象分析得出所有类型的元模型。确定元模型具有两种方式：依据成熟建模语言构建元模型和特定域建模。例如 SysML 中的需求即为一种元模型。（2）建立所确定的元模型与 GOPPRR 六种元素之间的映射关系，即确定使用 GOPPRR 中的哪种元元模型来表示所确定的元模型。例如可以使用对象元元模型进行实例化来表示 SysML 中的需求图元模型。（3）基于 GOPPRR 六种元元模型构建元模型。添加领域建模语言元模型所包含的语法和语义，构建 GOPPRR 表征的领域建模语言元模型，并构建这些元模型之间的关联约束。例如对象元元模型添加“需求”的语义、及其拥有的属性和其图形的具体形状，则变成一种格式良好的需求对象元模型。

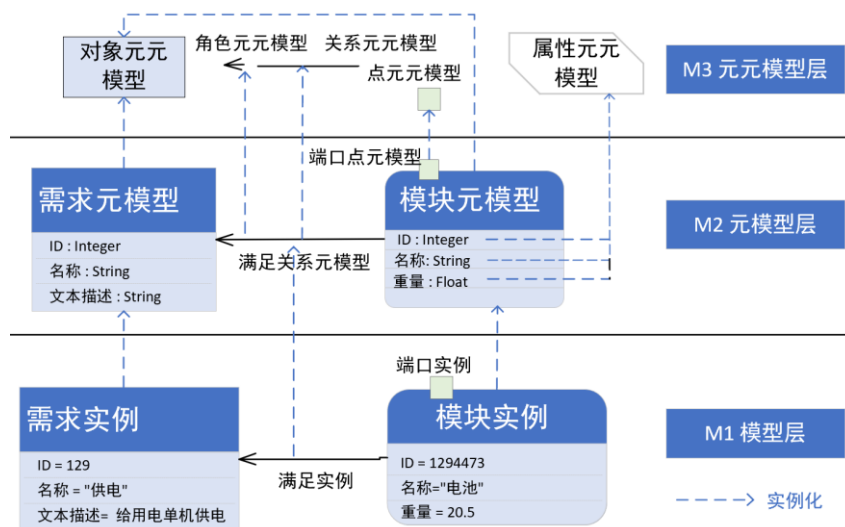


图 2.4 基于 GOPPRR-E 元建模方法的模型构建流程

复杂系统架构模型是由若干个具体的模型构成的。在模型构建阶段，从元建模的角度来看，一个具体的模型由元模型实例化而来。如图 2.4 所示，由需求对象元模型实例化为一个具体的需求实例，其 id 为 129，名称为供电，用于描述供配电系统的需求视角。从建模语言语法语义的角度来看，建模语言的抽象语法描述了元模型及其之间的正确关系以及其他的约束，因此模型的构建需要符合建模语言的抽象语法。从建模流程来看，复杂系

统架构模型包含系统开发过程所构建的所有具体模型，以及各个阶段模型之间的逻辑和时序关系，整个过程可能涉及多种学科和领域。因此，基于 GOPPRR-E 元建模方法，能够有效地支持多领域系统架构模型的统一构建。

2.3 系统架构模型正确性检查相关理论

在 MBSE 中，系统架构模型是最早期设计决策的体现，工程师以系统架构模型为核心指导后续的设计研发过程。目前，系统架构模型的设计过程仍是一个人工手动过程，导致多层次、多视角的模型之间存在不规范、数据不一致等错误。因此，对系统架构模型进行正确性检查具有必要性。为了提出系统架构模型正确性检查框架，本节将明确模型正确性检查相关理论，包括模型正确性检查的定义、方式和语言。

2.3.1 模型正确性检查的定义

在系统设计中，验证和确认是用于判断系统是否满足要求和规范、并实现其预期目的两种手段。其中，验证旨在检查系统是否符合一组设计规范，而确认旨在确保产生满足用户使用需要的系统^[58]。在 MBSE 中，模型正确性检查是一种将模型中的概念与（真实）系统的给定设计规范进行比较的手段，与验证的目的相同。因此，**模型正确性检查属于一种验证手段**。

复杂系统具有静态特性和动态特性，动态特性与系统的行为和状态相关，不同的状态下，系统需要满足的要求具有差别。本文的模型正确性检查面向是系统的静态特性，即不随时间和系统状态改变的特性。如式 2.1 所示，将模型的正确性检查定义为：给定的有限状态系统模型 M ，在任意时刻，判断 M 是否满足所有给定的静态特性 P_s 集的集合 SP ，若不满足，则认为模型是错误的。其中，**给定的静态特性 P_s 使用约束规则进行描述，约束规则由约束上下文和约束表达式组成**。约束上下文用于定义需要检查的模型元素组合（称为 Context Element, CE），约束表达式用于描述对 CE 施加的具体约束。

$$\forall t (if (\exists P_s, M. satifiy(P_s) == false) then let(M.correct = false) \quad (2.1)$$

以图 2.5 为例，系统需要满足的静态特性 P_s 为：“两个相连的接口之间是兼容的”。其中，“两个相连的接口”即为约束上下文，“是兼容的”则为约束表达式。图 2.5 使用 SysML 内部模块图模型描述了两个子系统之间的接口关系，可以发现 portA 的类型

是 String，而 portB 的类型是 Integer。这两个子系统通过具有不同类型的接口进行通信，这可能会导致严重错误。而通过模型正确性检查可基于 **Ps** 快速发现该模块中的错误，避免后续研发团队基于该错误的模型设计出难以集成的两个子系统。

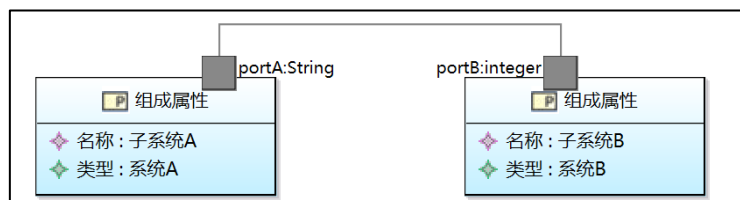


图 2.5 子系统 A 和 B 的内部模块图

2.3.2 模型正确性检查方式

目前的模型正确性检查方式主要分为三种类型：（1）基于逻辑推理和定理证明；（2）基于状态迁移的自动验证。（3）基于规则或模式。

基于逻辑推理与定理证明的核心思想是使用逻辑和推理规则对系统及其给定的规范进行形式化描述，构建逻辑模型，然后从公理、定理和推理规则出发，分析逻辑模型是否满足系统给定规范。该方法的优点是能够保证检查结果是逻辑上完全正确的，可以验证几乎所有的系统特性，但工程师需要花费精力将图形化模型转换为形式化语言描述的模型，且基于数学和逻辑的形式化技术对建模工程师来说较难理解^[59]。

基于状态迁移的自动验证也可以叫做模型检验，是一种用于验证有限状态系统是否满足使用时序逻辑表达的规范的技术。模型检验是一个完全自动化的方法，它将系统需要满足的规范使用时序逻辑进行表达，使用有限状态机表述系统及其行为模型。然后通过程序自动地遍历每一个状态，检查每一个状态下是否违反系统规范^[60]。模型检验方法的优点是自动化程度高，缺点是当系统状态过多时，会出现状态爆炸问题。

基于规则和模式的方法通过半形式化的语言定义模型必须满足的充分条件的规则或模式，通过对模型进行查询来发现模型中违反规则的元素^[61]。此种方法的优点是半形式化的规则或模式易于理解，便于建模工程师使用。它的劣势在于，规则和模式通常是由领域建模专家根据知识进行定义的，若规则定义不完整，则模型不正确性的识别也不完整^[62]。因此，此种检查方法是一个迭代过程，可以进行规则的修改或增添。

由于模型的正确性检查面向系统的静态特性，与系统所处的状态无关，且基于规则或模式的方法学习成本较低、可操作性较强。因此，本文以基于规则或模式的方式

为基础，结合逻辑推理实现模型的正确性检查。

2.3.3 模型正确性检查语言

模型正确性检查语言提供定义良好的语法和语义来形式化或半形式化地描述模型需要满足的约束规则，以根据约束规则执行正确性检查。在基于逻辑推理和定理证明中的检查手段中，以逻辑语言为检查语言，例如由一阶逻辑构成的一阶语言。基于状态迁移的自动验证的方法中，不同模型检验工具中具有不同的语言，例如 SPIN 工具中的 Promela。在基于规则或者模式的方法中，模型正确性检查语言本质上是一种半形式化的约束语言。此种语言可以与模型元素的元模型描述相融合，称为内嵌式；也可以在元模型外部进行约束的描述，称为外部集成方式^[63]。内嵌式语言直接在元模型的定义中描述该元模型的实例元素需要满足的约束规则，相比外部集成方式，其描述规范更为简单，但描述的约束仅限当前元模型，无法描述涉及多个元模型的约束。

由于本文以基于规则或模式的方式实现模型正确性检查，且内嵌式的语言与元模型紧密结合、易于理解，故将采用内嵌式的风格设计并实现模型正确性检查语言。

2.4 多领域系统架构模型正确性检查研究框架

在上文基于 GOPPRR-E 元建模的系统架构模型统一构建方法和模型正确性检查相关理论的基础上，提出系统架构模型正确性检查研究框架。如图 2.6 所示，在该框架中涉及到四个关键技术：①GOPPRR-E 元建模方法。②面向模型正确性检查的约束规则体系构建。③基于图模式匹配和一阶逻辑的系统架构模型正确性检查方法。④模型正确性检查语言及其插件设计。

四个关键技术之间的关系如下。首先通过 GOPPRR-E 元建模方法（即关键技术①）实现统一描述涉及多种领域建模语言的系统架构模型，为关键技术③提供检查对象。然后构建面向模型正确性检查的约束规则体系（即关键技术②），指导建模工程师完整地定义约束规则，为关键技术③的实施提供约束规则。在工程运用过程中，关键技术③（基于图模式匹配和一阶逻辑的系统架构模型正确性检查方法）的实施需要语言和支持。因此通过模型正确性检查语言及其插件（即关键技术④）支持约束规则的定义，并基于插件实现关键技术③中基于图模式匹配和一阶逻辑的模型正确性检查功能。

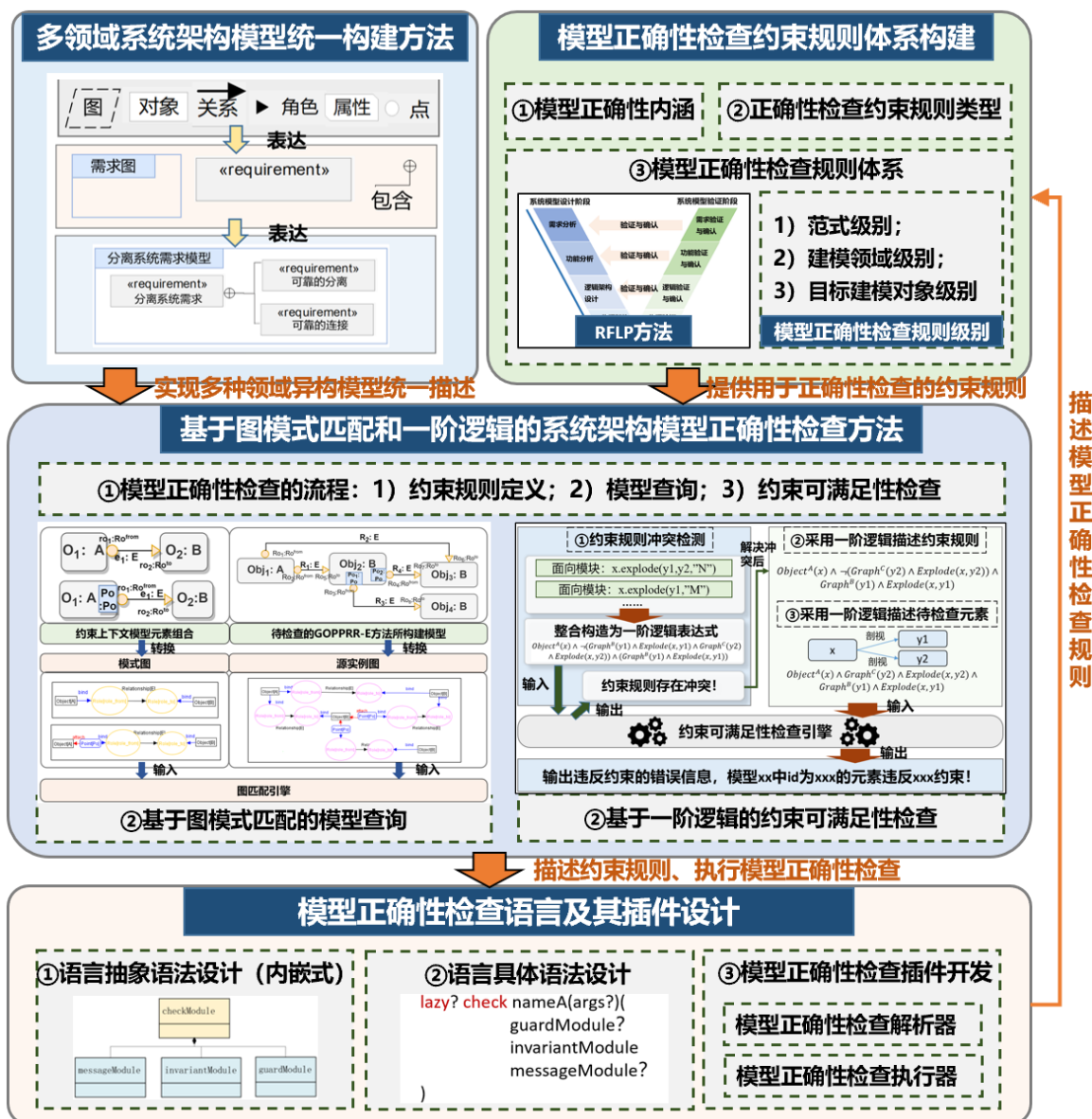


图 2.6 系统架构模型正确性检查研究框架

2.5 本章小结

本章主要介绍了复杂系统架构模型构建方法以及模型正确性检查相关理论。提出基于 GOPPRR-E 的元建模方法实现多领域系统架构模型的统一表达，为模型正确性检查奠定基础。此外介绍了有关模型正确性的定义、检查方式以及检查语言等内容。在此基础上，提出了系统架构模型正确性检查研究框架，为下文的研究提供整体指导。

第3章 面向模型正确性检查的约束规则体系构建

3.1 引言

模型正确性检查的目的是发现不满足给定设计规范的模型，而模型的正确性内涵中蕴含着系统需要满足的给定设计规范。例如在 SysML 中，“模型中包含正确的元素及元素之间的正确关系”中蕴含了数百条模型元素之间的连接关系规范。系统的给定设计规范通过约束规则进行描述，工程师根据建模所有阶段中需要关注的模型正确性内涵，能完整地定义面向正确性检查的约束规则。本章针对目前 MBSE 领域中、缺乏面向模型正确性检查的约束规则体系的问题，首先界定模型正确性的内涵，在此基础上，给出模型正确性检查的约束规则类型。然后结合 MOF 元建模框架和 RFLP 方法形成包含两个阶段（元模型和模型构建阶段）以及三个级别（范式级别-建模领域级别-目标建模对象级别）的约束规则体系。

3.2 模型正确性的内涵界定

模型正确性检查被广泛应用于模型驱动架构领域，是一种提高系统设计质量的有效方法。系统架构模型的质量直接影响对应用领域的理解质量，以及最终基于概念模型研发的最终产品的质量。因此、模型的正确性这一概念与模型的质量息息相关。在不同的研究中，模型的质量类型具有不同的定义。本节将明确模型正确性检查的面向的模型质量类型，确定模型正确性的内涵。

以 Mohagheghi 等人^[37]对模型质量分类的研究为基础，确定模型正确性内涵面向的模型质量特性。该文献中提到的完整性和受限性与建模人员的目的、对模型表达系统的抽象程度和粒度相关。这种目的和粒度难以描述和量化，因此难以使用自动化的手段进行检查，不纳入研究的范畴之内。此外，可理解性与模型的组织、美观程度相关，而可变性的内涵是支持模型的迭代更改，是目前基本建模手段都能够支持的，因此可理解性和可变性也不纳入范畴。故本文面向的模型质量类型包括正确性和一致性。

基于面向的模型质量类型，确定**模型正确性的内涵**：**（1）模型中包含正确的元素及元素之间的正确关系。（2）模型不违反建模规则和惯例；（3）模型正确地描述了所建模的领域；（4）一致性，其含义为模型内部以及模型之间没有矛盾。**其中，从模型描述的视角和建模阶段区分，模型的一致性可以分为水平一致性、垂直一致性和

普通跨图一致性。

- 水平一致性：属于相同抽象级别或建模阶段中，不同视角模型之间的一致性。
- 垂直一致性：处于不同的抽象级别或建模阶段中同一视角模型中的一致性。
- 普通跨图一致性：属于相同建模和同一视角的不同模型之间的一致性。

以软件工程领域和 UML 模型为例说明上文中(1)~(3)模型正确性的含义。UML 的类图可以表达软件工程领域种类与类之间的关系，类之间的关系包括：组合、依赖、继承、实现、聚合、单向关联和双向关联。这些即为**模型中包含正确的元素及元素之间的正确关系**，如果建模者使用“剖视”这描述两个类之间的关系，则是不正确的。在 Berenbach 等人^[39]梳理的 UML 建模惯例中，用例图中的每个参与者都需要与用例关联，此即为**模型不违反建模规则和惯例**。使用 UML 描述 JAVA 语言表示的类时，类无法同时继承多个类，而描述 C 语言表示的类时，类可以同时继承多个类，此即为**模型正确地描述了所建模的领域**。

以系统设计领域的 SysML 模型为例来说明一致性的内涵。在系统设计的逻辑架构分析阶段，具有描述系统行为视角的模型和描述系统结构视角的模型，两者之间的重叠信息需要保持一致，即**水平一致性**。功能分析阶段具有描述系统行为视角的模型，逻辑设计阶段也具有描述系统行为视角的模型，两者之间也存在的重叠信息也需要保持一致，即**垂直一致性**。在系统层功能分析阶段，SysML 用例图、活动图和序列图都可以用于描述系统功能活动，当三个图模型之间涉及到重叠信息时，需要保持一致，即**普通跨图一致性**。

3.3 模型正确性检查的约束规则类型

模型的正确性内涵中蕴含系统给定设计规范，这些给定规范通过约束规则进行描述。本节在明确了正确性内涵的基础上，结合基础的模型概念要素、其他建模语言中常见的模型约束以及 GOPRR-E 方法中的概念，确定面向模型正确性检查的基础约束规则类型。

以 2.2.1.2 节梳理的基础模型概念要素为对象，并结合 Kleppe 等人^[65]以及 Halpin^[66]的研究，梳理建模语言中与基础模型概念要素相关的约束类型，确定面向 GOPRR-E 方法所构建模型的约束规则类型以及该类型对应的模型正确性内涵。在 2.2.1 部分阐述的基础的模型概念要素中，一类概念是非属性概念，用于描述模型的结构，另一类

概念是属性相关概念，用于描述其他非属性概念的特性。因此，模型正确性检查的约束规则类型可按照非属性和属性相关进行划分，可以分为三类：（1）模型结构约束；（2）属性约束；（3）混合约束。具体含义如下文所示。

3.3.1 模型结构约束

模型结构约束用于限制模型元素之间的连接关系（在 GOPPRR 元建模方法中为图、对象、关系、角色、点实例之间的关系），它所能描述的模型正确性内涵包括：模型中包含正确的元素及元素之间的正确关系、模型不违反建模规则和惯例、模型正确地描述了所建模的领域。模型结构约束的具体类型如表 3.1 所示。

表 3.1 模型结构约束

约束名称	GOPPRR 概念	含义
连接性约束	对象、关系、角色、点	约束某一类对象实例可以通过某一类关系实例（及其两端的角色实例）与另一类（或同类）对象实例之间的连接关系。例如 A 类型对象实例可通过 C 类型关系实例与 B 类型对象实例相连。
强聚合约束	对象、关系	约束容器对象实例与被包含对象实例的组合关系，容器对象掌握被包含对象的生灭，例如删除容器对象，被包含对象也被删除。
弱聚合约束	对象、关系	约束容器对象实例与被包含对象实例的聚合关系，容器对象不掌握被包含对象的生灭。
抽象约束	对象	表示对象实例是抽象的。例如 java 中的接口。
继承约束	对象、关系	约束对象实例之间的继承关系，继承方需要重用被继承方的属性、行为和端口等。继承约束具有传递性，例如 A 继承 B，B 继承 C，则 A 也继承了 C。
对象多重性约束	对象、关系、角色	约束某类对象实例通过某类型关系实例，可以与另一个类对象实例相连的数量。包括（1）等值多重性；（2）最小多重性；（3）最大多重性；（4）范围多重性。例如最多有三个 A 类型对象实例通过 C 类型关系实例与 B 类型对象实例相连，是一种最大多重性。
点多重性约束	对象、点	约束某类对象实例可以包含某类点实例的数量。包括：（1）等值多重性；（2）最小多重性；（3）最大多重性；（4）范围多重性。

续表 3.1

约束名称	GOPPRR 概念	含义
点强制性约束	对象、点	约束某类对象实例必须包含至少一个某类型的点实例。例如 A 类型对象实例必须包含一个 B 类型的点实例，该点实例无法删除。
结构强制性约束	对象、关系、角色	约束某类对象实例必须（或必须不能）通过某类关系实例连接另一类对象实例。例如某个 A 类型对象实例不能单独存在，必须通过 C 类型关系实例与 B 类型对象实例相连。
结构唯一性约束	对象、关系、角色	是一种特殊的多重性约束，约束某类对象实例通过某类关系实例与另一类对象实例之间连接的一对一、多对一以及一对多关系。例如一个 A 类型对象实例只能通过 C 类型关系实例与一个 B 类型实例连接，两者之间是一对一关系。
分解约束	对象、关系、角色、点、图、	约束某类对象/关系/点/角色实例可以分解至何类图实例，表示该图实例是对象/关系/点/角色实例描述内容的细化。
分解强制性约束	对象、关系、角色、点、图	约束某类对象/关系/点/角色实例必须（或必须不能）分解至何类图实例。
剖视约束	对象、关系、角色、点、图	约束某类对象/关系/点/角色实例可以剖视至何类图实例，表示该图实例用于描述对象/关系/点/角色实例的某一特定方面。
剖视强制性约束	对象、关系、角色、点、图	约束某类对象/关系/点/角色实例必须（或必须不能）剖视至何类图实例。
集比较约束	对象、关系、角色	约束同一对象类型的实例集在不同结构或不同类型图实例中的比较关系。分为：（1）子集约束，即对象集 A 是对象集 B 的子集；（2）等价约束，即对象集 A 等价于对象集 B；（3）排斥约束，即对象集 A 没有任何与对象集 B 重合的元素。
环约束	对象、关系、角色	是特殊的连接性约束。包括：（1）自反性：表示为某一类对象实例可以通过某一类关系实例连接自身。（2）对称性：表示为从一个对象实例到另一个对象实例的关系实例，总是伴随着另一个方向相反的同类型关系实例。（3）传递性：描述一种“如果 A->B，B->C；则 A->C”的传递关系。（4）不成环：表示由某一类型关系实例连接对象实例组成的模型结构中不能成环。

3.3.2 属性约束

属性约束用于限制模型元素具有的属性的值、类型、多重性等内容。属性通常用于描述模型所表示的系统中某个事物或实体的具体性质，例如重量、体积等，属性的值、类型等与具体的建模领域和目标系统有关。在建模语言中，对每个实体概念及其关系拥有的属性做出了规定。在建模规则和惯例中，也会对属性做出规定，例如同一个元素无法具有两个相同名称的属性。因此，属性约束所能描述的模型正确性内涵包括：模型中包含正确的元素及元素之间的正确关系、模型不违反建模规则和惯例、模型正确地描述了所建模的领域。属性约束的具体类型如表 3.2 所示。

表 3.2 属性约束

约束名称	GOPRR 概念	含义
属性引用约束	属性、对象、图、关系、点、角色	约束某类对象/关系/图/点/角色实例具有何种类型的属性实例。例如规定 A 类型对象实例可以具有 D 类型和 E 类型属性实例。
属性值约束	属性、对象、图、关系、点、角色	是针对某类对象/关系/图/点/角色实例具有的属性实例的取值范围约束，包括：（1）等值约束：规定属性的值必须等于某个特定的数；（2）范围约束：规定属性的值的取值范围。例如规定 A 类型对象实例中具有 E 类型属性实例的值必须介于 0~3 之间。
属性多重性约束	属性、对象、图、关系、点、角色	（3）类型约束，规定属性的值必须是何种类型。 约束某类对象/关系/图/点/角色实例具有的同一类型属性实例的数量。包括：（1）等值多重性；（2）最小多重性；（3）最大多重性；（4）范围多重性。例如 A 类型对象实例只能拥有一个 ID 类型属性实例，这是一种等值多重性。
属性值唯一性约束	属性、对象、图、关系、点、角色	约束某类对象/关系/图/点/角色实例具有的属性实例值的唯一性。例如所有 ID 类型属性实例的值都是唯一的。
属性值比较约束	属性、对象、图、关系、点、角色	约束某类对象/关系/图/点/角色实例中，具有的不同属性实例值之间的比较关系。例如 A 类型的对象实例具有一个 D 类型的属性实例 d 和一个 E 类型的属性实例 e，规定 d 的值小于等于 e 的值。
属性强制约束	属性、对象、图、关系、点、角色	约束某类对象/关系/图/点/角色实例必须具有某种属性类型，无法删除。

3.3.3 混合约束

混合约束描述一组模型结构中，不同元素拥有的属性之间的约束。例如 A 类型对象实例 a 通过 C 类型关系实例与 B 类型对象实例 b 相连，这是一组模型结构，约束 a 的属性实例 d₁ 与 b 的属性实例 d₂ 的值相等，即为一个混合约束。不同元素属性之间的关系与建模领域有关，此外，一致性也发生在不同模型元素之间。因此、混合约束能够描述的模型正确性内涵包括：模型正确地描述了所建模的领域、一致性。混合约束可以分为两种类型：

(1) 模型内部混合约束：针一个模型内部的模型结构中不同元素属性之间的约束。关注的正确性内涵为：模型正确地描述了所建模的领域。

(2) 模型之间混合约束：针对不同的模型之间的模型元素构成的模型结构中、不同元素属性之间的约束。即该模型结构中的元素一部分位于模型 A，另一部分位于模型 B 中。关注的正确性内涵为：一致性。

3.4 面向模型正确性检查的约束规则体系

面向模型正确性检查的约束规则体系是一种指导工程师根据建模每一阶段需要关注的模型正确性内涵，基于正确性内涵中所蕴含的给定设计规范，定义约束规则的方法。在上文明确了模型正确性的内涵和约束规则类型的基础上，本小节从元建模阶段开始，研究系统架构模型的构建过程、以及过程中每个阶段所关注的模型正确性内涵，形成面向模型正确性检查的约束体系。

3.4.1 RFLP 建模方法

在 MOF 框架中，建模活动的开端是构建 M2 层的元模型，其过程在 2.2.3 节中已经阐述。然后，建模工程师基于所构建的元模型，根据所设计系统的特点来构建 M3 层的模型。构建模型的过程需要建模方法的指导，建模方法中规定的模型之间的关联关系也是一种系统给定设计规范，将影响约束规则的构建，因此需要确定所使用的建模方法。1.2.1.2 小节部分阐述了目前国内外主流的建模方法：Harmony SE、OOSEM 和 MagicGrid。如表 3.3 所示，通过对此三种方法的具体建模过程进行分析，可以发现将它们都遵循一套需求-功能-逻辑-物理（RFLP）的分析过程。以 MagicGrid 建模方法为例，其问题域利益相关者需求分析对应 RFLP 中的需求分析、问题域用例功能分析

对应 RFLP 中的功能分析、问题域逻辑子系统设计对应 RFLP 中的逻辑设计、解决域物理子系统实现对应 RFLP 中的物理设计。

表 3.3 RFLP 方法与 Harmony SE、OOSEM 和 MagicGrid 方法的过程映射

RFLP	Harmony SE	OOSEM	MagicGrid
R: 需求分析	需求分析	利益相关者需求分析、系统需求定义	问题域利益相关者需求分析
F: 功能分析	功能分析	定义逻辑体系结构	问题域用例、功能分析
L: 逻辑设计	设计综合: 架构分析与架构设计		问题域逻辑子系统设计
P: 物理设计		系统物理架构合成	解决域物理子系统实现

RFLP 是工程领域的正向设计方法之一，可以有效地为基于 MBSE 方法的系统设计和项目开发提供所需的整体支持。如图 3.1 所示，在 MBSE 中，RFLP 方法可以为 V 模型中的系统需求分析与架构设计部分提供流程指导。V 模型是系统开发生命周期的图形化表示，它描述了在产品开发过程中要执行的活动和产生的结果。V 模型将系统开发生命周期分为（1）系统定义域开发阶段，主要步骤为：系统运行概念分析、需求分析与架构设计和详细设计；（2）系统实现阶段；（3）系统集成与验证确认阶段。RFLP 方法应用于 V 模型的需求分析与架构设计阶段，将该阶段的流程进行细分为需求分析（R）-功能分析（F）-逻辑设计（L）-物理设计（P）四个方面，为系统架构模型的构建和验证提供了流程指导。

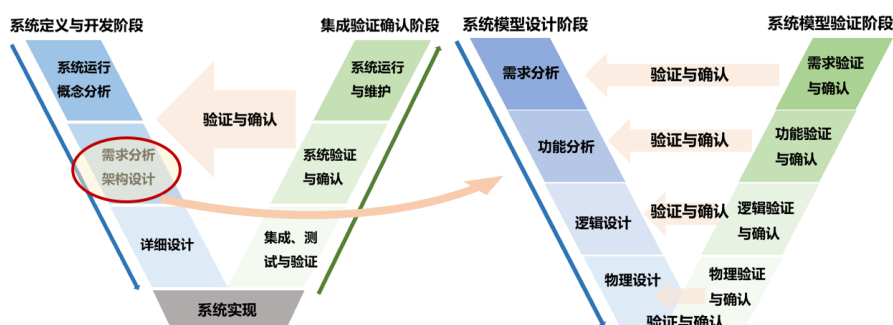


图 3.1 RFLP 方法应用于 V 模型的需求分析与架构设计阶段

如图 3.2 所示，在进行系统设计时，RFLP 方法遵循自顶向下的分解和分析流程，即按照系统-分系统-单机的多层级分析分解过程，并将需求分析-功能分析-逻辑设计-

物理设计的过程融入多层次系统分解过程中，具体步骤如下：

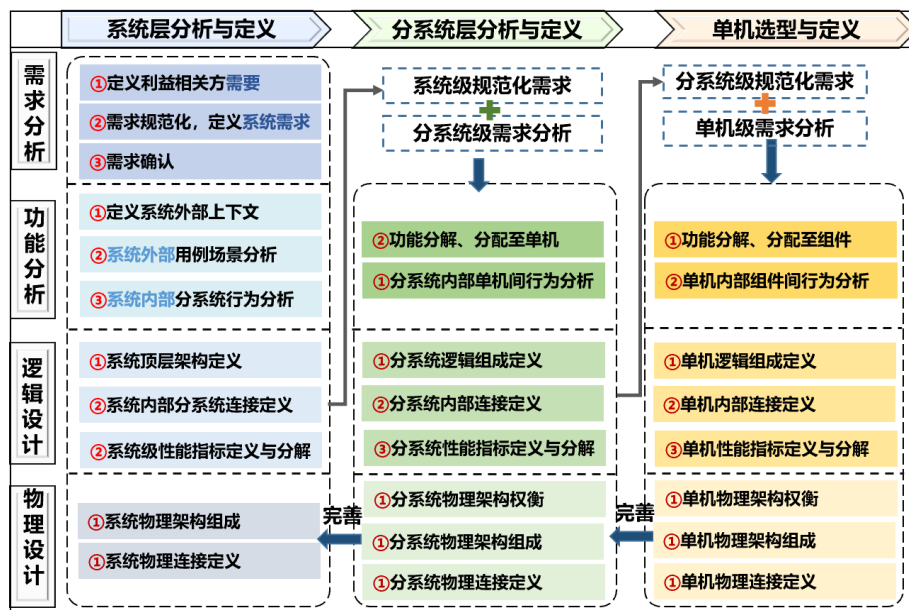


图 3.2 RFLP 方法建模具体流程

(1) 需求分析：

该阶段主要目标是将利益相关方需要转换为完整、正确、规范的系统需求。系统层的需求分析主要步骤包括：（1）定义利益相关方需要。系统设计的起源是利益相关方需要，需求分析阶段的主要输入是《系统设计任务书》，其中包含利益相关方需要。首先根据系统设计任务书，使用需求模型构建系统利益相关方需要，包括系统功能、六性要求、流程管理需求及其他设计约束要求。为每一个需求模块定义其需求 ID 和文本描述。（2）需求规范化：根据需求工程标准，提取利益相关方需要，并将需求规范化，确定需求类型、依据、验证方法等内容，形成初步的系统需求。（3）需求确认：明确初步系统需求之后，系统设计师需要进行需求确认工作。从利益相关方、功能与接口、系统性能指标三个方向进行需求确认。首先建立所有利益相关方与系统需求之间的追溯关系，确认系统需求捕获到了所有利益相关方对系统的需求。然后采用功能分析阶段的步骤，确定系统顶层功能需求，并得出系统与外部其他系统之间的外部接口。然后进行系统性能指标确认，从任务需求和设计约束中得出系统顶层的性能指标。

分系统层的需求分析和单机层的需求分析与系统层的过程类似，区别是分系统层输入的是系统层产生的规范化需求，单机层输入的是分系统层的规范化需求。在单机层，若所需单机是标准化成熟单机，则可根据分系统层相关需求直接选型，无需进行

单机内部结构的需求分析、结构分析和行为分析；若所需单机是需要新研制的，则需要进一步分析内部结构。

(2) 功能分析：

该阶段的主要目的是定义并分解系统的功能，即为满足利益相关者需要，系统、分系统、组件需要具有哪些功能。在系统层功能分析时，包含以下步骤：（1）定义系统外部上下文：即与系统具有交互活动的外部其他系统或用户。（2）系统外部用例场景分析：首先识别系统边界和用例，定义外部系统与系统用例之间的关联关系。针对每个用例，构建场景行为模型，分析参与用例的外部系统和所设计系统之间的行为交互。（3）系统内部分系统行为分析：此步骤的前提是，逻辑设计部分根据系统外部用例场景分析后，进行功能分配，得出系统顶层架构（包含若干分系统）。然后分析顶层架构中分系统之间的交互行为活动。

在分系统层和单机层的功能分析时，主要包括两个步骤：（1）功能分解：在分系统级别，将分系统级的功能进行分解，分配至系统单机；在单机级别，将单机级的功能进行分解，分配至组件。（2）行为分析：在分系统级别，分析单机之间的行为交互活动以及分系统本身的状态模式；在单机级别，分析组件之间的行为交互活动以及单机本身的状态模式。

(3) 逻辑设计：

该阶段的目的是确定组成系统的逻辑组件以及逻辑组件的之间的连接交互关系（即逻辑架构），主要包括三个步骤：（1）定义系统逻辑组成：在系统层，系统逻辑组成即系统的顶层架构，描述系统由哪些分系统组成以及包含的系统外部接口。此时的逻辑架构是简单粗略的。细化的逻辑架构需要由分系统和单机级别的逻辑架构设计得出。（2）定义内部连接关系：在系统层，内部连接关系描述了外部其他系统通过接口与若干个分系统之间的连接交互。在分系统层，则定义分系统内部的单机之间通过接口的连接交互。在单机层，则定义单机内部的组件之间通过接口的连接交互。（3）性能指标定义与分解。系统层的性能指标来自《系统设计任务书》，将其中的设计约束和性能要求提炼形成系统性能指标，并将指标分解分配至各个分系统。分系统层的性能指标来源于系统层，并将指标分解、分配至单机。单机层的性能指标来源于分系统层，参考组件的技术度量特性，将指标分解分配至组件。

(4) 物理设计：

该阶段的目的是确定系统的物理架构。物理架构定义了物理组成设备（及其接口）的参数及其之间的关系，其目的是描述具体的、可实现的解决方案。在对逻辑架构进行开发以后，物理架构为系统的研发提供能够支持功能分析和逻辑架构定义结果的具体方案，并且满足从非功能需求分析中推断出的预期设计属性和技术指标。物理架构的定义是从单机-分系统-系统逐层确定和完善的，主要包括的步骤为：（1）物理架构权衡：首先搜索能够满足逻辑架构和接口的系统物理单元，根据逻辑层的性能指标，选择较优的一种或多种物理单元，形成一套或者多套物理架构组成方案。针对每套物理架构组成方案，定义每个物理单元和接口的物理参数，然后根据逻辑层的性能指标进行计算权衡，确定最优的物理架构组成方案。（2）物理架构组成定义：根据最优物理架构组成方案，定义涵盖系统-分系统-单机级别的物理架构组成。（3）物理架构连接定义：描述物理架构组成中所有物理单元之间的交互连接关系。

3.4.2 约束规则体系构建

在 MOF 元建模框架和 RFLP 建模方法的基础上，构建面向模型正确性检查的约束规则体系。首先根据建模过程，将正确性检查约束规则的定义过程分为元建模构建阶段和模型构建阶段。元建模构建阶段定义范式级别和建模领域级别的约束规则，而模型构建阶段定义目标建模对象级别的约束规则。

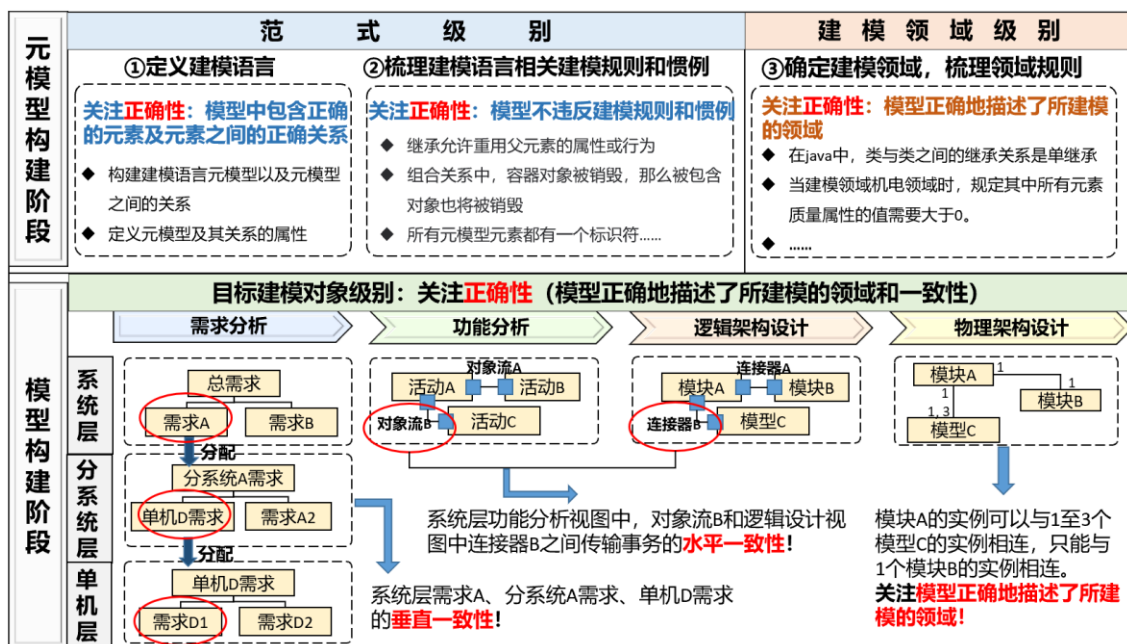


图 3.3 模型正确性检查约束规则体系

(1) 范式级别

范式级别关注的模型正确性内涵为：模型中包含正确的元素及元素之间的正确关系（内涵①）、模型不违反建模规则和惯例（内涵②）。首先在定义建模语言阶段关注内涵①，以建模语言中可以具有的元模型以及元模型之间的关系为约束规则，例如 UML 类图中的元素只能为类和接口、类与接口之间的关系只能为实现等。然后，再梳理与建模语言相关的建模规则和惯例，基于规则和惯例构建约束规则，例如一个模块中没有名称相同的两个属性、继承关系允许子类元素重用父类元素的属性和行为等。

范式级别要求建模工程师按照领域建模语言的指导手册，使用 GOPRR-E 元建模方法构建建模语言中的所有元模型及其之间的关系。同时，梳理与所使用的建模语言相关的建模规则和惯例，并使用约束规则进行表达。

(2) 建模领域级别

建模领域级别关注的模型正确性内涵是：模型正确地描述了所建模的领域。因此，建模领域级别定义的约束规则与具体建模领域的特性有关。例如建模领域为 JAVA 时，类被定义为一种元模型，规定类只能继承一个类。当建模领域为机电领域时，规定其中所有元素质量属性的值需要大于 0。

建模领域级别要求建模工程师对领域的知识具有足够深刻的了解，梳理领域中的一些约定俗成的、必须符合的规则和标准，并使用约束规则进行表达。这个过程通常需要领域专家的协助。

(3) 目标建模对象级别

目标建模对象级别关注的模型正确性内涵为：模型正确地描述了所建模的领域以及一致性。该级别所定义的约束规则与具体建模对象中存在的约束有关，因此结合 RFLP 方法来指导该级别约束规则的定义，从四个方面进行约束规则的构建：

(1) 确定需求-功能-逻辑-物理四种视图相关模型中，与“模型正确地描述了所建模的领域”相关的约束。以 SysML 语言为例，模块定义图（Block Definition Diagram, BDD）是用于描述系统的逻辑和物理组成，模块是 BDD 中主要的元模型。在进行卫星设计时，规定通信与数据通信子系统模块的有效实例必须只包含两个计算机模块。上述例子即与领域的建模对象本身相关的约束。

(2) 确定同一阶段、同一类型视角模型之间的普通跨图一致性约束。在 RFLP 建模方法中，同一视角的模型可能由建模语言的若干个图共同进行描述。以 SysML 为

例，描述系统结构的图有模块定义图、内部模块图、参数图等，描述行为的图有状态机图、序列图、用例图和活动图。这些在建模同一阶段描述系统同一视角的图模型之间可能存在重叠的信息。例如活动图模型 A 用于描述用例图中一个具体用例 B 的功能场景，那么活动图中的活动的参与者应该与用例 B 关联的参与者一致。

(3) 确定同一阶段（例如都处于系统层设计阶段）中需求-功能-逻辑-物理视图模型的垂直一致性约束。在 RFLP 方法中，系统的功能和行为分配给了对应的系统逻辑组成。以 SysML 语言为例，活动图常用于描述系统的功能和行为，活动图中的主要元素是动作，不同的动作之间可以通过对象流来连接，用于表示动作之间传输的事务或信息。不同的动作将分配至不同的逻辑单元。逻辑单元之间的交互连接关系使用内部模块图描述，通过连接器连接逻辑单元上的接口来表示逻辑单元之间传输的事务或信息。此时，具有分配关系的动作之间传输的信息与逻辑组件之间传输的信息应该是一致的。此示例说明了功能视图和逻辑视图之间的垂直一致性约束。

(4) 确定不同阶段（例如系统层阶段，分系统层阶段）中同一类型视图（例如均为功能视图）中模型的垂直一致性约束。在 RFLP 建模方法中，系统设计阶段分为系统层，分系统层，单机层。系统需求、功能、逻辑的设计分解过程自顶向下，反过来物理设计自底向上进行综合完善，因此不同层级的同一类型视图中存在重叠元素。以 SysML 语言为例，在系统层的逻辑设计时，确定了系统的外部接口，外部接口分配在各个逻辑分系统上，使用内部模块图 A 描述系统层通过外部接口的连接交互关系。在分系统层的逻辑设计时，将分系统上的系统外部接口分配至组成分系统的各个单机上，并分析单机之间的内部接口，然后使用内部模块图 B 描述分系统层通过接口的连接交互关系。此时内部模块图 A 的接口和内部模块图 B 中的部分接口应该是一致的。此示例说明了处于系统层设计阶段和分系统层设计阶段逻辑视图中的垂直一致性约束。

3.5 本章小结

本章首先对模型正确性的内涵进行界定，然后结合基础的模型概念要素以及 GOPRR-E 方法中的概念，确定了本文支持的约束规则类型以描述模型正确性内涵。最后构建了面向模型正确性检查的约束规则体系，该体系明确了元建模以及建模过程中每个阶段关注的模型正确性内涵，以根据所关注的正确性内涵完整地定义约束规则，有利于系统架构模型正确性检查实施的落地。

第4章 系统架构模型正确性检查方法

4.1 引言

本章针对现有方法难以统一实现多领域系统架构模型构建和正确性检查的问题，研究面向 GOPPRR-E 方法所构建模型的正确性检查方法。首先研究面向 GOPPRR-E 模型的正确性检查过程，该过程包含模型查询和模型约束可满足性检查两个关键步骤。在此基础上，研究基于图模式匹配实现模型正确性检查过程中的模型查询过程，并采用一阶逻辑对约束规则进行形式化表达，以支持对查询结果进行约束可满足性检查。

4.2 面向 GOPPRR-E 模型的正确性检查流程

本文采用的模型正确性检查方式为基于规则或模式的方式，该方式通过对模型进行查询来发现模型中违反规则的元素。本小节结合基于规则或模式的方法定义面向 GOPPRR-E 方法所构建模型的正确性检查流程。如图 4.1 所示，正确性检查的过程可以分为三个步骤：

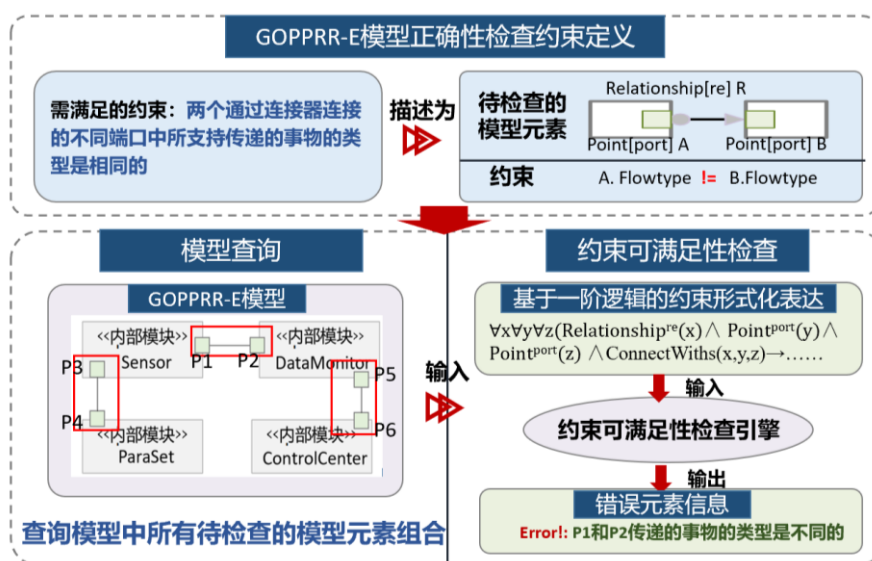


图 4.1 面向 GOPPRR-E 方法所构建模型的正确性检查流程

(1) 模型正确性检查约束规则定义：此部分内容的目的是明确建模过程中需要对哪些模型元素（及其组合）施加何种约束规则，可根据第三章中面向模型正确性检查的约束规则体系进行定义。约束规则中包含约束上下文和约束表达式，约束上下文用于描述约束表达式的约束范围是“哪些类型的模型元素”，不同的约束规则具有不同

的作用范围。

(2) 模型查询：模型查询是在模型中识别并获取指定模型元素的过程。在进行模型正确性检查时，通过模型查询，可以识别所有模型中符合约束上下文中指定类型模型元素（及其组合）的所有模型元素组合，并作为查询结果输入值约束可满足性检查部分。

(3) 约束可满足性检查：该部分将判断模型查询部分得到的查询结果是否满足对应约束规则中的约束表达式，并输出不满足约束表达式的错误信息。

4.3 基于图模式匹配的约束上下文模型查询

在 4.2 节的面向 GOPPRR-E 模型的正确性检查流程中，模型查询过程用于查找模型中所有符合约束上下文中指定类型模型结构的所有待检查模型元素组合，本节研究基于图模式匹配方法实现模型查询。首先需要明确图模式匹配相关理论和概念，然后建立 GOPPRR-E 元建模方法中元模型及实例至图论中概念的映射关系，以实现基于图的 GOPPRR-E 元模型及实例的表达。然后将模型查询过程映射为图模式匹配过程，以利用成熟的图匹配引擎实现模型正确性检查过程中的模型查询。

4.3.1 图模式匹配相关理论与概念

在图论中，图的基本组成是顶点和边，在不同的应用场景具有多种表现形式，例如有向图、属性图、标记图等^[67]。有向图是具有方向性的图，它由一组顶点和一组有方向的边组成，每条有方向的边都连接着一对有序的顶点。标记图是一个有向图，顶点和边都与标签相关联^[68]。标签用于标记节点的分组，多个节点可以有相同的标签，一个节点可以有多个标签，通过标签可为顶点和边赋予类型。属性图中的顶点和边可以拥有属性，属性是一个键值对，每个节点或边可以具有一个或多个属性。

在基于 GOPPRR-E 元建模方法所构建的模型中，每个元素都具有类型，它们的类型即为它们对应的元模型。此外，G（图）O（对象）P（点）R（关系）R（角色）实例可以具有一个或多个属性实例，且对象实例之间的关系是有向的。因此，采用带标记的有向属性图来对 GOPPRR-E 元建模方法所构建的模型进行形式化表达。

4.3.1.1 带标记的有向属性图

一个带标记的有向图 G_L 的定义如式 4.1 所示，其中 NODES 代表由若干个节点

node 组成的有限集, EDGES 代表由若干个边 edge 组成的有限集。source 和 target 是两个映射函数, 用于给每条边分配一个源节点和目标节点。label 代表标签映射, 用于分配标签符号给每个节点和每条边。通过构建类型标签可以为每一个节点和边分配一个类型, 类型标签使用关键词 type 表示, 例如 $\text{type}(\text{edge}_i)$ 、 $\text{type}(\text{node}_i)$ 。

$$G_L = (\text{NODES}, \text{EDGES}, \text{source}, \text{target}, \text{label}) \quad (4.1)$$

属性图在带标记的有向图上增加了属性的概念, 属性可以是不同类型的, 例如可以是一个数字、一个文本、一个表达式或者一个列表。扩展了属性 (attrs) 的标记图 G 可以表示为式 4.2:

$$G = (\text{NODES}, \text{EDGES}, \text{source}, \text{target}, \text{label}, \text{attrs}) \quad (4.2)$$

出于结构化和正确性的目的, 通常在带标记的属性图上强加一种规程 (类似于编程语言的语法) 来规定节点与边之间的连接关系, 这种强加了规程的标记图叫做类型图^[69] (Type Graph)。基于类型图, 可以创建正确的实例图 (Instance Graph)。以下将分别介绍类型图和实例图的定义。

(1) 类型图

如式 4.3 所示, 一个类型图 G_T 是一个五元组, 其中 NType 是类型节点 node_T 的集合, EType 是类型边 edge_T 的集合。类型节点用于描述一类具有相同含义的节点类型。类型节点只允许在类型图中出现, 且仅能出现一次。类型边用于描述一类具有相同含义的边类型, 类型边只允许在类型图中出现。此外, 类型节点和边中允许定义类型属性, 表示此种类型的节点/边具有某种类型的属性, 但不能为属性赋予值。source_T 和 target_T 是两个映射函数, 定义类型边两端可连接的类型节点。TAttrs 表示该类型图可以拥有的属性类型的集合。所有类型图 G_T 的集合为 GType。

$$G_T = (\text{NType}, \text{EType}, \text{source}_T, \text{target}_T, \text{TAttrs}) \quad (4.3)$$

(2) 实例图

如式 4.4 和式 4.5 所示, 实例图 G_I 是类型图的一个具体实例, 由所有实例节点 (node_I) 的集合 NInstance、所有实例边 (edge_I) 的集合 EInstance、source_I 和 target_I 以及所有属性实例 attr_I 的集合 attrs_I 组成。其中, 所有节点的类型 $\text{type}(\text{node}_I) \in \text{NType}$, 所有边的类型 $\text{type}(\text{edge}_I) \in \text{EType}$, 可以为实例节点或边中具有的属性赋予具体的值。所有属性实例的类型 $\text{type}(\text{attr}_I) \in \text{attrs}_T$ 。

$$G_I = \text{instance}(G_T) \quad (4.4)$$

$$G_I = (NInstance, EInstance, source_I, target_I, attr_I) \quad (4.5)$$

4.3.1.2 图模式匹配

在图论中，图模式匹配是一种子图同构问题^[70]，要求在另一个图（源图）中找到一个给定图（即模式图）的图像（即匹配）。通常，图模式匹配严格基于结构相似性进行匹配。然而，由于图也有多种类型，例如上文提到的属性图、标记图等，这些图具有语义信息。所以图模式匹配不仅要考虑结构相似性，还需要考虑语义相似性。从结构和语义两个方面进行分类，图模式匹配可以分为结构匹配和语义匹配^[71]。结构匹配严格根据结构相似性找到匹配，而语义匹配考虑结构相似性的同时还需考虑顶点和边的类型以及它们拥有的属性。本文的模型正确性检查面向的是具有类型和属性的 GOPPRR-E 方法所构建的模型，故使用语义匹配来实现模型查询。

4.3.2 GOPPRR 模型元素至图的映射

为了使用图模式匹配方法实现 GOPPRR-E 方法所构建模型的查询过程，首先构建 GOPPRR-E 元模型至类型图的映射，然后将 GOPPRR-E 模型映射至实例图。

4.3.2.1 GOPPRR-E 元模型至类型图的映射

在元建模方法中，元模型定义了某一模型的规范，即组成模型的元素和元素之间的关系。在图论中，类型图用于对具有类型标签的节点和边之间的连接施加约束。从元模型和类型图的作用来看，两者是一致的。因此，本小节构建 GOPPRR-E 元模型至类型图的映射关系，以使用类型图对 GOPPRR-E 元模型进行表达。

首先对类型图中类型节点和类型边的概念进行扩展以描述 GOPPRR-E 元模型。如图 4.2 所示，将类型节点分为对象类型节点 ($node_T^O$)、点类型节点 ($node_T^P$)、角色类型节点 ($node_T^R$)、分解图类型节点 ($node_T^D$) 以及剖视图类型节点 ($node_T^E$)。将类型边分为关系类型边 ($edge_T^R$)、绑定类型边 ($edge_T^B$)、依附类型边 ($edge_T^A$)、分解类型边 ($edge_T^D$)、剖视类型边 ($edge_T^E$) 以及跨模型关联类型边 ($edge_T^C$)。

- $G_T^G = (graphType, NType, EType, source_T^G, target_T^G, attrs_T^G)$ ，用于描述图元模型， G_T^G 的集合为 $GType$ 。其中 $graphType$ 用于表示图元模型的具体类型，例如模块定义图元模型。 $NType$ 由所有 $node_T^O$ 的集合 ($NType^O$)、所有 $node_T^P$ 的集合 ($NType^P$)、所有 $node_T^R$ 的集合 ($NType^R$)、所有 $node_T^D$ 的集合 ($NType^D$)

以及所有 $node_T^E$ 的集合($NType^E$)构成。 $EType$ 由所有 $edge_T^R$ 的集合($EType^R$)、 $edge_T^B$ 、 $edge_T^A$ 、 $edge_T^C$ 、 $edge_T^D$ 以及 $edge_T^E$ 构成。 $source_T^C$ 和 $target_T^C$ 用于定义关系类型边两端可以连接的角色类型节点。 $attrs_T^C$ 用于描述图元模型具有的所有属性元模型。

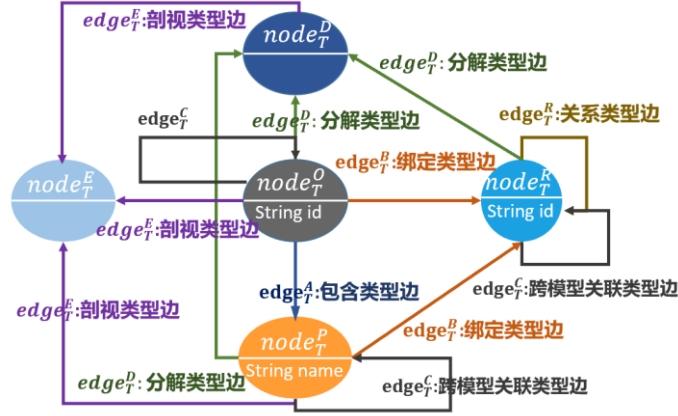


图 4.2 基于图表达 GOPPRR-E 元模型

- $node_T^O = (objectType, attrs_T^O)$, 用于描述对象元模型。其中 $objectType$ 表示对象元模型的具体类型, 例如模块对象元模型。 $attrs_T^O$ 用于描述对象元模型具有的所有属性元模型。
- $node_T^P = (pointType, attrs_T^P)$, 用于描述点元模型。其中 $pointType$ 表示点元模型的具体类型, $attrs_T^P$ 用于描述点元模型具有的所有属性元模型。
- $node_T^R = (roleType, attrs_T^{Role})$, 用于描述角色元模型。其中 $roleType$ 表示角色元模型的具体类型, $attrs_T^{Role}$ 用于描述角色元模型具有的所有属性元模型。
- $node_T^D = (dGraphType)$: 描述与对象/关系/角色/点元模型具有分解关系的图元模型, $dGraphType$ 代表分解的图元模型的具体类型。
- $node_T^E = (eGraphType)$: 描述与对象/关系/角色/点元模型具有剖视关系的图元模型, $eGraphType$ 代表剖视的图元模型的具体类型。
- $edge_T^R = (relaType, attrs_T^{Rela})$, 用于描述关系元模型。其中 $relaType$ 表示关系元模型的具体类型, $attrs_T^{Rela}$ 用于描述关系元模型具有的所有属性元模型。
- $edge_T^A$: 用于描述对象元模型与点元模型之间的包含关系。
- $edge_T^B$: 用于描述对象元模型(或点元模型)与角色元模型之间的绑定关系。
- $edge_T^C$: 用于描述对象/点/角色元模型的跨模型关联关系。

- $edge_T^D$: 用于描述描述对象/关系/角色/点元模型至图元模型的分解关系。
- $edge_T^E$: 用于描述描述对象/关系/角色/点元模型至图元模型的剖视关系。
- 属性元模型: $attr_T = (propertyType)$ 。 $propertyType$ 表示属性元模型的具体类型, 例如名称类型。属性元模型集合为 $attrs_T$ 。

4.3.2.2 GOPPRR-E 模型至实例图的映射

在元建模中, 模型是元模型的实例, 而在图论中, 实例图可以看作是类型图的实例。本节建立 GOPPRR-E 模型实例至实例图中要素的映射关系, 以使用实例图形式化表达 GOPPRR-E 模型实例。

首先对实例图中实例节点和实例边的概念进行扩展。将实例节点分为对象实例节点($node_I^O$)、点实例节点($node_I^P$)、角色实例节点($node_I^R$)、分解图实例节点($node_I^D$)以及剖视图实例节点($node_I^E$)。将实例边分为关系实例边($edge_I^R$)、绑定实例边($edge_I^B$)、依附实例边($edge_I^A$)、分解实例边($edge_I^D$)、剖视实例边($edge_I^E$)和跨模型实例边($edge_I^C$)。基于扩展的概念描述 GOPPRR-E 实例, 如下所示:

- $G_I^G = instance(G_T^G)$, 也可以记作: $G_I^G = (gtype, NInstance, EInstance, source_I, target_I, attrs_I^G)$, 用于描述图实例。其中 $gtype \in GType$ 。 $NInstance$ 是所有 $node_I^O$ 、 $node_I^P$ 、 $node_I^R$ 、 $node_I^D$ 、 $node_I^E$ 的集合, 而 $EInstance$ 是所有 $edge_I^R$ 、 $edge_I^B$ 、 $edge_I^A$ 、 $edge_I^C$ 、 $edge_I^D$ 以及 $edge_I^E$ 的集合。 $source_I^G$ 和 $target_I^G$ 用于定义关系实例边两端连接的角色实例节点。 $attrs_I^G$ 用于描述图实例中具有的所有属性实例。
- $node_I^O = (nType^O, attrs_I^O)$, 用于描述对象实例。其中 $nType^O \in NType^O$, $attrs_I^O$ 用于描述对象实例中所有属性实例。
- $node_I^P = (nType^P, attrs_I^P)$, 用于描述点实例。其中 $nType^P \in NType^P$, $attrs_I^P$ 用于描述点实例中所有属性实例。
- $node_I^R = (nType^R, attrs_I^{Role})$, 用于描述角色实例。其中 $nType^R \in NType^R$, $attrs_I^{Role}$ 用于描述角色实例中所有属性实例。
- $node_I^D$: 用于描述分解图实例。其中 $type(node_I^D) \in NType^D$ 。
- $node_I^E$: 用于描述剖视图实例。其中 $type(node_I^E) \in NType^E$ 。
- $edge_I^R = (eType^R, attrs_I^{Rela})$, 用于描述关系实例。其中 $eType^R \in EType^R$,

$attrs_I^{Rela}$ 用于描述关系实例中所有属性实例。

- $edge_I^A$: 用于描述对象实例与点实例之间的包含关系。其中 $type(edge_I^A) \in EType^A$ 。
- $edge_I^B$: 用于描述对象实例（或点实例）与角色实例之间的绑定关系。其中 $type(edge_I^B) \in EType^B$ 。
- $edge_I^D$: 用于描述对象/关系/点/角色实例与图实例之间的分解关系。
- $edge_I^F$: 用于描述对象/关系/点/角色实例与图实例之间的剖视关系。
- $edge_I^C$: 用于描述实例跨模型关联关系，其中 $type(edge_I^C) \in EType^C$ 。
- 属性实例: $attr_I$ ，其中 $type(attr_I) \in TAttrs$ ，属性实例的集合为 $attrs_I$ 。

4.3.3 基于图模式匹配的模型查询流程

在基于图对 GOPPRR-E 元模型和实例进行形式化表达的基础上，研究基于图模式匹配的模型查询流程。首先采用模式图对约束上下文给定的模型元素组合进行描述，然后使用实例图对所有需检查的图模型进行描述。然后将模式图和实例图输入图匹配引擎，经图引擎执行后可得到所有符合约束上下文的待检查模型元素组合。

在图模式匹配中，模式图（Pattern Graph, P）用于指定一组图结构以及结构中节点和边的语义信息。**模型图本质上也是一个实例图**。图模式匹配就是在实例图中找到一组符合 P 所描述的图结构和语义的子图 G_{sub} （也称为**匹配**）。模式图的定义如式 4.6 所示。其中 $PNode$ 是所有模式节点 $pNode$ 的集合， $PEdge$ 是所有模式边 $pEdge$ 的集合； $source_p, target_p$ 是从模式边指向其两端模式节点的映射函数。此外 $pNode$ 和 $pEdge$ 都具有类型，其类型为类型图中定义的类型节点和类型边。即 $type(pNode) \in Ntype$ ， $type(pEdge) \in EType$ 。而模式图 P，它可具有或不具有类型。当 P 具有类型时，其类型为具体的图元模型，即 $type(P) \in GType$ 。

$$P = (PNode, PEdge, source_p, target_p) \quad (4.6)$$

在定义了面向 GOPPRR-E 元素的图模式的基础上，将模型正确性检查流程中的模型查询问题转化为一个图模式匹配问题。如图 4.3 所示，使用模式图描述约束上下文中定义的模型元素组合，使用源实例图描述待检查的 GOPPRR-E 方法所构建的模型。然后将模式图和源实例图输入图匹配引擎进行执行。在执行时，当模式图 P 具有类型时，先定位到所有符合模型图类型的实例图，在根据模式图中定义的图结构得到所有

匹配。当 P 没有类型时,将面向所有源实例图搜索符合 P 中所定义图结构的所有匹配。然后获取匹配结果中所有元素的特征信息(包括属性信息和结构信息)形成查询结果,以支持查询结果的约束可满足性检查。

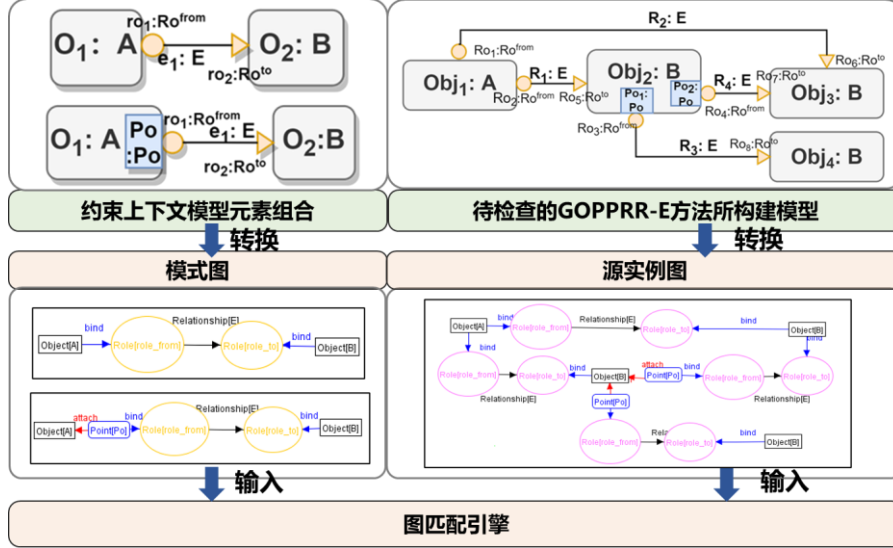


图 4.3 基于图模式匹配模型查询

4.4 基于一阶逻辑的 GOPRR-E 模型约束可满足性检查

面向 GOPRR-E 方法的模型正确性检查过程离不开对模型查询结果进行约束可满足性检查。本节研究基于一阶逻辑的模型约束可满足性检查方法。首先对一阶逻辑相关理论进行介绍,在此基础上,建立 GOPRR-E 模型元素概念与一阶逻辑表达式之间的映射关系,基于映射关系,研究基于一阶逻辑实现模型约束规则中约束表达式的形式化表达,实现基于一阶逻辑的约束可满足性检查过程。

4.4.1 一阶逻辑理论

一阶逻辑又可以称为一阶谓词演算或者谓词逻辑,是一种将自然语言形式化为可计算格式的方法,在数学、语言学、计算机科学以及软件工程领域应用广泛^[72]。一阶逻辑可以用于形式化描述模型以及模型的约束,从而实现模型的约束检查。一阶逻辑可以理解为命题逻辑^[73]的延伸,以下将介绍一阶逻辑相关理论。

(1) 命题及命题公式

命题是使用语言、符号或式子表达的陈述句,其值必须为真或者为假,不能同时

是两者^[74]。例如，小明是男生，即为一个命题，且它的值为真。原子命题是最简单的命题，它不含与或非等连接词，其值的真或假不依赖于任何其他命题。由命题组合形成的更复杂的命题可称为命题公式（或复合命题），它是用逻辑连接词从原子命题中构造出来的。逻辑连接词包括：合取连接词（ $\wedge$ ，也叫与）、析取连接词（ $\vee$ ，也叫或）、否定连接词（ $\neg$ ，也叫否）、蕴含连接词（ $\rightarrow$ ）以及等价连接词（ $\Leftrightarrow$ ）。其中，蕴含的含义是“如果…那么…”， $A \rightarrow B$ 可看作 A 是 B 的充分条件，意思就是有 A 必有 B 。等价的含义是在任何情况下，两个命题的真值是同步相等的， $A \Leftrightarrow B$ ，表示当且仅当 A 为真时， B 为真。命题公式的真值可以从它包含的原子命题的真值计算出来。真值表允许人们根据原子命题的真值，唯一地确定命题公式的真值。

(2) 一阶逻辑命题符号化

在一阶逻辑中，命题被分解为个体词、谓词和量词。个体词是可以独立存在的客体，它可以是一个具体的事物，也可以是一个抽象的概念^[75]。例如，牛肉、整数等都可以是个体词。谓词是用于描述个体词的性质或者描述个体词之间关系的词语，例如李明是男生。量词是用于刻画个体常项或者个体变项之间数量的词，包括：全称量词（ $\forall$ ）和存在量词（ $\exists$ ）。例如， $\forall xF(x)$ 表示个体域中的所有个体都具有性质 F ， $\exists xF(x)$ 表示个体域里存在个体具有性质 F 。有关个体词、谓词的相关概念如表 4.1 所示。

表 4.1 有关个体词、谓词的相关概念和符号

概念	解释
个体常项	表示具体的个体的词，常使用小写英文字母 $a, b, c \dots$ 表示。
个体变项	表示抽象或泛指的个体的词，常使用小写英文字母 $x, y, z \dots$ 表示。
个体域	表示个体变项的取值范围，也叫论域。个体域可以是有限事物的集合，例如 $\{5, 6, 7, 8\}$ ，也可以是无限事物的集合，例如正整数集合。
全总个体域	当无特殊声明时，将宇宙间的一切事务组成个体域。
谓词常项	表示具体性质或关系的谓词，用大写英文字母 $F, G, H \dots$ 表示。例如，可以用 F 表示“…是红色的”。
谓词变项	表示抽象的或泛指的谓词，也用大写英文字母表示，例如 $F(x)$ 表示 x 具有性质 F 。

续表 4.1

概念	解释
谓词	元数用于描述谓词中所包含的个体词的数目, 含有 n ($n \geq 1$) 个个体词的谓词称为 n 元谓词。一元谓词通常表现个体词的性质, n ($n \geq 2$) 元可用于描述个体词之间的关系。使用 $P(x_1, x_2, \dots, x_n)$ 表示泛指
	的 n 元谓词。 $P(x_1, x_2, \dots, x_n)$ 不是命题, 其真值不能确定。当指定某一谓词常项代替 P , 指定 n 个个体常项 $a_1, a_2, \dots, a_n$ 代替 n 个个体变项时它才变成命题。例如 $M(x, y)$ 是一个二元谓词, 但不是命题, 当令 M 表示 x 小于 y , 且取 x 为 2、 y 为 3 后才是命题。
	0 元谓词 特殊地, 将不含个体变项的谓词称为 0 元谓词, 例如 $P(a_1, a_2)$
特性谓词	是描述个体特征的谓词。例如 $U(x)$ 表示 $x \in \{\text{人类}\}$ 。

此外, 为了更好的表示谓词逻辑, 引入了函数的概念, 通常使用小写字母 $f, g, h, \dots$ 表示函数符号。函数是一种关系, 用于将输入的值根据特定的解释映射为输出的值。函数的返回值很丰富, 需要提前定义, 例如 $f(x)$ 可以表示 “ $x+5$ ” 也可以表示 x 的颜色是红色, 而谓词可认为是一个返回真假布尔值的函数。

(3) 一阶逻辑公式

一阶逻辑公式是由非逻辑符号以及逻辑符号构造的包含布尔逻辑的表达式。一阶逻辑公式的构造涉及字母表、项以及合成公式。其具体概念如表 4.2 所示。

表 4.2 一阶逻辑公式相关概念

概念	解释
字母表	(1) 非逻辑符号: 个体常项符号、函数和谓词符号; (2) 逻辑符号: 个体变项符号、量词符号、连接词符号 ($\neg, \wedge, \vee, \rightarrow, \leftrightarrow$) 以及括号、逗号和等号。
项	(1) 个体常项和个体变项是项; (2) 若 $\Phi(x_1, x_2, \dots, x_n)$ 是任意的 n 元函数, $t_1, t_2, \dots, t_n$ 是任意的 n 个项, 则 $\Phi(t_1, t_2, \dots, t_n)$ 是项。(3) 所有的项都是通过有限次使用 (1) 和 (2) 得到的。
原子公式	设 $R(x_1, x_2, \dots, x_n)$ 是任意 n 元谓词, $t_1, t_2, \dots, t_n$ 是任意的 n 个项, 则称 $R(t_1, t_2, \dots, t_n)$ 是原子公式。

续表 4.2

概念	解释
合成公式	(1) 原子公式是合成公式；(2) 若 A 是合成公式，则 $(\neg A)$ 也是合成公式； (3) 若 A, B 是合成公式，则 $(A \wedge B)$ 、 $(A \vee B)$ 、 $(A \supset B)$ 、 $(A \Leftrightarrow B)$ 也是合成公式；(4) 若 A 是合成公式，则 $\forall xA$ 以及 $\exists xA$ 也是合成公式。(5) 只有有限次地应用 (1) ~ (4) 构成的式子才是合成公式。
分类	设 B 为一个公式，若 B 在任何情况下均为真，则 B 为永真式；若 B 在任何情况下都为假，则 B 为矛盾式；若至少存在一个情况使 B 为真，则 B 为可满足式。

4.4.2 基于一阶逻辑的模型约束形式化表达

为了使用一阶逻辑描述形式化描述约束规则，首先需要构建一阶逻辑公式的字母表。字母表中的逻辑符号以及个体常项符号与一阶逻辑公式中的一致，而关于谓词的解释则与具体的应用上下文有关，本文的应用上下文是 GOPPRR-E 元建模方法构建的模型。因此首先建立 GOPPRR-E 概念与一阶逻辑中谓词的映射关系。

4.4.2.1 GOPPRR-E 概念向一阶逻辑的映射

GOPPRR 元模型可以看作是 GOPPRR 实例的类型，将其映射为如下一元谓词：

(1) 图元模型： $Graph^{GraphType}(x)$ ，例如： $Graph^{blockDiagram}(x)$ 表示 x 的类型是一种 blockDiagram 图元模型；

(2) 对象元模型： $Object^{ObjectType}(x)$ ，例如 $Object^{block}(x)$ 表示 x 的类型是一种 block 对象元模型。

(3) 点元模型： $Point^{PointType}(x)$ ，例如 $Point^{proxyPort}(x)$ 表示 x 的类型是一种 proxyPort 点元模型；

(4) 关系元模型： $Relationship^{RelaType}(x)$ ，例如 $Relationship^{refine}(x)$ 表示 x 的类型是一种 refine 关系元模型；

(5) 角色元模型： $Role^{RoleType}(x)$ ，例如 $Role^{refine_from}(x)$ 表示 x 的类型是一种 refine_from 角色元模型；

(6) 属性元模型： $Property^{PropertyType}(x)$ ，例如 $Property^{name}(x)$ 表示 x 的类型是一种 name 属性元模型。

此外，将衍生概念 (Extension) 中的关系映射为以下的多元谓词：

(1) 绑定: $Bind(x, y)$ 。对象和角色之间绑定可以表示为 $Object(x) \wedge Role(y) \wedge Bind(x, y)$ 。

(2) 关系与两端角色之间的关系: $EndWith(x, y, z)$, 即 x 的末尾两端分别为 y 和 z , 例如 $Relationship(x) \wedge Role(y) \wedge Role(z) \wedge EndWith(x, y)$ 。

(3) 分解: $Decompose(x, y)$, 代表 x 分解为 y 。对象和图之间的分解可以表示为 $Object(x) \wedge Graph(y) \wedge Decompose(x, y)$ 。

(4) 剖视: $Explode(x, y)$, 代表 x 剖视为 y 。点和图之间的剖视可以表示为 $Point(x) \wedge Graph(y) \wedge Explode(x, y)$ 。

(5) 对象与点之间的、图与对象和关系之间的包含关系: $Include(x, y)$, 代表代表 x 包含 y 。例如 $Graph(x) \wedge Object(y) \wedge Include(x, y)$ 。

(6) 属性引用关系: $Have(x, y)$, 表示 x 具有 y 属性。例如图和属性之间的引用关系表示为: $Graph(x) \wedge Property(y) \wedge Have(x, y)$ 。

(7) 跨模型关联关系: $Reference^{rType}(x, y)$ 表示 x 和 y 之间具有跨模型关联关系, $rType$ 表示具体的关联关系类型, 例如 x 与 y 之间的跨模型分配关系可以描述为 $Reference^{Allocate}(x, y)$ 。

4.4.2.2 模型正确性检查约束规则向一阶逻辑的映射

明确了 GOPPRR-E 元模型在一阶逻辑中的表达后, 将面向 GOPPRR-E 元素的模型约束规则使用一阶逻辑表达式进行形式化描述。

(一) 模型结构约束

(1) 连接性约束: 引入 $ConnectWith(x, y, z)$ 谓词描述两个对象实例之间通过关系实例的连接关系, 其中, x 代表关系实例, y 和 z 为对象实例。以 SysML 为例, “任意连接两个内部模块之间的关系可以是项目流和控制流” 可以使用式 4.7 表达如下。

$$\begin{aligned} & \forall x (Object^{partProperty}(y) \wedge Object^{partProperty}(z) \\ & \quad \wedge ConnectWith(x, y, z) \rightarrow Relationship^{itemFlow}(x) \\ & \quad \vee Relationship^{controlFlow}(x)) \end{aligned} \quad (4.7)$$

(2) 强聚合约束: 引入二元谓词 $Composite(x, y)$ 表示 y 是 x 的组成部分。对于类型为 A 的对象实例 x 和 y , x 具有类型为组合部分的属性 z , 且 z 的值为 y , 可以等价于 $Composite(x, y)$ 。

$$\begin{aligned} &Object^A(x) \wedge Object^A(y) \wedge Property^{composition}(z) \wedge Have(x, z) \\ &\wedge Equals(Value(z), y) \Leftrightarrow Composite(x, y)) \end{aligned} \quad (4.8)$$

(3) 弱聚合约束：引入二元谓词 $Aggregate(x, y)$ 表示 y 是 x 的聚合部分。对于类型为 A 的对象实例 x 和 y ， x 具有类型为聚合部分的属性 z ，且 z 的值为 y ，可以等价于 $Aggregate(x, y)$ 。

$$\begin{aligned} &Object^A(x) \wedge Object^A(y) \wedge Property^{aggregation}(z) \wedge Have(x, z) \\ &\wedge Equals(Value(z), y) \Leftrightarrow Aggregate(x, y)) \end{aligned} \quad (4.9)$$

(4) 抽象约束：引入一元谓词 $Abstract(x)$ 来表示 x 是抽象的。例如，式 4.10 表示任意类型为接口的对象实例是抽象的。

$$\forall x (Object^{Interface}(x) \rightarrow Abstract(x)) \quad (4.10)$$

(5) 继承约束：引入谓词 $Inherit(x, y)$ 表示 y 继承 x 。式 4.11 表示任意两个类型为 $Block$ 的对象实例 y 、 z 通过类型为 $Generalize$ 的关系实例 x 相连，则 z 继承 y 。

$$\begin{aligned} &\forall x \forall y (Object^{Block}(x) \wedge Object^{Block}(y) \wedge Relationship^{Generalize}(z) \\ &\wedge ConnectWith(z, x, y) \rightarrow Inherit(x, y)) \end{aligned} \quad (4.11)$$

(6) 对象多重性约束：引入四个谓词 $Equals(x, n)$ 、 $Max(x, n)$ 、 $Min(x, n)$ 、 $Between(x, m, n)$ 分别描述对象的等值多重性、最大多重性、最小多重性和范围多重性约束。其中 x 为个体项， n 和 m 表示具体的数量。此外，为了使一阶逻辑公式更加精简，引入符号 “ $\cup_1^n P_i$ ” 以表示对 P_1 至 P_n 连续使用运算符 $\wedge$ 。以最大多重性为例，式 4.12 表示类型为 Kid 的对象实例 y 最多只能通过类型为 $ParentOf$ 的关系实例与两个类型为 $Parent$ 的对象实例相连。

$$\begin{aligned} &\forall y (Object^{Kid}(y) \wedge \exists x_1, \dots, x_n \exists z_1, \dots, z_n (\cup_1^n Relationship^{ParentOf}(x_i) \\ &\wedge \cup_1^n Object^{Parent}(z_i) \wedge \cup_1^n ConnectWith(x_i, y, z_i)) \rightarrow Max(n, 2)) \end{aligned} \quad (4.12)$$

(7) 点多重性约束：描述点多重性的谓词与对象多重性的一致。例如，式 4.13 表示任意类型为 $Block$ 的对象实例上可以具有 3~5 个类型为 $ProxyPort$ 的点实例。

$$\begin{aligned} &\forall x (Object^{Block}(x) \wedge \exists y_1, \dots, y_n (\cup_1^n Point^{ProxyPort}(y_i) \\ &\wedge \cup_1^n Include(x, y_i)) \rightarrow Between(n, 3, 5)) \end{aligned} \quad (4.13)$$

(8) 点强制性约束：以任意 A 类型对象实例必须包含一个 B 类型的点实例为例，可以表示为式 4.14：

$$\forall x (Object^A(x) \rightarrow \exists y (Point^B(y) \wedge Include(x, y))) \quad (4.14)$$

(9) 结构强制性约束：以任意 A 类型的对象实例必须通过 C 类型的关系实例与 B 类型的对象实例相连为例，可以表示式 4.15：

$$\forall y(Object^A(y) \rightarrow \exists z \exists x (Object^B(z) \wedge Relationship^C(x) \wedge ConnectWith(x, y, z))) \quad (4.15)$$

(10) 结构唯一性约束：它是一种特殊的对象多重性约束，此处不赘述。

(11) 集比较约束：引入谓词 $Set(x_1, \dots, x_n)$ 表示由若干个体项构成的集合，使用谓词 $SubOf(x, y)$ 、 $EquivalentTo(x, y)$ 、 $Disjoint(x, y)$ 分别表示 x 是 y 的子集、x 与 y 是等价的、x 与 y 是排斥的。使用符号 $\in$ 表示个体与集合之间的从属关系。以子集约束为例，式 4.16 表示集合 $y_1, \dots, y_n$ 是集合 $o_1, \dots, o_m$ 的子集。

$$SubOf(Set(y_1, \dots, y_n), Set(o_1, \dots, o_m)) \Leftrightarrow \bigcup_1^n (y_i \in Set(o_1, \dots, o_m)) \quad (4.16)$$

(12) 分解约束：以任意类型为 A 的对象实例可以分解为类型为 B 或 C 的图实例为例，可以表示为式 4.17：

$$\forall x (Object^A(x) \wedge Decompose(x, y) \rightarrow Graph^B(y) \vee Graph^C(y)) \quad (4.17)$$

(13) 分解强制性约束：以任意类型为 A 的对象实例必须分解至类型为 B 的图实例为例，可以表示为式 4.18：

$$\forall x (Object^A(x) \rightarrow \exists y (Graph^B(y) \wedge Decompose(x, y))) \quad (4.18)$$

(14) 剖视约束：以任意类型为 A 的对象实例可以剖视为类型为 B 或者类型为 C 的图实例，可表示为式 4.19：

$$\forall x (Object^A(x) \wedge Explode(x, y) \rightarrow Graph^B(y) \vee Graph^C(y)) \quad (4.19)$$

(15) 剖视强制性约束：以类型为 A 的对象实例必须剖视至类型为 B 的图实例为例，可以表示为式 4.20：

$$\forall x (Object^A(x) \rightarrow \exists y (Graph^B(y) \wedge Explode(x, y))) \quad (4.20)$$

(16) 环约束：以自反性约束为例，任意类型为 C 的关系实例两端连接的对象实例为同一个，可以表示为式 4.21。

$$\forall x (Relationship^C(x) \wedge Object(y) \wedge ConnectWith(x, y, y)) \quad (4.21)$$

(二) 属性约束

(1) 属性引用约束：表示图/对象/关系/点/角色实例可以拥有什么类型的属性，例如任意 A 类型的对象实例可以拥有 B 和 C 类型的属性实例，可表示为式 4.22。

$$\forall x (Object^A(x) \wedge Have(x, y) \rightarrow Property^B(y) \vee Property^C(y)) \quad (4.22)$$

(2) 属性值约束：使用 $Value(x)$ 表示属性的值，并使用 $Gt(x,y)$ 、 $Ge(x,y)$ 、 $Equals(x,y)$ 、 $Lt(x,y)$ 、 $Le(x,y)$ 五个二元谓词来分别表示 x 与 y 之间的“大于”、“大于等于”、“等于”、“小于”、“小于等于”关系。例如，任意类型为 Age 的属性实例的值大于等于 0 可以表示为式 4.23：

$$\forall x(Property^{Age}(x) \rightarrow Ge(Value(x), 0)) \quad (4.23)$$

(3) 属性多重性约束：以任意 A 类型的对象实例只能拥有一个类型为 B 的属性实例，可以表示为式 4.24：

$$\begin{aligned} \forall x(Object^A(x) \wedge \exists y_1, \dots, y_n (\bigcup_1^n Property^B(y_i) \wedge \bigcup_1^n Have(x, y_i)) \\ \rightarrow Equals(n, 1)) \end{aligned} \quad (4.24)$$

(4) 属性值唯一性约束：引入一元谓词 $Unique(x)$ 来表示 x 的值是唯一的。以任意类型为 ID 的属性实例的值是唯一的为例，可表示为式 4.25：

$$\forall x(Property^{Age}(x) \rightarrow Unique(x)) \quad (4.25)$$

(5) 属性值比较约束：例如，任意 A 类型的对象实例 x 具有一个 D 类型的属性实例 y 和一个 E 类型的属性实例 z ， y 的值须小于等于 z 的值，可表示为式 4.26：

$$\begin{aligned} \forall x(Object^A(x) \wedge Property^B(y) \vee Property^E(z) \wedge Have(x, y) \\ \wedge Have(x, z) \rightarrow Lt(Value(y), Value(z))) \end{aligned} \quad (4.26)$$

(6) 属性强约束：以任意类型为 Action 的对象实例必须具有类型为 Type 的属性实例为例，表示为式 4.27：

$$\forall x(Object^{Action}(x) \rightarrow \exists y(Property^{Type}(y) \wedge Have(x, y))) \quad (4.27)$$

(三) 混合约束

以模型内部混合约束为例：任意类型为 A 的点实例 y 和点实例 z ，它们通过类型为 B 的关系实例 x 相连，且 y 具有类型为 P 的属性实例 k_1 ， z 具有类型为 P 的属性实例 k_2 ，规定 k_1 的值必须 k_2 的值相等。可以使用式 4.28 表示：

$$\begin{aligned} \forall y \forall z (Relationship^B(x) \wedge Point^A(y) \wedge Point^A(z) \\ \wedge ConnectWith(x, y, z) \wedge Property^P(k_1) \\ \wedge Property^P(k_2) \wedge Have(y, k_1) \wedge Have(z, k_2) \\ \rightarrow Equals(Value(k_1), Value(k_2))) \end{aligned} \quad (4.28)$$

4.4.3 基于一阶逻辑的约束可满足性检查过程

本节在采用一阶逻辑对 GOPPRR-E 模型约束规则进行形式化表达的基础上,构建约束可满足性检查过程。将检查过程分为:(1)约束规则冲突检测,(2)实施检查。

约束规则冲突检测的目的是判断所定义约束规则中的约束表达式之间是否存在矛盾,导致不存在一个可行的解能满足所有约束表达式。进行约束规则冲突检测的步骤包括:(1)提取:提取约束规则中面向同一元素的所有约束表达式。例如规则 A 中针对元素 x 定义了约束表达式“ $x+3<5$ ”,规则 B 中针对元素 x 定义了约束表达式“ $x-1>4$ ”。(2)整合:将提出的约束表达式使用逻辑连接符 $\wedge$ 构造为一阶逻辑表达式,例如“ $(x+3<5)\&\&(x-1>4)$ ”。(3)冲突判断:基于约束可满足性求解引擎判断构造的一阶逻辑表达式是否存在可行解,若不存在一个可行解,则认为约束规则存在矛盾,模型元素在任何情况下均无法满足约束规则中的所有约束表达式。(4)冲突提示:若存在冲突,则提示用户存在冲突的约束规则,然后由用户手动解决冲突。

在进行约束规则冲突检测并得到无冲突的约束规则后,实施模型的可满足性检查。可满足性检查用于确定系统架构模型 M 是否满足约束规则中的系统静态特性 P_s 。因此采用于一阶逻辑对 M 和 P_s 进行形式化表达,使用逻辑连接符 $\wedge$ 连接 M 和 P_s ,构建一阶逻辑表达式: $M \wedge P_s$, 并确定该式的真值。若该式的真值为假,则表示 M 和 P_s 存在冲突, M 不满足 P_s 。

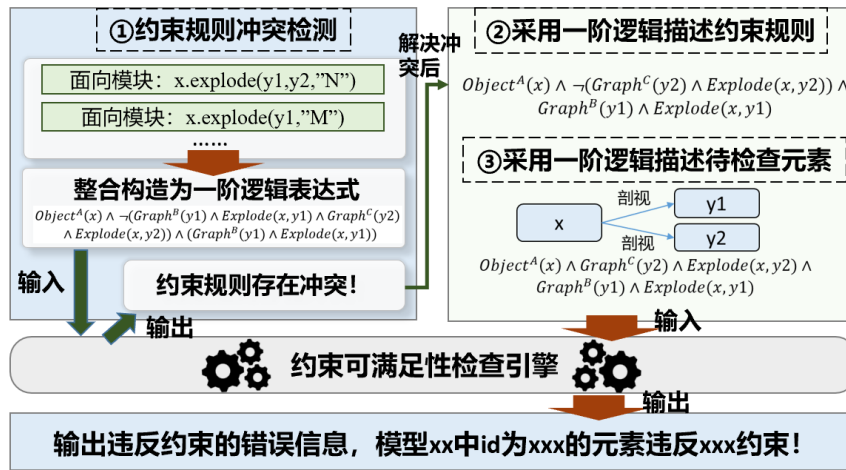


图 4.4 基于一阶逻辑的约束可满足性检查过程

如图 4.4 所示,为了实现 GOPPRR-E 方法所构建模型的可满足性检查,首先将约束规则中的约束表达式基于一阶逻辑进行表达(称为 FOL^C)。然后基于模型查询部分

得到的查询结果，将结果的每个匹配中与约束表达式相关的所有属性信息和结构信息基于一阶逻辑进行表达，即： $List_{FOL} = \{FOL_1^M, \dots, FOL_i^M, \dots, FOL_n^M\}$ 。对于每一个匹配，构建一阶逻辑表达式“ $E^{FOL}: FOL^C \wedge FOL_i^M$ ”。然后将该表达式输入约束可满足性检查引擎，求解出 E^{FOL} 是否可满足，若 E^{FOL} 是可满足的，则认为该组匹配满足所定义的约束；若 E^{FOL} 是不可满足的，则认为该组查询结果不满足所定义约束，将获取匹配中与所定义约束具有冲突的模型元素信息，并生成错误信息。待查询结果中所有匹配检查完毕后，将所有的错误信息输出，建模工程师即可依据错误信息定位到有问题的模型，并对模型进行改正和完善。

4.5 模型正确性检查方法应用案例

依据上述流程，以 UML 建模的温度控制系统活动图为案例阐述本节所提模型正确性检查方法的应用过程。图 4.5 展示了案例部分使用到的 UML 活动图元模型，包括：动作对象元模型、四种控制节点（即分支节点、初始节点、最终节点、集合节点）对象元模型、栓点元模型、控制流关系元模型和对象流关系元模型（以及两个关系元模型两端的四个角色元模型）。动作上可以拥有栓，所有控制节点和动作之间、控制节点之间以及动作之间均可以通过控制流相连。控制节点和动作上的栓，以及动作之间的栓可以通过对象流相连。该案例中，以“分支节点的输入流和输出流的类型需一致”为约束规则（ruleA）进行正确性检查。

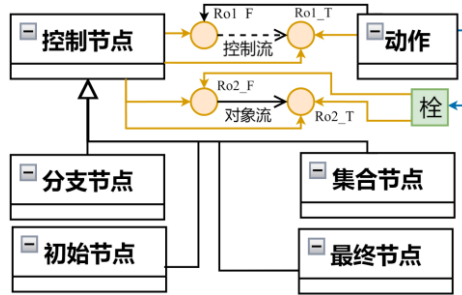


图 4.5 UML 活动图中的元模型（部分）

首先基于图模式匹配进行模型查询。在 ruleA 中，若分支节点的输入流是对象流，而输出流却是控制流，则由对象流传递的对象信息在分支节点处丢失了。因此，以输入流是对象流的分支节点所构建的模型结构为约束上下文进行模型查询。如图 4.6 所示，约束上下文的模型结构包括：类型为栓点元模型的 po1、类型为分支节点对象元

模型的 obj1、类型为对象流关系元模型的 rel、类型分别为 Ro2_F 和 Ro2_T 角色元模型的 ro1 和 ro2。其中，ro1 与 po1 绑定，ro2 与 obj1 绑定，rel 两端为 ro1 和 ro2。该模型结构可使用模式图 P 表示。此外，待检查的温度控制活动图模型可表示为图 4.6 中的源实例图。经图模式匹配执行后可以在待检查模型中获得一个匹配，即匹配①。在匹配①中，属性信息包括所有元素的 localName、id 以及其他自定义属性信息。而结构信息包括“obj1（分支关系）通过控制流关系与展示温度动作相连”等。该匹配中所有元素的属性信息和结构信息构成查询结果，输入至约束可满足性检查部分。

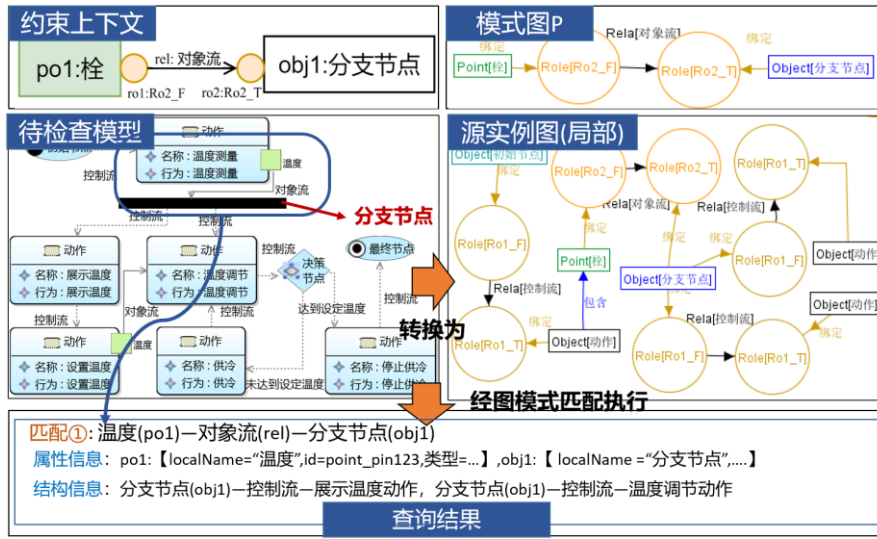


图 4.6 案例模式图、源实例图及查询结果

然后基于一阶逻辑进行约束可满足性检查。由于分支节点只允许通过控制流和对象流与动作以及动作上的栓相连，故 ruleA 可以转化为“若分支节点的输入是与栓连接的对象流，那它的输出流不能是与动作连接的控制流”，ruleA 可基于一阶逻辑表示为图 4.7 中的 FOL^C 。由模型查询步骤得到的查询结果包含一个匹配，即图 4.7 所示匹配①，该匹配中与约束规则相关的结构信息可基于一阶逻辑表达为 FOL_1^M 。那么 E^{FOL} 为 “ $FOL^C \wedge FOL_1^M$ ”，它是不可满足的。观察图 4.7 中的模型可以发现，该模型中分支节点的输入流是对象流，传输对象令牌信息，而输出流是控制流，即对象令牌信息在分支节点处丢失了，没有传输至下游需要该对象令牌的动作，即该模型没有正确地描述温度控制系统的功能活动。

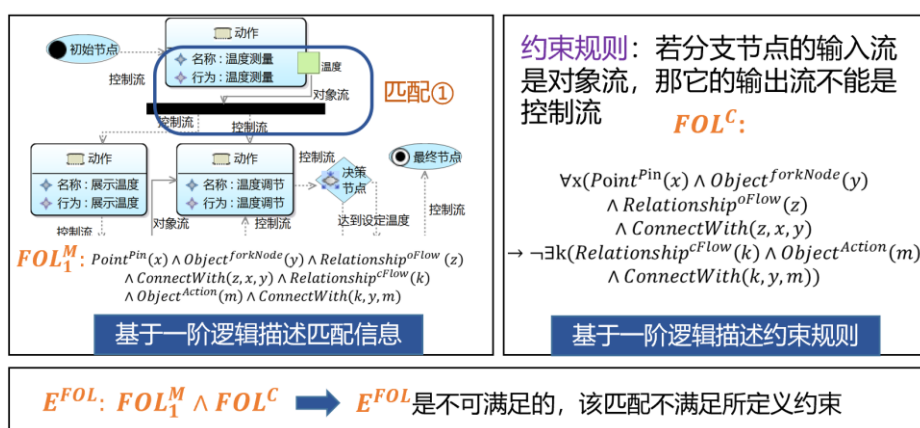


图 4.7 温度控制系统活动图约束可满足性检查示例

4.6 本章小结

本章阐述了面向多领域系统架构模型正确性检查方法。首先，面向 GOPRR-E 方法所构建多领域系统架构模型，建立了涵盖模型查询和约束可满足性检查的模型正确性检查流程。然后，提出了基于图模式匹配的约束上下文查询方法，实现约束上下文中所有待检查模型元素的查询。其次，提出了基于一阶逻辑的模型约束可满足性检查方法，实现对模型查询部分所有待检查模型元素的约束可满足性求解，最终实现 GOPRR-E 所构建多领域系统架构模型的正确性检查。

第5章 模型正确性检查语言设计与实现

5.1 引言

在系统架构模型正确性检查理论方法的基础上，在工程实践中进行模型正确性检查的关键步骤是明确并定义约束规则，约束规则可以通过模型正确性检查语言来描述。模型正确性检查语言是基于某种范式（文本式或图形式）、具有良好语法和语义结构的计算机语言。本章针对目前缺乏同时支持模型构建和模型正确性检查的建模语言的问题，研究支持统一形式化描述多领域系统架构模型构建、模型正确性检查的建模语言，该研究在鲁金直等人^[17]所提出的 KARMA 语言的基础上开展。首先分析基于 GOPPRR-E 元建模方法的 KARMA 语言的已有语法结构和语义风格，然后在基于一阶逻辑的约束形式化表达的基础上，扩展 KAMRA 语言，采用内嵌式和声明式的风格设计和定义模型正确性检查部分的抽象语法和具体语法语义。最后，基于语言解释技术构建模型正确性检查插件，以实现基于图模式匹配和一阶逻辑的正确性检查功能。

5.2 基于 KARMA 的模型检查语言抽象语法

本节首先将对 KARMA 语言的已有语法结构和具体语法语义风格进行介绍，在此基础上，设计并定义模型正确性检查部分的抽象语法。

5.2.1 KARMA 语言介绍

KARMA 文本式建模语言是对 GOPPRR-E 元建模方法的实现，提供语法语义来支持 GOPPRR-E 元元模型、元模型与模型的形式化表达，具备多领域系统架构模型一体化建模的能力。KARMA 语言的模型描述部分是整个 KARMA 的基础和核心，它的语法风格是声明式、面向对象的。

如图 5.1 所示，KARMA 模型描述部分的根节点是 Project 类，用于描述一个建模项目，例如运载火箭系统设计项目。Language 类用于描述建模语言，一个 Project 类的实例下可以包含多个 Language 类的实例，即多个建模语言。Language 类由 MetaModelClass 类和 ModelClass 类组成。MetaModelClass 类用于定义和描述 GOPPRR 元模型，包括：

- MetaObject: 用于描述对象元模型。MetaPointRef 和 MetaObjectProperty 分别

用于描述对象元模型拥有的点元模型和属性元模型

- **MetaRelationship**: 用于描述关系元模型。**MetaRelaProperty** 和 **MetaRoleRef** 分别用于描述关系元模型拥有的角色元模型和属性元模型。
- **MetaPoint**: 用于描述点元模型。**MetaPointProperty** 用于描述点元模型拥有的属性元模型。
- **MetaRole**: 用于描述角色元模型。**MetaRoleProperty** 用于描述角色元模型拥有的属性元模型。
- **MetaProperty**: 用于描述属性元模型。
- **MetaGraph**: 用于描述图元模型。**MetaObjectRef**、**MetaRelaRef** 以及 **MetaGraphProperty** 分别用于描述图元模型包含的对象元模型、关系元模型和属性元模型。**ConstraintClause** 用于描述图元模型需要满足的约束。

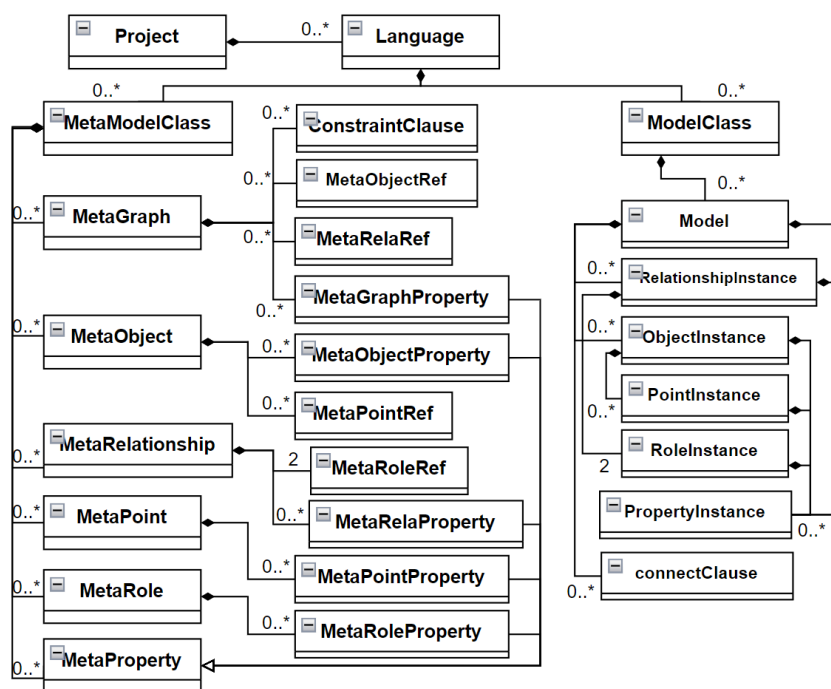


图 5.1 KARMA 语言模型描述部分抽象语法结构

ModelClass 类用于描述模型层的 GOPRR 的实例，特殊地，图实例可以被称为模型，在图 5.1 中使用 **Model** 类表示。**ModelClass** 由若干个 **Model** 组成，**Model** 类中的 **ObjectInstance** 类、**RelationshipInstance** 类和 **PropertyInstance** 类分别用于描述模型中具有的对象实例、关系实例和属性实例。其中，**ObjectInstance** 中可以具有 0 至多个 **PointInstance**（用于描述点实例），**RelationshipInstance** 中必须具有两个 **RoleInstance**（用

于描述角色实例)。connectClause 用于描述对象/关系实例之间的连接关系,以及对象/关系/点/角色实例与图实例之间的剖视关系。图 5.2 中以图元模型和图实例(即模型)的具体语法语义为例,展示了 MetaModelClass 类和 ModelClass 类的具体语法语义。

<pre> 1. // GraphMetaModels 图元模型 2. partial Graph diagramSystemName //图元模型定义 3. initial Property unique graphProperty1= "" extends height1 annotation (localLabel="重量"); //MetaPropertyRef:图中属性 4. Object requirement1; //MetaObjectRef:图中声明组成该图的对象 5. Relationship relat1; //MetaRelaRef 图中声明组成该图的关系 6. Relationship relat2; 7. Constraint (//图中约束的集合 8. bind connector(requirement1.inPoint1, relat2.satisfyTo), //对 象中的点与关系下的角色绑定 9. bind connector(requirement1.outPoint1, relat2.satisfyFrom), 10. bind connector(requirement1, relat1.deriveFrom), //对象与关系下的 角色绑定 11. bind connector(requirement1, relat1.deriveTo), 12. requirement1.decompose(Diagram1, Diagram2), //对象的分解,约束该 对象在实例化之后可以作为该对象分解图的图元模型 13. requirement1.explode(Diagram2, Diagram3), //对象的剖视,约束该对 象在实例化之后可以作为该对象剖视图的图元模型 14. relat1.deriveTo.explode(Diagram3), //角色的剖视 15.); 16. //图的格式、外观等信息 17. annotation (18. localLabel="图", 19. description="描述", 20. type = diagram, 21.); 22. end diagramSystemName; </pre>	<pre> 1. Model modelSystemName instanceof graphSystemName //定义模型 2. Property graphProperty112 = test1 refer graphProperty1 annotation (localLabel="地址1"); //PropertyInstance: 模型属性 3. //ObjectInstance:对象实例定义 4. Object objInstanceSystemName refer objectSystemName 5. Property cel="test" refer cell annotation (localLabel="矩阵"); 6. this.decompose(model2); //对象实例的分解 7. this.explode(model3); //对象实例的剖视 8. annotation (9. localLabel="需求", 10. ___//对象实例外观样式定义 11.) 12. end objInstanceSystemName 13. RelationshipInstances; //关系实例的定义,此处不展开说明 14. --- 15. Connect(connector(req1.po1,r1.role2),connector(req2,r1.role1)); 16. //ConnectClause: 描述对象/点/关系/角色实例之间的绑定关系 17. //描述样式、外观等信息 18. annotation (19. localLabel="System 建模", //模型名称 20. NodePlug-in, 21. LinkModelPlug-in, 22. --- 23.); 24. end modelSystemName; </pre>
--	--

图 5.2 MetaModelClass 类和 ModelClass 类的具体语法语义示例

5.2.2 KARMA 语言模型正确性检查部分抽象语法

采用内嵌式设计模型正确性检查语言抽象语法,其约束规则可在元模型层指定,以描述一类元素或者元素结构需要满足的约束。因此,本小节在 KARMA 语言元模型层(即 MetaModelClass)的基础上,扩展模型正确性检查部分的抽象语法。如图 5.3 所示,对 ConstraintClause 模块进行了扩展,主要包括两个方面:(1)将用于定义约束的 ConstraintClause 的应用范围扩展到了所有的 GOPPRR 元模型。在 KARMA 语言已有的语法中,ConstraintClause 只能应用于图元模型的定义中,现将其范围扩展至了对象/点/关系/属性/角色元模型的定义中,使约束规则的作用范围更广泛。(2)增加了用于模型正确性检查的模块,即 checkModule。在 KARMA 语言已有语义中,已经包含了绑定(bindClause)、分解(decomposeClause)、剖视(explodeClause)三种类型的语法,可以用于支持 3.3 节中定义的连接性约束、分解以及剖视约束。本文增加了 checkModule,用于描述除连接性、分解、剖视约束外的其他约束规则。下文将就 checkModule 进行详细介绍。

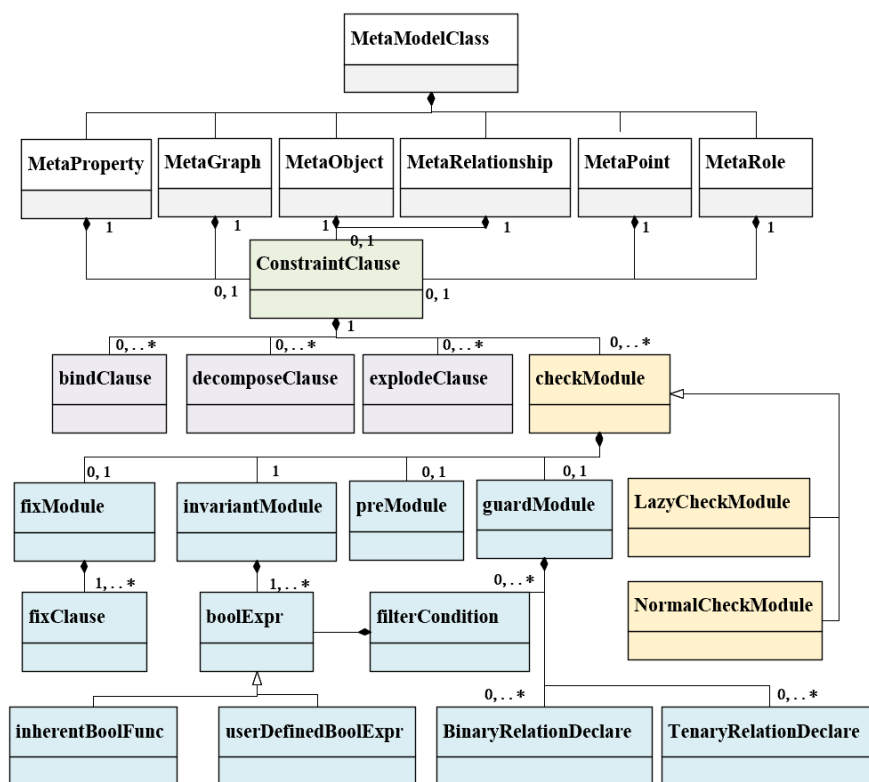


图 5.3 基于 KARMA 扩展的模型正确性检查部分抽象语法

(1) checkModule:

该模块用于描述模型正确性检查部分的约束规则。checkModule 具有两种类型：NormalCheckModule 以及 LazyCheckModule。NormalCheckModule 即普通检查模块，通常用于支持模型结构的约束，支持实时地在建模过程中检查模型是否满足所定义的约束，而无需用户手动执行。LazyCheckModule 即懒惰检查模块，不支持实时地在建模过程中检查，只有当用户主动执行检查时，才会去判断所构建的模型是否满足约束规则。由于属性约束和混合约束通常与具体领域的设计参数相关，建模过程中难以完全地确定模型元素的属性值，故属性相关的约束规则可以通过 LazyCheckModule 进行描述，在完成一次完整的模型构建后由用户手动进行检查。

checkModule 包括头部和主体部分，头部用于定义约束上下文元素，主体部分由 preModule、guardModule、invariantModule 和 fixModule 组成，下文将进行详细介绍：

(2) preModule

用于约束规则中除约束上下文元素以外的其他变量的定义，所定义的变量的作用域为所在的检查模块。

(3) guardModule

此模块用于限制所要进行正确性检查的约束上下文。其中 `filterCondition` 即过滤条件，由布尔表达式组成，可用于限制约束上下文中所定义模型元素的属性值，从而得到约束上下文的子集。而 `binaryRelationDeclare` 和 `tenaryRelationDeclare` 用于描述上下文元素之间需要满足的二元关系（例如对象与点之间的包含关系）和三元关系（例如关系与其两端角色之间的关系），从而限制约束上下文中模型元素的结构。

(4) invariantModule

该模块用于定义约束上下文中模型元素需要满足的约束表达式，即 `boolExpr`，具有两种类型：`inherentBoolFunc` 和 `userDefinedBoolExpr`。`inherentBoolFunc` 是模型正确性检查部分固有的、返回值为布尔类型的函数，而 `userDefinedBoolExpr` 是用户自定义的布尔表达式，通常用于描述属性约束和混合约束。

(5) fixModule

此模块的功能是对违反约束的模型信息进行自动修复，由修复语句 `fixClause` 组成。例如约束表达式中规定模块 A 需要继承模块 B，但是经检查后发现 A 未继承 B 的属性，在 `fixModule` 即可通过 `fixClause` 使 A 继承 B 的所有属性。

5.3 基于 KARMA 的模型正确性检查语言具体语法

模型正确性检查语言的抽象语法定义了检查模块的组成概念及概念之间的关系，而具体语法用于描述语言的具体文本化表达形式，本小节将详细介绍模型正确性检查部分核心的具体语法语义。

5.3.1 模型正确性检查语言关键字

关键字是计算机语言里预先定义的、有特殊内涵的保留字。关键字不能用作自定义的变量或者参数名。模型正确性检查部分的关键字如表 5.1 所示。

表 5.1 模型正确性检查语言关键字

check	Constraint	inv	lazy	guard	Object
Relationship	Point	Role	Graph	Property	this
bind	connector	fix	pre	instanceof	

5.3.2 语言具体语法设计

以下将进行模型正确性检查语言的具体语法规义介绍。首先对具体语法中出现的符号进行规定。“*”代表可出现零次或多次，“？”代表可出现可不出现。“*”以及“？”都可以和括号一起使用，描述括号内所有元素的出现次数。

5.3.2.1 ConstraintClause 具体语法规义

ConstraintClause 部分由关键词 **Constraint** 标识，由一对括号表示该部分的开始和结束。该部分可以定义：（1）**bindClause**：用于描述连接性约束，由关键词 **bind** 和 **connector** 方法进行描述。**connector** 方法的第一个参数是对象元模型或点元模型的ID，第二个参数是角色元模型的 ID。（2）**decomposeClause**：用于描述分解约束，由 $x.decompose(y_1, y_2, \dots, y_n)$ 函数进行描述，其中参数 x 用于描述对象/点/关系/角色的类型， $y_1, y_2, \dots, y_n$ 用于描述 x 可分解成的图类型。（3）**explodeClause**：用于描述剖视约束，由 $x.explode(y_1, y_2, \dots, y_n)$ 函数进行描述。其中参数 x 用于描述对象/点/关系/角色类型， $y_1, y_2, \dots, y_n$ 为 x 可剖视成的图类型。（4）**checkModule**：在下文详细描述。ConstraintClause 的具体语法规义示例如下所示。

```
1.  Constraint(//bindClause: 对象与关系中的角色绑定
2.      bind connector(requirement1, relat2.satisfyTo),
3.      requirement1.decompose(Diagram1, Diagram2) // decomposeClause: 定义
         requirement1 类型的对象实例能够分解成 Diagram1 和 Diagram2 类型的图实例
4.      requirement1.explode(Diagram2, Diagram3) // explodeClause: 定义
         requirement1 类型的对象实例能够剖视成 Diagram1 和 Diagram2 类型的图实例
5.      checkModule* // 下文将详细阐述该部分具体语法
6.  )
```

5.3.2.2 checkModule 具体语法规义

checkModule 用于描述约束规则，具体语法规义如下所示。其头部由一个 **check** 关键词标识，**nameA** 表示约束规则的标识符。关键字 **lazy** 用于表示该约束规则是懒惰的。**arg** 即参数定义，可以用于定义该约束规则的约束上下文，如果不定义 **arg**，则默认约束上下文为该 **checkModule** 所在元模型的所有实例。

```
1.  lazy? check nameA(args?)(//可以定义参数也可以不定义参数
2.      preModule? //在下文介绍
3.      guardModule? //在下文介绍
```

```

4.    invariantModule//在下文介绍
5.    fixModule? //在下文介绍
6. )

```

其中 `arg` 部分涉及到约束上下文中模型元素类型的指定，其具体语法如下所示，`elemetType` 即 `Object/Point/Property/Role/Relationship/Graph`。由于模型元素的类型即其相应的元模型，所以语法中的 `id` 为元模型的 `id`。

```

1. elementType[id] eleName (, elementType[id] eleName)*
2. //例如以下模型元素变量的指定
3. check testA(Object[requirement] obj1,Relationship[satisfy] rela.
   Point[proxyPort] point1) (//该检查模块的检查上下文为所有 requirement 类型的对
   象实例、所有 satisfy 类型的关系实例以及所有 proxyPort 类型的点实例
4.      .....
5. )

```

此外，为了支持不同类型模型间的一致性检查，模型元素类型可以是复合类型，以指定该类型所在的图元模型。可以通过“!”符号指定某一个图元模型下的对象元模型或者关系元模型，通过“.”符号指定某一对象元模型下的点元模型或某一关系元模型下的角色元模型。例如：

```

1. Graph[BDD]!Object[block] obj1//obj2 的类型是 BDD 图元模型中的 block 对象元模型
2. Graph[UserCase]!Object[block] obj2 //obj2 的类型是 UserCase 图元模型中的
   block 对象元模型
3. Graph[IBD]!Object[partProperty].Point[proxyPort] po1; //po1 的类型是 IBD 图
   元模型中的 partProperty 对象元模型下的 proxyPort 点元模型

```

5.3.2.3 preModule 具体语法

`preModule` 部分使用关键词 `pre` 标识，用于其他变量的定义。支持的数据基本数据类型有基本数据类型 `Int`、`Real`、`Boolean`、`String` 和模型元素类型。例如：

```

1. String str = "hello world!";
2. Property[pro_Type] p1;

```

5.3.2.4 guardModule 具体语法语义

`guardModule` 用于限制正确性检查的约束上下文，该部分由关键词 `guard` 进行标识，其具体语法如下所示。

```

1.  guard(
2.      (filterCondition
3.          |binaryRelationDeclare
4.          |tenaryRelationDeclare)*
5.  )

```

其中 `filterCondition` 的具体语法语义与 `invariantModule` 中的 `userDefinedBoolExpr` 一致，将在下文详细介绍。`binaryRelationDeclare` 和 `tenaryRelationDeclare` 部分分别用于描述约束上下文元素之间需要满足的二元关系和三元关系。二元和三元关系表达式是一种布尔表达式，用于描述元素之间需要满足的关联关系，具体语法如表 5.2 所示。

表 5.2 二元关系和三元关系

N 元关系	描述	具体语法
二元关系	对象与点之间的包含和被包含关系：使用 <code>include</code> 和 <code>containdIn</code> 函数描述。	<code>object1.include (point1)</code> <code>point1.containdIn(object1)</code>
	图/对象/关系/点/角色与属性之间的拥有关系：使用 <code>have</code> 函数描述。	<code>object1.have(property1)</code>
	对象/点/关系/角色与图之间的分解和剖视关系：使用 <code>decomposeTo</code> 和 <code>explodeTo</code> 函数描述。	<code>object1.decomposeTo(graph1)</code> <code>object1.explodeTo(graph1)</code>
	对象与角色、点与角色之间的绑定关系：使用 <code>bindWith</code> 函数描述。	<code>object1.bindWith(role1)</code> <code>role1.bindWith(object1)</code> <code>po1.bindWith(role2)</code>
	关系与其首末两端角色之间的关系：使用 <code>endWith</code> 函数描述。	<code>rela.endWith(fromRole, toRole)</code>
	关系与其始端、末端相连的对象或点之间的关系：使用 <code>connectWith</code> 函数描述。（该三元关系省略了关系与其首末两端角色之间的关系和角色与对象之间的绑定关系）	<code>rela.connectWith(source, target)</code>
三元关系	跨图对象、点、角色、关系之间的关联关系：使用 <code>mappintTo</code> 函数描述。	<code>po1.mappingTo(po2,associa)</code>

例如，以下约束规则中 `guard` 部分中的二元和三元关系描述了图 5.4 所示的由一

个 A 类型的对象实例 o1 及 o1 包含的一个类型为 P 的点实例 p1、一个 B 类型的对象实例 o2 及其包含的一个类型为 P 的点实例 p2、一个类型为 Re 的关系实例 re1 及其两端包含的类型为 FR 的角色实例 ro1 和类型为 TR 的角色实例 ro2 组成的模型结构。其中，ro1 与 p1 是绑定关系，ro2 与 p2 是绑定关系。此模型结构即为约束上下文。

```

1.  check testA(Object[A] o1, Object[B] o2, Relationship[Re] re1, Point[P] p1,
    Point[P] p2, Role[FR] ro1, Role[TR] ro2) (
2.      guard(
3.          o1.include(p1);
4.          o2.include(p2);
5.          re1.endWith(ro1, ro2);
6.          p1.bindWith(ro1);
7.          p2.bindWith(ro2);
8.      )
9. )

```



图 5.4 模型结构实例

此外，为了描述约束上下文中某些相似的模型元素多次出现的场景（例如某个对象实例出现 n 次，则需要在约束上下文中列举出 n 个对象元模型），支持在二元关系和三元关系的定义中通过控制字段描述每个元素出现的次数（称为基数）。基数可以是一个确定的数值（例如“2”），也可以是一个数字范围（例如“1~3”），或者为“ n ”，代表可出现 1 到多次。依据 GOPPRR 元素的特点，基数可在二元关系 include、have、explodeTo 和三元关系 connectWith、mappingTo 中使用。以下代码分别描述了图 5.5 中所示的模型结构：

```

1.  obj1.include(p1,"3");//描述模型结构①，将类型为 P 的点 p1~p3 合并成 p1，使用基数 3 表示 obj1 具有三个类型为 P 的点
2.  obj1.have(prop1,"2");//描述模型结构②，将类型为 VP 的属性 prop1 和 prop2 合并为 prop1，使用基数 2 表示 obj1 具有两个类型为 VP 的属性
3.  obj1.explodeTo(g1,"2");//描述模型结构③，将类型为 G 的两个剖视图 g1、g2 合并为 g1，使用基数 2 表示 obj1 具有两个类型为 G 的剖视图
4.  re11.connectWith(obj1,"2",obj2);//描述模型结构④，将类型为 O 的两个对象 obj1A、obj1B 合并为 obj1，使用基数 2 表示 obj2 与两个类型为 O 的对象相连

```

5. `rel1.connectWith(obj1,"2",obj2,"2");`//描述模型结构⑤, 将类型为 O 的两个对象 obj1A、obj1B 合并为 obj1, 将类型为 B 的两 obj2A、obj2B 合并为 obj2, 并规定 obj1 的基数为 2, obj2 的基数为 2, 表示两个类型为 O 的对象与两个类型为 B 的对象依次相连。
6. `po1.mappingTo(po2,c,"2");`//描述模型结构⑥, 将类型为 P 的两个点 po2、po3 合并为 po2, 使用基数 2 表示 po1 与两个类型为 P 的点通过跨模型分配关系相连

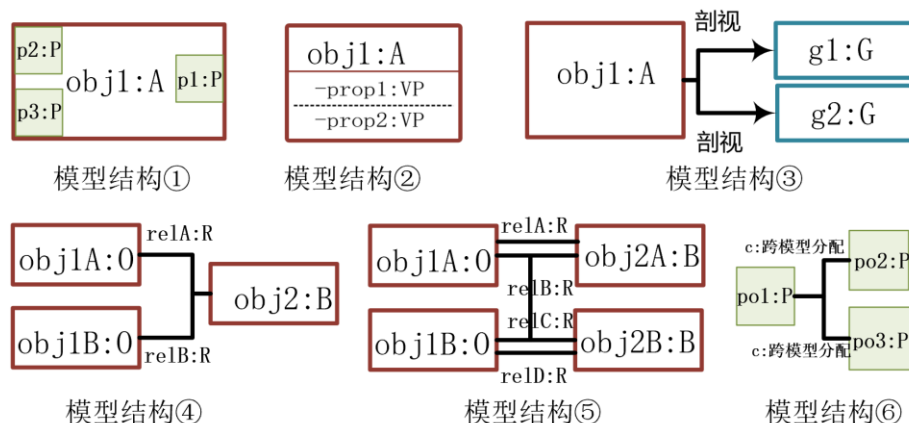


图 5.5 含有基数的模型结构示例

5.3.2.5 invariantModule 具体语法规义

invariantModule 部分由一个关键字 **inv** 标识, 用于描述约束规则中的约束表达式, 其具体语法规义如下所示

```

1.  inv(
2.      (inherentBoolFunc
3.      |userDefinedBoolExpr)*
4.  )

```

用户自定义约束支持使用逻辑运算符“&&”和“||”连接的若干个布尔表达式来约束 GOPPRR 模型实例的固有信息和自定义属性。其中, 单个布尔表达式可由比较运算符“==、!=、<、>、<=、>=”连接左右两边的算术表达式构成。GOPPRR 实例的固有信息包括: id、localName, 可以通过“x.id/localName”的形式获取对象/关系/角色/图/点/属性实例的固有信息。对于自定义属性, 可以通过“.Property[metaId]”的形式从对象/关系/角色/图/点实例中获得元模型 id 为 metaId 的属性实例的值。此外, 单个布尔表达式还可以通过 instanceof 运算符构建, “A instanceof B”表示 A 的类型是 B。以下的代码中定义了一个约束规则: 对于所有类型为 Human 的对象实例, 其类型为 age 的属性实例的值必须大于 0, 其类型为 career 的属性实例的值的类型, 必须是类型

为 careerType 的对象实例。

```
1. check testA(Object[Human] obj)(
2.     inv(
3.         obj.Property[age] >0;
4.         obj.Property[career] instanceof Object[careerType]
5.     )
6. )
```

inherentBoolFunc 即固有布尔函数，5.3.2.6 小节将进行详细阐述。

5.3.2.6 inherentBoolFunc 具体语法语义

(1) 强聚合约束和弱聚合约束

分别使用 isCompositionOf 和 isAggregationOf 函数描述, x.isCompositionOf(y)表示 x 是 y 的组成部分, 例如:

```
1. a.isCompositionOf(b) //a 是 b 的组成部分, 描述强聚合关系
2. a.isAggregationOf(b) //a 是 b 的聚合部分, 描述弱聚合关系
```

(2) 抽象约束和继承约束

分别使用 isAbstract 和 isSubClassOf 函数描述, x.Abstract()表示 x 是抽象的, x.isSubClassOf(y)表示 x 是 y 的子类型。例如:

```
1. a.isSubClassOf(b) //A 是 B 的子类型
2. a.isAbstract(true/false) //a 是抽象的, 无法实例化
```

(3) 对象多重性约束

使用 multiplicity 函数描述。multiplicity 函数的第一个参数是源节点, 第二个参数是目标节点, 第三个参数是连接源节点与目标节点的关系类型。第四个参数是控制字段, 通过“source”和“target”两个控制字表示需要界定的是源节点还是目标节点的多重性。此外, 可以通过符号“>”、“>=”、“==”、“<”、“<=”以及逻辑符号“&&”来支持对象多重性约束中的等值、最大、最小以及范围多重性。例如:

```
1. multiplicity(a,c,b," source" ) [ >=|>|<|<=|==] 3 //在参与的 B 关系中, A 的多重性大于等于/大于/小于等于/小于/等于 2
2. multiplicity(a,c,b," source" ) >0 && multiplicity(a,c,b," source" ) <3 //范围多重性可以采用与运算符进行组合
```

(4) 点和属性多重性约束

使用 `multiplicity` 函数描述。某一类型对象和某一类型点之间的包含关系，以及某一类型图/对象/关系/角色/点与某一类型属性之间的关系是一对一的，故描述点和属性多重性约束的 `multiplicity` 函数的参数只有一个，即受约束的某一类型点或属性。例如：

```
1. multiplicity(point) [>=|>|<|<===] 3 //描述点的多重性，在包含该点的对象实例
   上，拥有的该类型点的实例数量大于等于/大于/小于等于/小于/等于 3
2. multiplicity(prop1) [>=|>|<|<===] 3 //描述属性的多重性
```

(5) 结构强制性约束

使用函数：`x.isConnectedTo(a,b,c)`进行描述。其中 `x` 表示受约束的主体元素，第一个参数 `a` 是 `x` 必须相连的源或目标对象，第二个参数 `b` 用于描述通过何种关系相连。第三个参数是控制字段，“M”代表 mandatory，即强制的，“N”代表 no，表示 `x` 一定不能通过 `b` 与 `a` 相连。例如：

```
1. A.isConnectedTo(B,C,"M"); //A 不可以单独存在，A 必须通过 C 与 B 相连
2. A.isConnectedTo(B,D,"N") //A 一定不能通过 D 与 B 相连
```

(6) 分解强制性约束和剖视强制性约束

在描述分解约束和剖视约束的函数的基础上，添加控制字段来描述“强制性”。控制字段为“M”代表强制的，即必须分解或剖视至某类型图实例；为“N”代表一定不能分解或剖视至某类型图实例。由于一个对象/点/关系/角色（OPRR）实例只允许分解至一个图实例，分解强制性约束只允许定义两个参数，即 `x.decompose(y,"M/N")`，`y` 即为分解至的图实例类型。而 OPRR 实例允许剖视至多个图实例，即 `x.explode(y1,y2,...yn,"M/N")`。例如：

```
1. A.decompose(B,"M") || A.decompose(D,"M"); //A 必须分解至 B 或者分解至 D
2. A.explode(C,D,"N"); //A 一定不能剖视至 C 和 D
```

(7) 结构唯一性约束

是特殊的多重性约束，可借用多重性部分的 `multiplicity` 函数进行约束，例如：

```
1. multiplicity(A,C,B,"source")=1//对于一个确定的 B 类型实例，A 类型实例是唯一的
```

(8) 集比较约束

子集约束、等价约束、排斥约束分别使用 `isSubSetOf`、`isEquivalentTo` 以及

isDisjointTo 函数进行描述, $x.isSubSetOf(y)$ 表示 x 是 y 的子集。用法示例如下:

```
1.  check testA(Object[A] a1,a2; Object[B] b, Object[D] d,Relationship[C] c,
    Relationship[E] e)(
2.      guard(
3.          c.connectWith(a1,b);
4.          e.connectWith(a2,d);
5.      )
6.      inv(//类型为 A 对象元模型的所有实例 a1 是类型为 B 的所有实例 a2 的子集
7.          a1.isSubSetOf(a2);
8.          a1.isEquivalentTo(a2);//a1 与 a2 是等价关系
9.          a1.isDisjointTo(a2);//a1 与 a2 是排斥关系
10.     )
11. )
```

(9) 环约束

分别使用 isReflexive()、isSymmetric()、isTransitive()以及 isCyclical()函数来描述自反性约束、对称性约束、传递性约束和不成环约束。 $x.isReflexive(true)$ 表示 x 是自反的, 若将 true 改为 false, 则是非自反的。例如:

```
1.  E.isReflexive(true);//关系 E 两端连接的节点可以是同一个
2.  E.isSymmetric(true); //关系 E 两端连接的节点具有对称性
3.  E.isTransitive(true);//关系 E 两端连接的对象具有传递性: A->B, B->C; 则 A->C
4.  E.isCyclical(false);//关系 E 两端连接的结构是不成环的
```

(10) 属性强制约束和点强制约束

使用函数 isMandatory()描述, $x.isMandatory(true)$ 表示 x 是强制存在的, 若为 false, 则是非强制存在的。例如:

```
1.  property.isMandatory(true);//属性强制性约束
2.  point.isMandatory(true);//点强制性约束
```

(11) 属性值唯一性约束

使用函数 unique()进行描述, $x.unique(true)$ 表示 x 是唯一的, 例如:

```
1.  property.unique(true);
```


5.3.2.7 fixModule 具体语法规义

fixModule 部分通过关键字 **fix** 标识，主体部分由若干个 fixClause 构成。目前支持的内置 fixClause 包括: executeExtends、executeComposite 以及 executeAggregate 函数，分别用于实施元素之间的继承关系、组合关系和聚合关系。以下给出用法示例：

```
1. fix( fixClause*)
2. executeExtends(x,y); //执行继承, 使 y 继承 x 的所有属性、端口等
3. executeComposite(x,y); //执行组合, 使 x 拥有 y 这个组成部分属性
4. executeAggregate(x,y); //执行聚合, 使 x 拥有 y 这个聚合部分属性
```

5.4 模型正确性检查插件开发

在模型正确性检查语言的基础上，本节研究模型正确性检查的插件开发。通过插件实现对模型正确性检查语言描述的约束规则进行解析，提取其中的约束上下文和约束表达式。然后基于图匹配引擎和约束求解器，对第四章中所提基于图模式匹配和一阶逻辑的模型正确性检查方法进行实现。

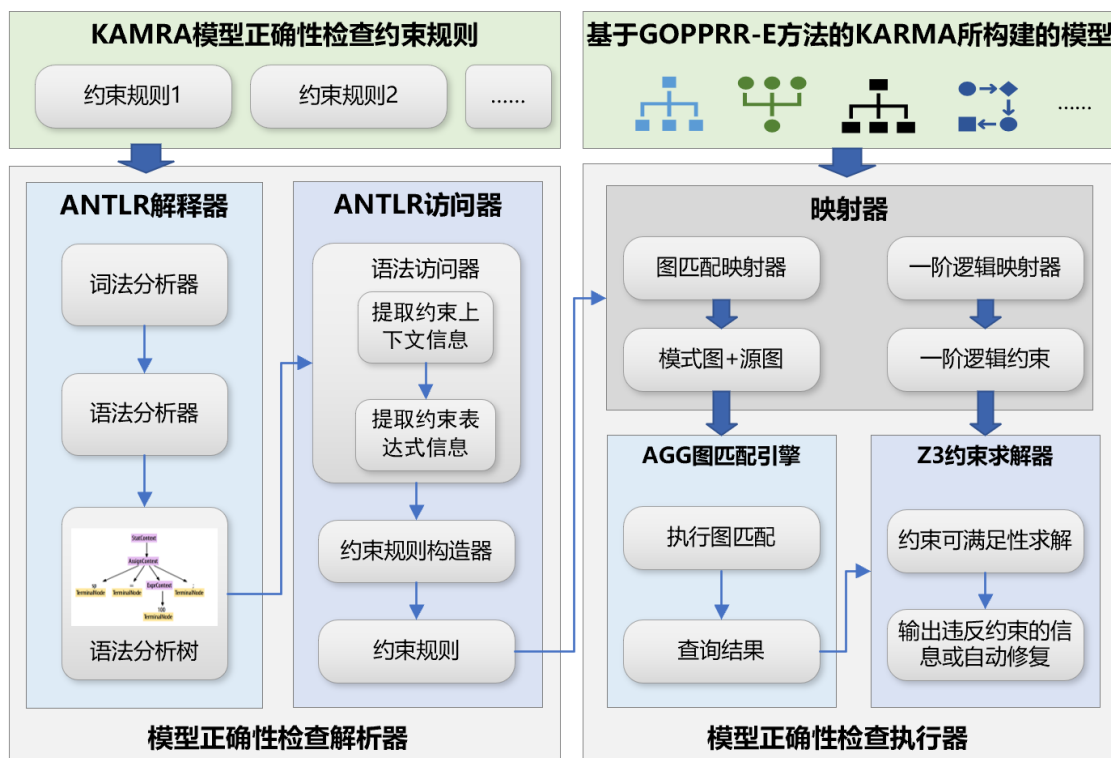


图 5.6 模型正确性检查插件开发框架

模型正确性检查插件的整体框架组成如图 5.6 所示，主要包括解析器和执行器两

个部分。其中，基于 ANTLR 技术实现解析器，基于 AGG 图匹配引擎实现模型查询，基于 Z3 求解器实现约束可满足性检查。下文首先将涉及到的技术进行介绍，然后详细描述模型正确性检查解析器和执行器的开发。

5.4.1 ANTLR 技术介绍

ANTLR (ANother Tool for Language Recognition) 是一款开源的语法分析器生成器，用于读取、处理、执行或翻译结构化的文本或二进制文件^[76]。ANTLR 为特定域语言的构建提供了一个通过语法规则来自动构造特定域语言的解析器的框架。其中，解析器包括词法分析器和语法分析器。词法分析器将分析量化输入的文本中那些毫无意义的字符流，将它们翻译成离散的字符组，即一个一个的 Token。语法分析器则将这些 Token 组织起来，按照定义的目标语言语法规则，转换成语法分析树（描述语法与输入文本之间匹配关系的树状结构）。此外，ANTLR 也支持自动生成语法分析树的访问器，用户可以通过访问器访问树中的节点，从而实现自定义的业务逻辑。

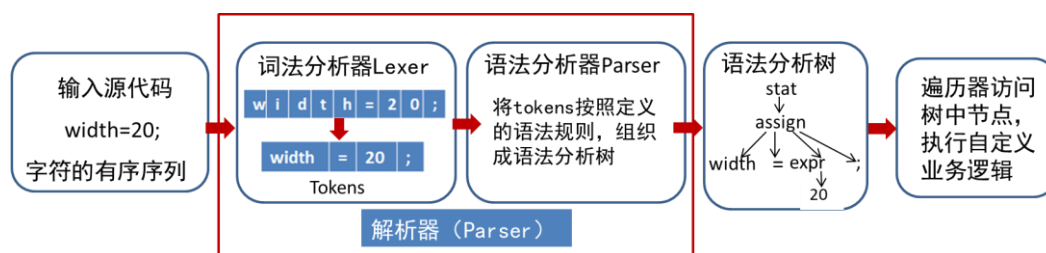


图 5.7 基于 ANTLR 的语法解析过程

5.4.2 Z3 约束求解器介绍

Z3 是一个可满足模理论求解器，被用于各种软件验证和分析应用程序^[77]。可满足模理论问题是一个关于一阶逻辑公式的决策问题，涉及到许多理论，例如算术、位向量、数组和未解释函数理论。Z3 可以用来检验一阶逻辑公式在一个或多个理论上的可满足性。以算术理论为例，假设 x 和 y 是两个数值型变量， x ， y 具有三个约束条件： $x \geq 0$ 、 $(x + y \leq 5) \vee (x + 2 * y \geq 6)$ 、 $(x + y \geq 2) \vee (x + 2 * y > 4)$ 。Z3 求解器可以求解是否存在 x 和 y 同时满足以上三个约束条件。Z3 求解器是一种底层工具，为了方便使用，Z3 提供了许多 API 工具，以便通过 Java、Python 高级编程语言使用 Z3。本文的模型正确性检查执行器即是基于 Z3 提供的 Java API 进行约束表达式的可满足性求解。

5.4.3 AGG 图匹配引擎介绍

AGG 是一个开源的图转换工具,支持使用代数方法进行属性图的转换^[78]。在 AGG 中,复杂的数据或者模型结构被构建为属性图,属性图通过类型图进行类型化。然后采用基于规则的方式实现属性图的转换,其规则由 LHS (Left Hand Side)、RHS (Right Hand Side) 和从 LHS 至 RHS 的映射组成。在应用规则进行图转换时,需要转换的图称为源图 (Source Graph),生成的图称为目标图 (Target Graph)。应用规则的第一步称为态射 (也称为匹配),即在源图中标记每一组符合 LHS 描述的图结构的顶点和边的组合,一个组合称为 LHS 的一个匹配^[79]。AGG 提供一个图匹配引擎来查询源图中 LHS 的所有匹配,本文将基于该引擎实现模型正确性检查部分的模型查询过程。

5.4.4 模型正确性检查解析器和执行器构建

5.4.4.1 模型正确性检查解析器构建

解析器的作用是将 KARMA 语言描述的约束规则识别成计算机可理解、可操作的约束规则。本文基于 ANTLR 技术构建解析器,将解析器分为两个部分:(1) ANTLR 解释器;(2) ANTLR 访问器。解释器的构建需要 g4 语法文件。根据 ANTLR 指定的语法和词法规则规范,将模型正确性检查语言的具体语法语义定义为 g4 语法文件,ANTLR 将自动根据 g4 文件生成语言的词法分析器 (lexer) 和语法分析器 (parser) 以及空的访问器。lexer 和 parser 可以将模型正确性检查语言描述的约束规则编译成语法分析树。通过重写空的访问器实现对语法分析树中约束上下文和约束表达式信息的提取,并输入至约束规则构造器,构造成计算机可操作的约束规则。

5.4.4.2 模型正确性检查执行器构建

执行器的作用是执行解析器部分得到的约束规则,获得不满足规则的错误信息,本节基于 AGG 和 Z3 求解器构建执行器,包括三个部分:(1) 映射器;(2) AGG 图匹配引擎;(3) Z3 约束求解器。首先将约束规则和基于 GOPRR-E 方法的 KARMA 语言所构建的模型输入映射器,映射器将提取约束规则中的约束上下文信息,将其中定义的模型元素组合转换为模式图,并将 KARMA 语言所构建模型转换为源实例图。然后将模式图和所有源实例图输入至 AGG 图匹配引擎。同时映射器将提取约束规则中的约束表达式,将其转换为一阶逻辑公式并输入 Z3 约束求解器。最后基于 AGG 执

行图匹配，获得查询结果输入至 Z3，Z3 首先判断约束规则种是否存在冲突，然后执行约束可满足性求解，获取查询结果中违反规则的信息并输出。此外，若在约束规则中定义了自动修复模块，则根据修复函数自动对模型进行错误的修正。

5.4.4.3 模型正确性检查插件的集成

为了实现 KARMA 语言所构建多领域架构模型的正确性检查，将模型正确性检查插件集成至多架构建模平台 MetaGraph^[80]中。如图 5.8 所示，MetaGraph 是中科蜂巢科技有限公司联合北京理工大学系统工程团队开发的、用于支持 KARMA 语言的建模和基于模型的操作等功能的设计平台。MetaGraph 能同时支持 KARMA 语言所构建模型的图形化显示和文本式描述与存储。本文将模型正确性检查插件集成至 MetaGraph 中，实现了多领域架构模型构建、模型正确性检查的平台一体化。

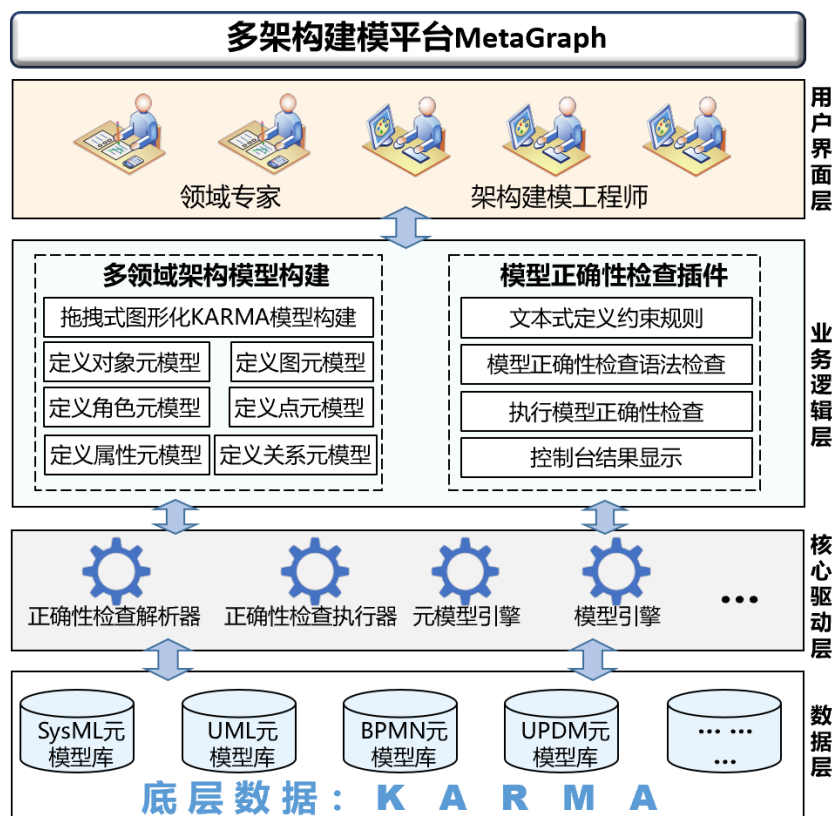


图 5.8 MetaGraph 集成模型正确性检查插件

基于 MetaGraph 实现 KARMA 模型的正确性检查的流程如图 5.9 所示。首先在 MetaGraph 前端，在元模型配置框中定义 GOPRRR 元模型，MetaGraph 将根据配置框中的信息生成描述元模型的.karma 文件。基于.karma 文件，在 KARMA 文本编辑界面

基于模型正确性检查部分语言定义约束规则。然后在工具后端执行检查，以约束规则和模型为输入，通过检查解析器进行解析得到约束规则。然后通过检查执行器执行得到不满足约束规则的模型信息，并将信息输出至前端的结果输出界面。

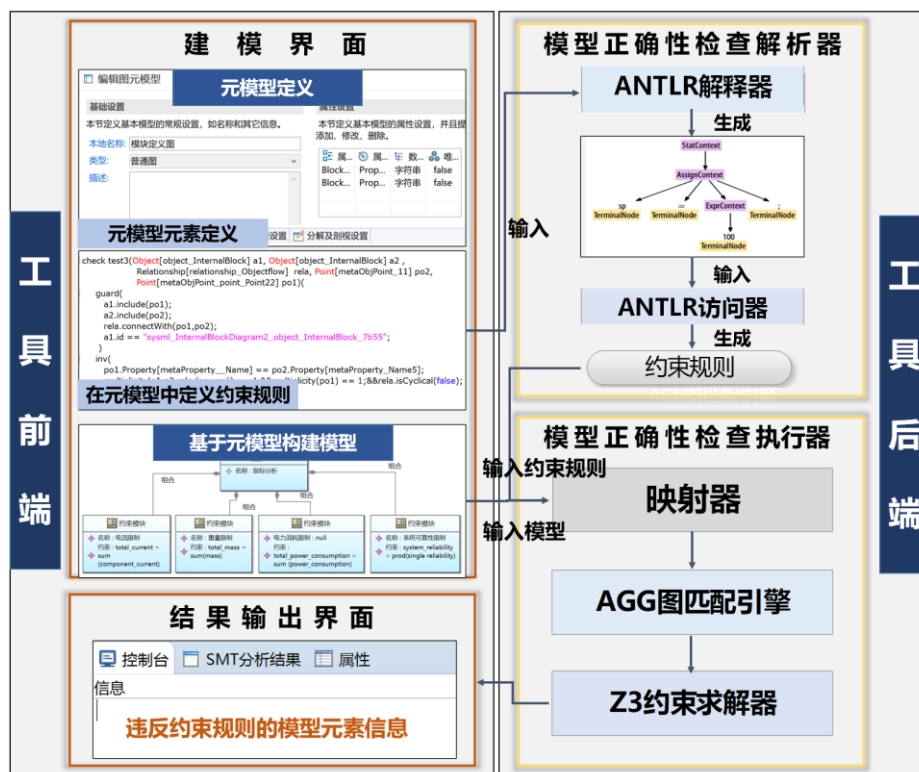


图 5.9 基于 MetaGraph 的模型正确性检查流程

5.5 模型正确性检查语言应用案例

以 4.5 中的 UML 温度控制系统活动图为例，说明如何使用模型正确性检查语言构建约束规则并执行检查。首先采用模型正确性检查语言描述规则“若分支节点的输入是与栓连接的对象流，那它的输出流不能是与动作连接的控制流”。如下所示，在 UML 活动图元模型的 Constraint 部分，基于 check 模块定义了一个约束规则 ruleA。在 ruleA 的头部定义了五个参数：po1、obj1、ro1、ro2 和 rel，并在 guard 中通过二元关系 bindWith 和三元关系 endWith，表示约束上下文为所有符合“栓一对对象流一分支节点”的模型结构。此外，在 pre 中定义了类型为控制流关系元模型的参数 rel2 和类型为动作对象元模型的参数 obj2。在 inv 中通过结构强制性约束表示当 obj 的输入流是对象流时，它不能在输出时通过控制流与动作相连。

```

1. Graph UML_ActivityDiagram
2. .....
3. Constraint(
4.     check ruleA(Object[Pin] po1, Object[ForkNode] obj1, Role[Ro2_F] ro1,
        Role[Ro2_T] ro2, Relationship[oFlow] rel)
5.     pre( Relationship[cFlow] rel2; //类型为控制流关系元模型的参数
6.         Object[Action] obj2; //类型为控制流关系元模型的参数 )
7.     guard( //描述“栓—对象流—分支节点”构成的模型结构
8.         po1.bindWith(ro1);
9.         obj1.bindWith(ro2);
10.        rel.endWith(ro1, ro2);
11.    )
12.    inv(obj1.isConnectTo(obj2, rel2, "N")); //结构强制性约束
13. )
14. );
15. end UML_ActivityDiagram

```

然后，在 MetaGraph 工具中采用 KARMA 语言构建温度控制系统活动图模型并执行检查。如图 5.10 所示，首先点击模型正确性检查执行按钮，然后选择 ruleA 所在的 UML_ActivityDiagram.karma 文件，点击确认。经模型正确性检查插件执行后，控制台显示了一条错误信息，如图 5.11 所示。从错误信息中可以发现，图中的分支节点输入流为栓传递过来的对象流，携带对象令牌的信息（在该案例中为温度），但输出流为控制流，不携带对象令牌信息，即对象令牌信息无法正确传递至下游活动。

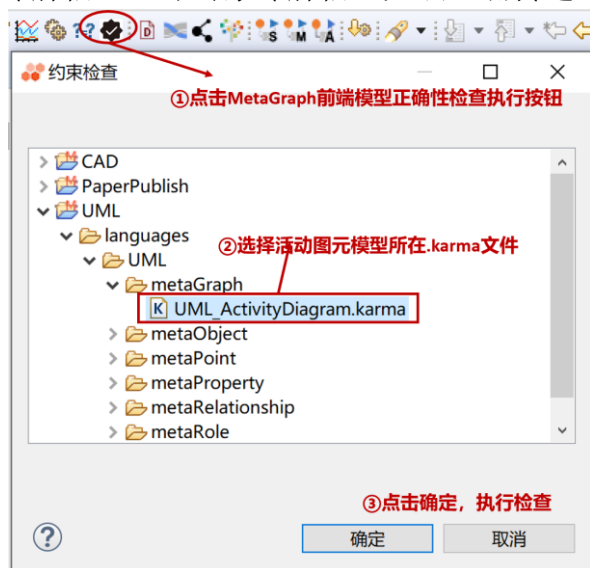


图 5.10 在 MetaGraph 中执行模型正确性检查示例

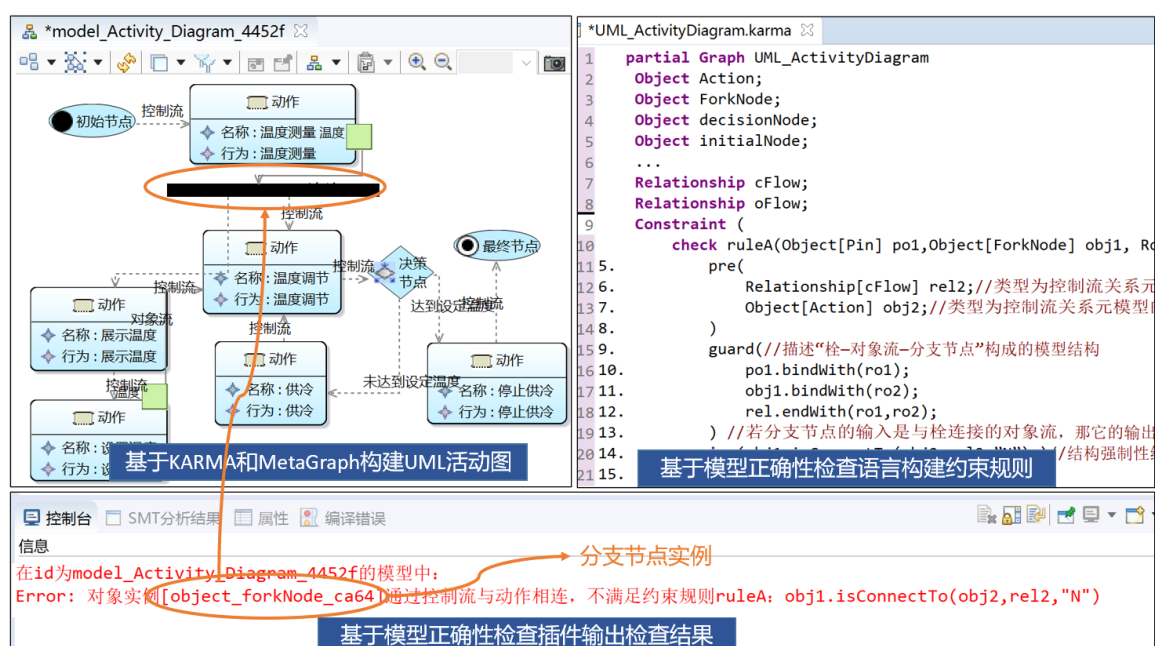


图 5.11 模型正确性检查语言实现约束规则的构建和执行示例

5.6 本章小结

本章在第四章的基础上设计并实现模型正确性检查语言，为系统架构模型正确性检查的实施提供媒介。首先介绍了 KARMA 语言的抽象语法结构和语义风格，然后扩展了 KAMRA 语言的元模型类，添加了用于描述检查约束规则的抽象语法。在此基础上，设计了模型正确性检查语言的具体语法语义。最后基于 ANTLR、AGG、Z3 等技术构建了模型正确性检查的插件，并将其集成至支持多领域系统架构模型构建的 MetaGraph 工具中，实现了模型构建以及正确性检查的平台一体化。

第6章 供配电系统的模型正确性检查案例

6.1 引言

本章以二级运载火箭供配电系统为对象，验证前文所提出的约束规则体系、模型正确性检查方法和语言的有效性。首先介绍供配电系统的设计背景，然后定义用于供配电系统架构模型正确性检查的所有约束规则，包括：（1）梳理供配电系统元模型构建阶段中有关范式级别和建模领域级别的约束规则；（2）基于 RFLP 方法构建供配电系统架构模型，并梳理模型构建阶段中目标建模对象级别的约束规则。最后，采用模型正确性检查方法和语言对所梳理的约束规则进行描述并执行正确性检查，以验证上述研究所提方法和语言的有效性。

6.2 运载火箭供配电系统设计背景

供配电系统是运载火箭电气系统的重要组成部分，它的主要功能是为全箭（包括运载火箭一子级和二子级）电气设备提供工作电源，从而保证用电设备正常工作^[81]。随着航空航天技术的快速发展，运载火箭电气系统复杂程度和一体化程度越来越高，这给供配电系统的集成化、智能化、快速化设计带来了更高的要求。



图 6.1 运载火箭供配电系统（局部）

供配电系统需要为全箭用电单机供电，因此与其他系统之间存在大量交互关系，涉及众多交互接口。此外，在新一代运载火箭的研制过程中，在不断地创新和运用更先进的供配电技术，例如固态配电技术。新技术的研发和运用需要与多个交互系统进

行沟通，并通过多轮迭代确定最终的技术方案。若以传统的基于文档的系统工程方式进行供配电系统的研制，将存在以下两个问题：（1）数据传递效率低、一致性差。在传统的研制模式下，将产生大量与其他系统用电电机之间的设计输入输出文档和接口数据。当涉及到数据变更时，需要大量的手动工作。（2）难以在设计早期验证供配电系统设计的正确性。通常在后期实物验证阶段对系统设计进行验证，难以做到尽早发现错误，从而难以满足快速迭代设计的要求。

改案例采用 MBSE 方法进行供配电系统的正向设计，通过基于模型的方式提高设计过程中数据传递效率。此外，采用系统架构模型正确性检查方法和语言，实现尽早地发现以模型为载体的早期系统设计中的不规范、不一致等错误，降低早期设计中错误传递到设计后期的隐患，从而提高供配电系统的设计效率。

6.3 供配电系统架构模型约束规则的定义

本节依据第三章所构建的约束规则体系，确定供配电系统元模型和架构模型构建阶段中的所有约束规则，以支持后续的模型正确性检查。首先介绍所使用的建模工具和建模语言，根据所选择建模语言和供配电系统所在领域确定范式级别和建模领域级别的约束规则。然后基于 RFLP 方法进行供配电系统架构模型的构建，并梳理目标建模对象级别的约束规则，形成完整的、面向供配电系统架构模型的约束规则。

6.3.1 建模工具和建模语言

该案例采用的建模工具为 MetaGraph，采用的元建模语言为 KAMRA，采用的建模语言为 SysML。供配电系统架构建模所使用到的 SysML 中的图类型包括：（1）需求图，用于需求分析；（2）用例图、活动图，用于功能分析；（3）模块定义图、内部模块图、包图、参数图，用于逻辑设计和物理设计。然后采用 KARMA 语言，按照 GOPRR-E 元建模方法将上述所需的图类型在 MetaGraph 工具中构建出来。如图 6.2 所示，以 SysML 需求图为例，采用 KARMA 语言将需求图描述为需求图元模型。在 KARMA 语言描述的文本式需求图元模型中明确了其拥有的正确的元素以及元素之间的正确关系（例如需求图元模型中包括需求对象元模型、用例对象元模型等、以及需求对象元模型之间的关系可以是包含关系元模型、追溯关系元模型等）。

元素	含义	固定语法	<pre> partial Graph Requirement_Diagram initial PTo_Do_c1fbaf = "null" extends To_Do annotation(localLabel = "待办事项"); initial Context_9e54db = "null" extends Context annotation(localLabel = "背景"); Object Object_Requirement; Object Object_Extended_Requirement; Object Object_Business_Requirement; Object Object_Usability_Requirement; Object Object_Functional_Requirement; Object Object_Interface_Requirement; Object Object_Performance_Requirement; Object Object_Physical_Requirement; Object Object_Design_Constraint; Object Object_Constraint_Block; Object Object_Use_Case; Object Object_Safety_Requirement; Relationship Relationship_Satisfy; Relationship Relationship_Derive; Relationship Relationship_Copy; Relationship Relationship_Trace; Relationship Relationship_Verify; Relationship Relationship_Refine; Relationship Relationship_Contain; Constraint (bind connector(Object_Business_Requirement, Relationship_Satisfy.Role_Client), bind connector(Object_Interface_Requirement, Relationship_Satisfy.Role_Client), Object_Requirement.explode(Requirement_Diagram), Object_Extended_Requirement.explode(Requirement_Diagram), Object_Physical_Requirement.explode(Requirement_Diagram),); annotation(localLabel="需求图", type = diagram, description="",); end Requirement_Diagram; </pre>
对象元模型	包	具备共同特征的元素集合	
	用例	场景活动的抽象描述	
	约束模块	需求元素满足的约束条件	
	需求	利益相关者相关需求	
	接口需求	有关接口方面的需求	
关系元模型	包含	需求的分解	
	追溯	弱依赖关系	
	复用	需求的继承关系	
	改善	需求的细化	
	满足	模块满足需求	
	验证	测试用例满足需求	

图 6.2 基于 KARMA 描述的图形式和文本式需求图元模型

6.3.2 元模型构建阶段约束规则梳理

依据第三章中面向模型正确性检查的约束规则体系，在元模型构建阶段中关注的是范式级别和建模领域级别的检查内容，以使所构建的模型能遵循建模语言的正确语法且不违反约定俗成的建模惯例和领域约束。在范式级别，定义建模语言阶段关注的正确性内涵为：模型中包含正确的元素及元素之间的正确关系，该内涵通过 KARMA 语言模型描述部分的语法进行描述，并在使用 KARMA 语言定义 SysML 元模型的过程中实现。因此，无需再通过额外的约束规则进行描述和检查。因此，下文梳理的范式级别和建模领域级别的约束规则面向的模型正确性内涵为：模型不违反建模规则和惯例（简称为①）、模型正确地描述了所建模的领域（简称为②）。

本章的建模领域为航空航天领域，结合 Berenbach^[39]、Delligatti^[9]的等人对 SysML 语言相关建模惯例和规则研究，以及航天领域特点和需求工程标准化要求，梳理范式级别（简称为 F）和建模领域级别（简称为 D）的约束规则如表 6.1 所示。

表 6.1 范式级别和建模领域级别约束规则

SysML 图	编号	级别	约束规则	正确性内涵
需求图	1	F	每一个需求有且仅有一个名称，名称的值在该需求图中是唯一的。	①
	2	F	每一个需求都需要被模块满足。	①
	3	D	依据需求工程标准化，每一个需求都需具有文本描述以描述需求的内容。	②
	4	D	依据需求工程标准化，每一个需求都需有来源。	②
	5	D	依据需求工程标准化，每一个需求都需有验证方法。	②
	6	D	依据需求工程标准化，每一个需求都需有确认方法。	②
用例图	7	F	每一个用例以及抽象用例都必须有且仅有一个名称，且名称的值唯一。	①
	8	F	每一个模块都需要关联到至少一个用例，以确保所建模用例涵盖了与所有上下文模块的交互活动。	①
	9	F	每一个用例都至少需要剖视到一个活动图以描述该用例相关的场景。	①
	10	F	抽象的用例需要通过继承的方式被其他用例实现。	①
活动图	11	F	每一个泳道都具有类型，该类型为模块或者参与者。	①
	12	F	每一个泳道中的所有动作将分配给该泳道代表的模块，表示该模块具有这些动作表示的功能活动。	①
	13	F	每一个调用行为动作和调用操作动作都有一个类型，该类型为活动。	①
	14	F	每一个的栓都有类型，该类型为模块或者值类型。	①
	15	F	每一个活动参数都有一个类型，该类型为模块或者值类型。	①
	16	F	每一对通过对象流相连的栓是兼容的，即两个栓的类型是相同的。	①
	17	F	初始节点没有输入边。	①

续表 6.1

SysML 图	编号	级别	约束规则	正确性内涵
活动图	18	F	最终节点没有输出边。	①
	19	F	决定节点只有一个输入边。	①
	20	F	合并节点有两条及以上的输入边且只有一条输出边。	①
	21	F	分支节点必须拥有一条输入边和两条以上输出边。	①
	22	F	集合节点有两条及以上输入边，而只有一条输出边。	①
模块定义图	23	F	每一个模块有且仅有一个名称且名称的值是唯一的。	①
	24	F	每一个约束模块有且仅有一个名称且名称的值唯一。	①
	25	F	模块 A 与模块 B 为组合关系时，表示 A 具有 B 这个组成部分属性，且 A 有掌握 B 生死的权力。	①
	26	F	组合关系连接的模型结构不能成环。	①
	27	F	模块 A 与模块 B 为聚合关系时，表示 A 具有 B 这个聚合部分属性，A 无掌握 B 生死的权利。	①
	28	F	聚合关系连接的模型结构不能成环。	①
	29	F	模块 A 与模块 B 之间为泛化关系，表示 B 继承了 A 的所有属性、行为和端口。	①
	30	F	模块之间的泛化关系描述的模型结构不能成环。	①
	31	D	航空航天领域，允许模块之间的多继承。	②
	32	F	模块上的代理端口具有类型，其类型是接口模块。	①
内部模块图	33	F	模块上的完整端口具有类型，其类型是模块。	①
	34	F	每一个内部模块都具有类型，其类型是模块。	①
	25	F	每一个内部模块有且仅有一个名称且名称的值唯一。	①
	36	F	每一对通过连接器连接的端口之间是兼容的。	①
	37	F	每一个端口和其连接的连接器是兼容的。	①
包图	38	F	每一个包有且仅有一个名称，且名称的值是唯一的。	①
参数图	39	F	约束属性具有类型，其类型是约束模块。	①

6.3.3 基于 RFLP 方法构建供配电系统架构模型

在基于 KARMA 语言定义了 SysML 元模型并梳理了元模型构建阶段中约束规则的基础上，本小节基于 RFLP 方法进行供配电系统架构模型的构建。RFLP 方法由系统层-分系统层-单机层自顶向下进行正向分解和分析，该案例的建模对象为供配电系统，实质上是一个分系统，因此其建模过程只涉及分系统层和单机层。基于 RFLP 的供配电系统架构模型的构建流程如图 6.3 所示（图中 ABCD...字母的顺序代表模型构建的顺序），以下将简要介绍各个建模阶段构建的主要模型。

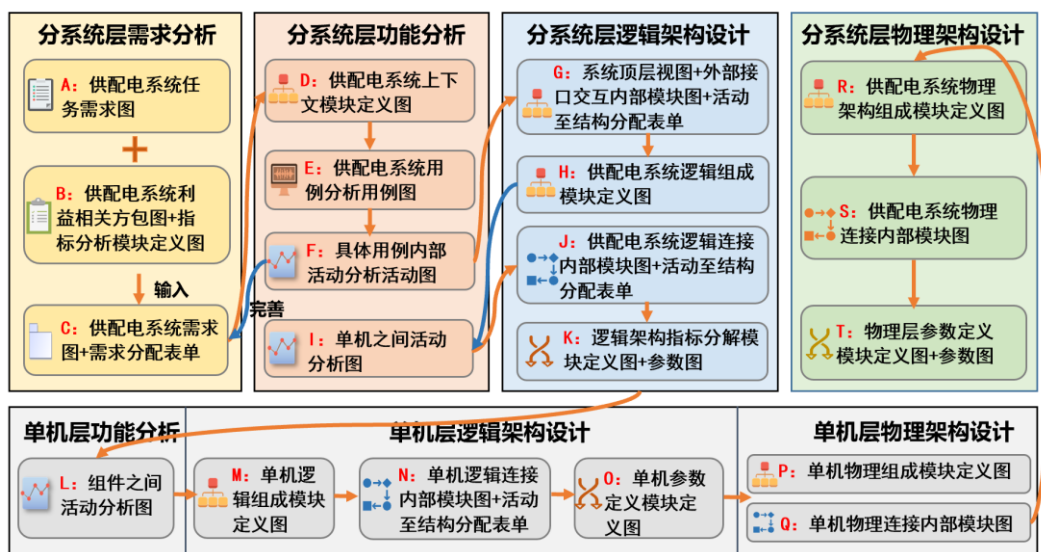


图 6.3 供配电系统建模流程

6.3.3.1 分系统层需求分析

供配电系统的设计任务来自上层电气系统下发的《供配电系统设计任务书》，根据设计任务书得出任务需求，构建**供配电系统任务需求图**如图 6.4 所示。然后对任务需求进行初步分配与分解，构建任务需求至系统需求的直接分配表单。并根据需求工程的标准和要求，形成初步的供配电系统需求。然后通过需求确认对系统需求进行完善：需求分析阶段构建**利益相关方包图**完成利益相关方的确认；构建**指标分析模块定义图**完成性能需求的确认。最后，形成**供配电系统需求图**如图 6.5 所示。

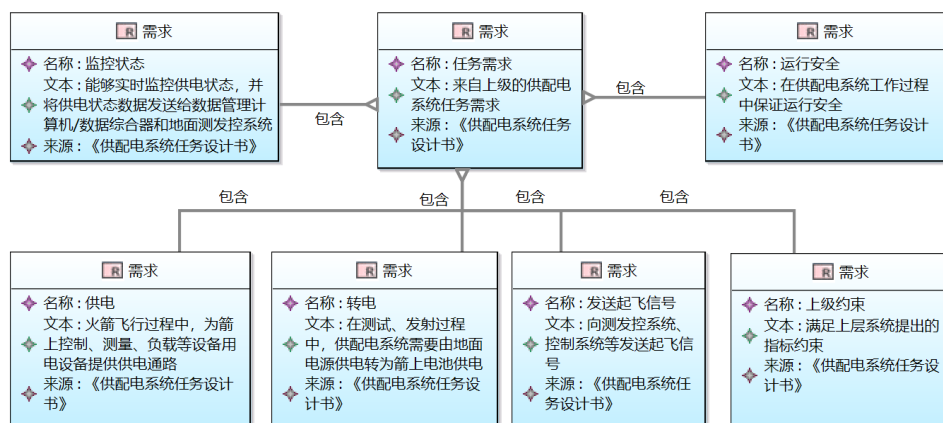


图 6.4 供配电系统任务需求图

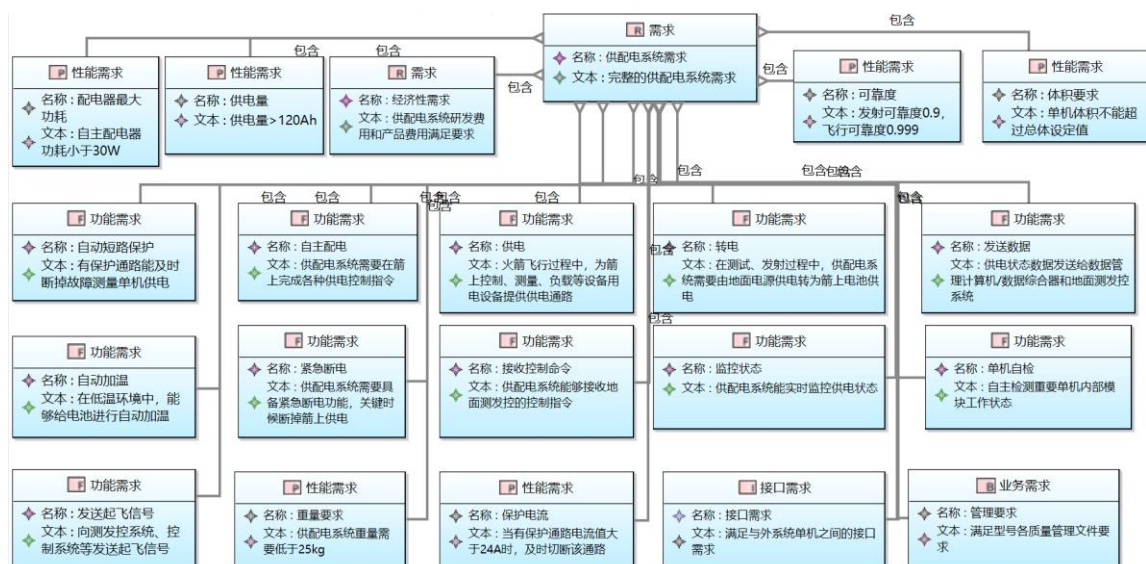


图 6.5 供配电系统完整需求图

6.3.3.2 分系统功能分析

首先分析供配电系统设计域中所有上下文系统，构建供配电系统上下文模块定义图，如图 6.6-A 所示。然后识别供配电用例场景，定义供配电系统用例分析图，描述上下文系统与用例之间的关联关系。如图 6.6-B 所示，供配电系统包括六个用例：自动短路保护、紧急断电、状态监控、转电、发送起飞信号和供电。这六个用例是系统的顶层功能。最后针对每个用例，构建具体用例的内部活动分析图，如图 6.7 所示。

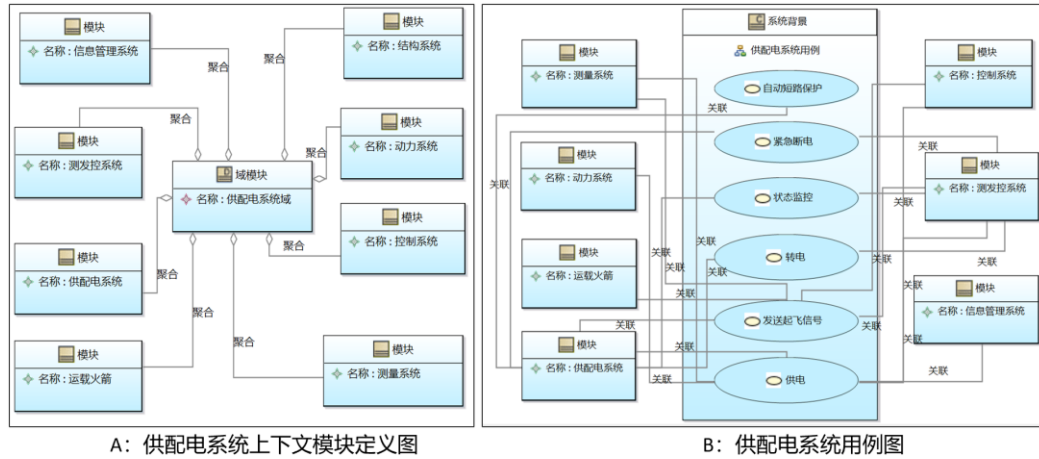


图 6.6 供配电系统上下文和供配电系统用例图

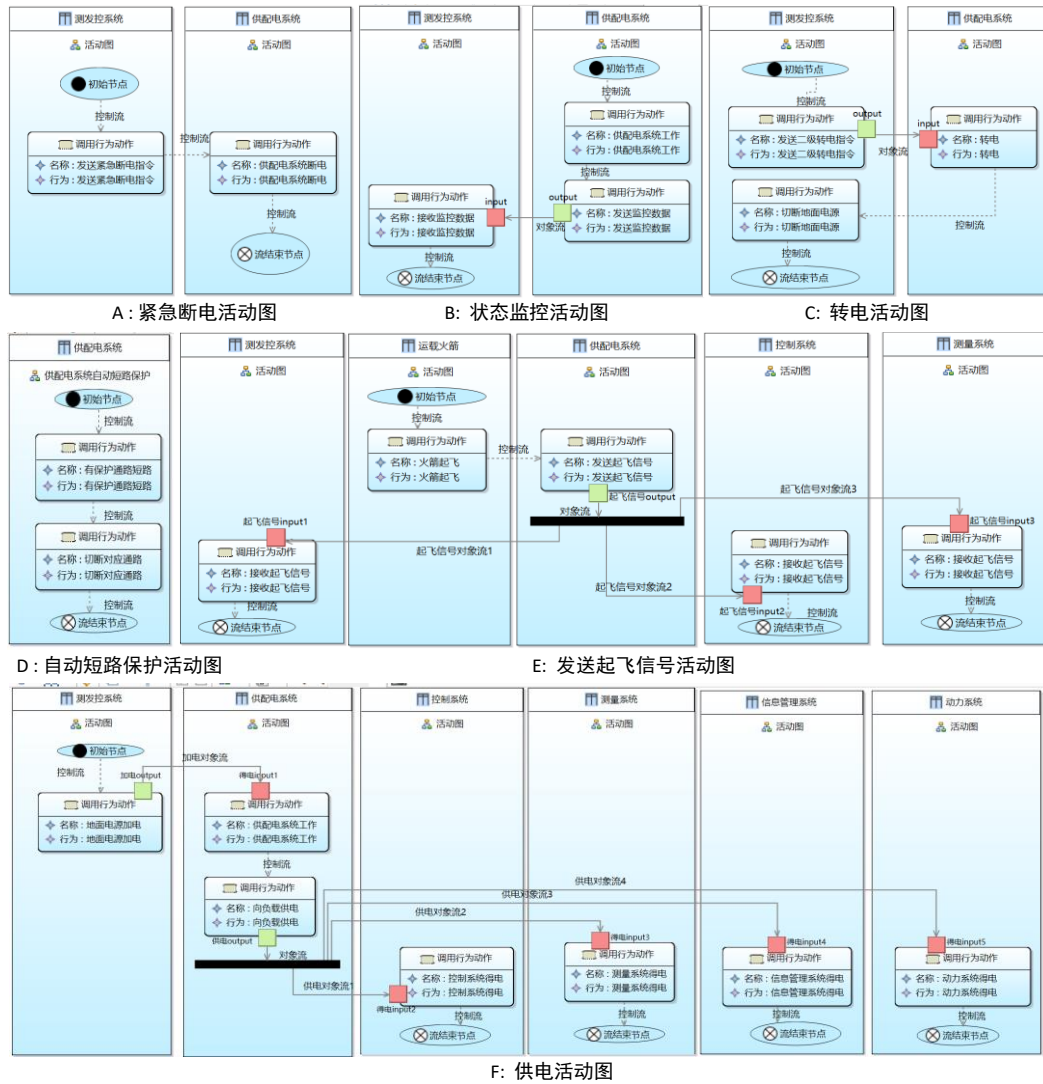


图 6.7 描述所有用例场景的活动分析图

经过用例分析后，在逻辑架构分析阶段可以将用例代表的系统顶层功能分配给逻辑组成，得出初步的逻辑架构。逻辑架构分析还需明确单机之间的交互活动，因此需要进行功能细化，分析单机之间的活动。图 6.8 和图 6.9 展示了**紧急断电以及状态监控功能的细化与分配的活动图**。以状态监控功能为例，将状态监控细分为供电状态监测和电池温度监测。一级和二级自主配电器监测各用电单机供电状态，并将状态信息通过电缆网传输。此外，一级和二级自主配电器通过电缆网分别接收一级电池和二级电池的温度，并判断温度是否过低，若过低则对电池进行加温。

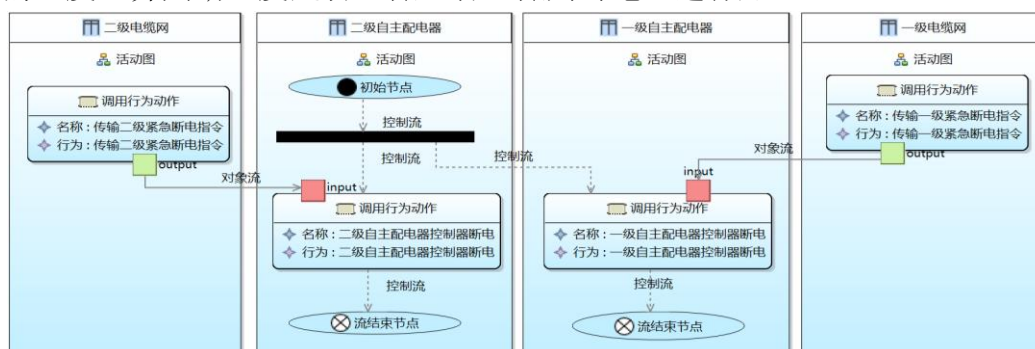
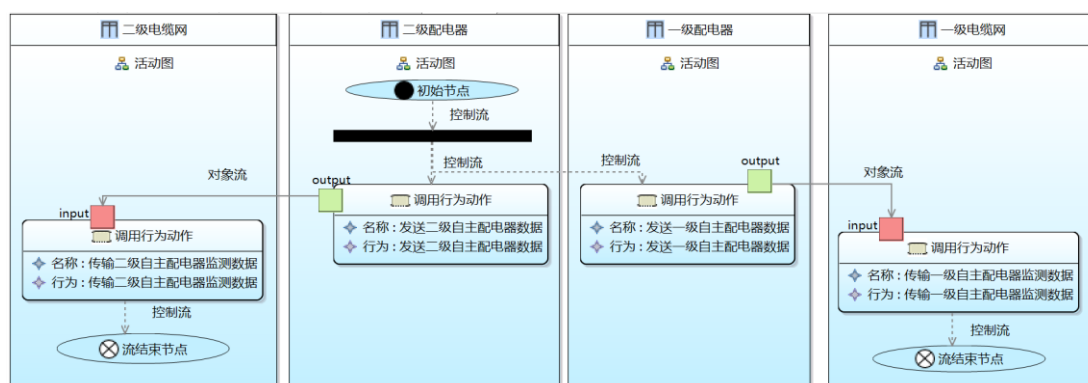
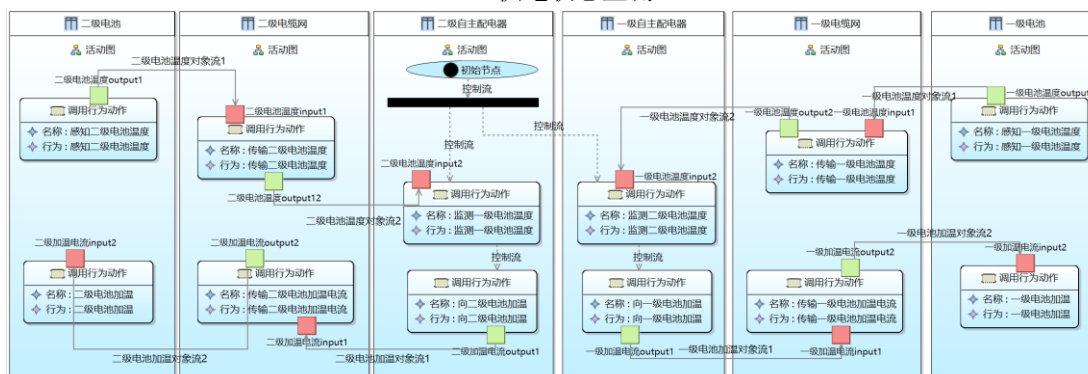


图 6.8 紧急断电功能细化与分配



A: 供电状态监测



B: 电池温度监测

图 6.9 状态监控功能细化与分配

6.3.3.3 分系统逻辑架构分析

首先基于功能分析部分的活动图，得出供配电系统与其他外部系统之间的交互关系，建立具有接口的供配电系统顶层视图、外部接口交互内部模块图以及活动至结构分配表单，如图 6.10、图 6.11 和图 6.12 所示。

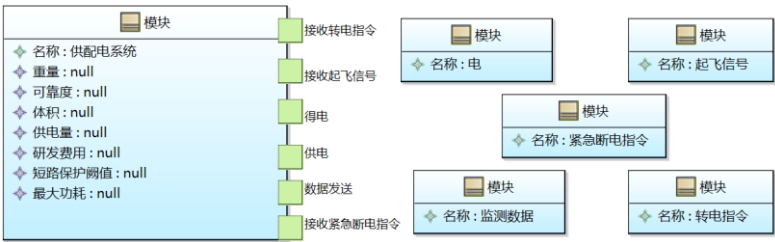


图 6.10 供配电系统顶层视图

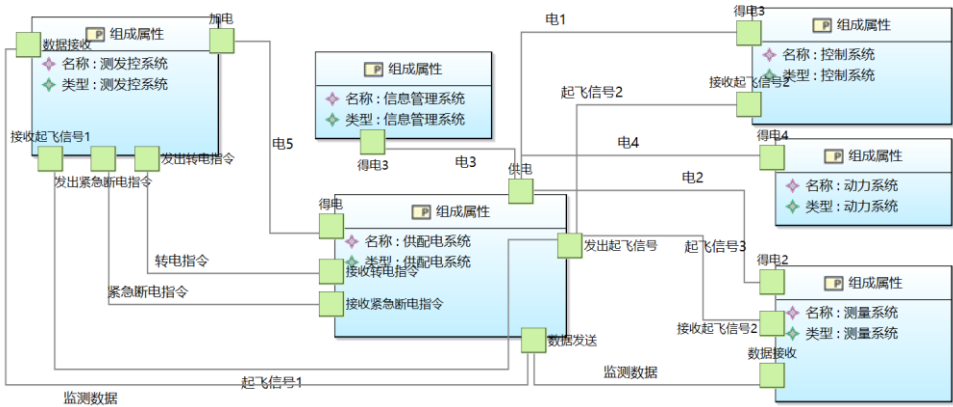


图 6.11 外部接口交互内部模块图

	外部接口交互内部模块图中代理端口元素						
供电活动图中栓元素	供电	得电	加电	得电3	得电2	得电4	得电3
加电output			分配				
得电input3				分配			
得电input1		分配					
供电output	分配						
得电input2					分配		
得电input4						分配	
得电input5							分配

A：供电活动图至外部接口交互内部模块图中元素的分配

列	外部接口交互内部模块图中代理端口元素			
发送起飞信号活动图中栓元素	接收起飞信号1	发出起飞信号	接收起飞信号2	接收起飞信号3
起飞信号output		分配		
起飞信号input1	分配			
起飞信号input3				
起飞信号input2			分配	

B：发送起飞信号活动图至外部接口交互内部模块图中元素的分配

图 6.12 活动至结构分配表单（部分）

然后结合二级运载火箭供配电系统历史型号参考,构建供配电系统的逻辑组成模块定义图,如图 6.13 所示,其逻辑组成包括电池、电缆网和配电器,并进一步分为一级和二级电池、一级和二级电缆网、一级和二级配电器(分别用于支持火箭一二级的供配电)。然后基于功能分析部分的单机之间行为活动,构建供配电系统逻辑连接内部模块图,如图 6.14 和 6.15 所示。最后,使用模块定义图和参数图对供配电系统的逻辑层指标进行分析,将指标分解至单机中。

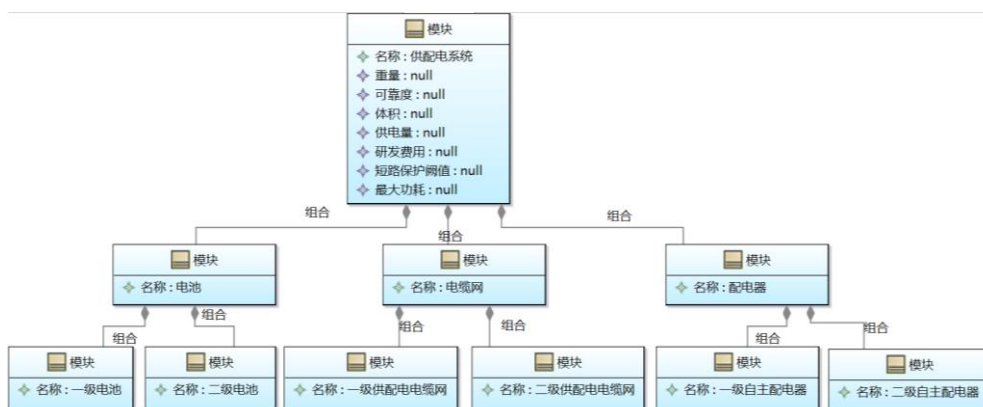


图 6.13 供配电系统逻辑组成模块定义图

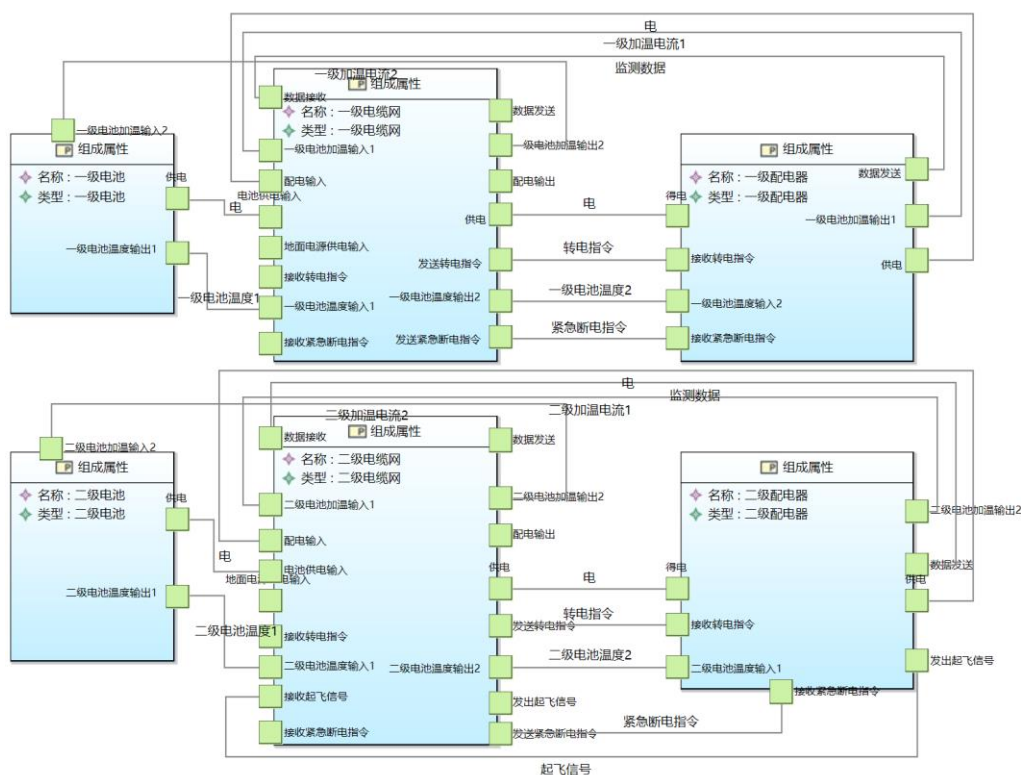


图 6.14 供配电系统逻辑连接内部模块图

	供电配电系统逻辑连接内部模块图中代理端口元素							
电池温度监测活动图中栓元素	一级电池温度输出1	一级电池加温输入2	二级电池温度输出1	二级电池加温输入2	一级电池温度输入2	一级电池加温输出1	二级电池温度输入1	
一级电池温度input2					分配			
一级加温电流output1						分配		
一级电池温度input1								
一级电池温度output2								
一级加温电流input1								
一级加温电流output2								
二级电池温度input1							分配	
二级电池温度output12								
二级加温电流input1								
二级加温电流output2								
一级电池温度output1	分配							
一级加温电流input2		分配						
二级电池温度output1			分配					
二级加温电流input2				分配				
二级加温电流output1								
二级加温电流output2								
二级电池温度input2								

图 6.15 活动至结构分配表单（部分）

6.3.3.4 单机层功能分析、逻辑和物理设计

确定供电配电系统完整的物理架构首先需要确定每个单机的物理选型，因此进行分系统逻辑分析后，进行单机层的功能分析、逻辑和物理设计。最后进行物理综合，形成分系统物理架构。电池是成熟单机，其逻辑组成是确定的。当电池的型号确定后，内部组件的型号及其内部连接关系也确定了，图 6.16 即 **DC0 电池的物理组成模块定义图**。自主配电器是新研制单机，需要确定单机内部逻辑组成、组件之间的功能活动以及活动至结构的分配关系，然后进行物理选型，形成最终的自主配电器物理设计。一级自主配电器的逻辑组成包括：加温继电器、紧急断电继电器、FPGA、转电继电器以及供电继电器。二级自主配电器在一级的基础上增加了起飞信号继电器。图 6.17 展示了二级自主配电器组件之间活动分析图（部分）。基于逻辑组成和功能活动确定自主配电器的物理组成模块定义图以及物理连接内部模块图如图 6.18 和图 6.19 所示。

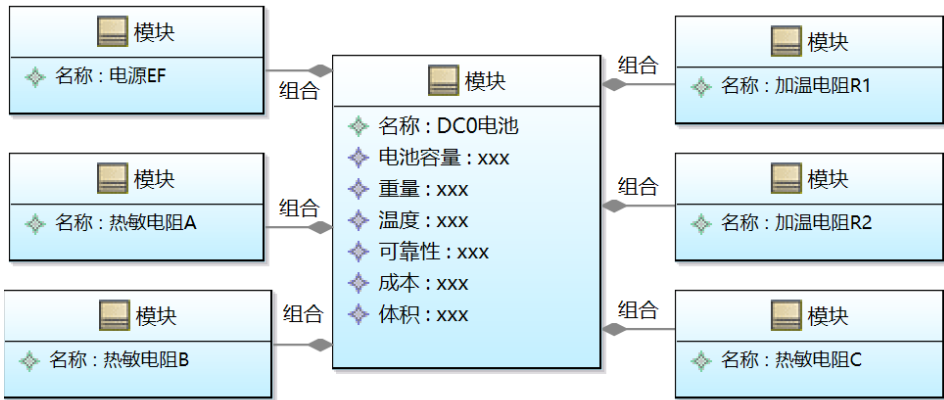


图 6.16 DC0 电池的物理组成模块定义图

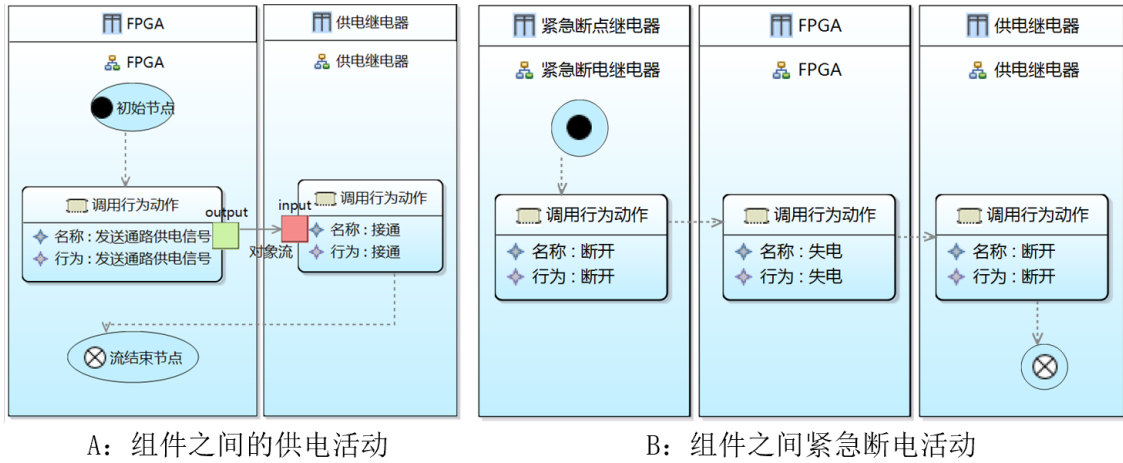


图 6.17 组件之间活动分析图（部分）

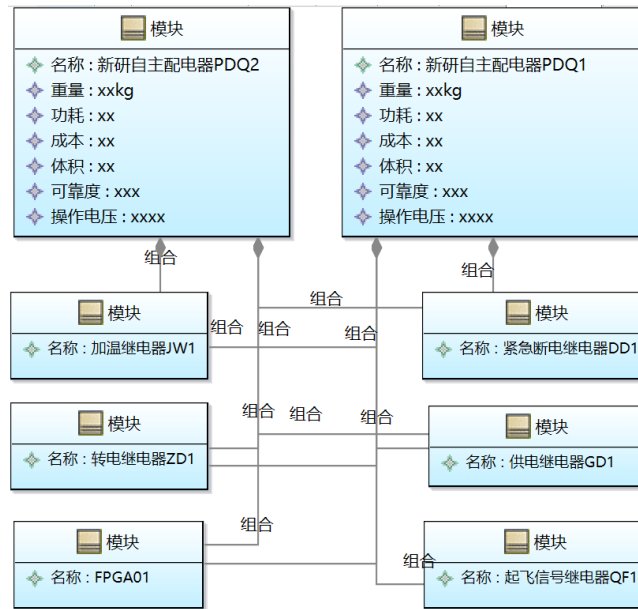


图 6.18 一级和二级自主配电器物理组成

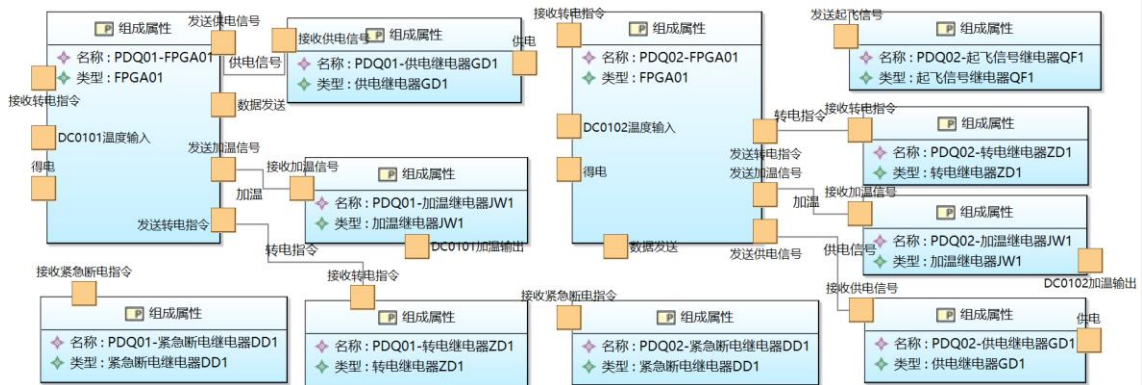


图 6.19 自主配电器物理连接内部模块图

在确定了单机层物理组成的基础上,如图 6.20 所示,构建供配电系统物理组成模块定义图描述供配电系统的物理架构。此外、构建物理单机之间的接口连接内部模块图,如图 6.21 所示。

图 6.20 供配电系统物理组成模型定义图

图 6.21 供配电系统物理组成内部接口交互图

6.3.4 供配电系统架构模型构建阶段的约束规则梳理

在采用 RFLP 方法构建供配电系统架构模型的基础上，梳理第三章所提约束规则体系中目标建模对象级别（简称为 T）的约束规则。此案例的目标对象为供配电系统，结合供配电系统设计任务书中的约束，梳理供配电系统架构模型中有关：模型正确地描述了所建模的领域、普通跨图一致性、垂直一致性和水平一致性等方面的约束规则（分别简称为②、③、④、⑤），如表 6.2 所示。

表 6.2 目标建模对象级别约束规则

供配电系统模型	编号	约束规则	正确性内涵
供配电系统逻辑组成模型和供配电系统物理组成模型定义图。	40	供配电系统的重量属性值不高于 25kg；可靠性属性值不低于 0.999；供电量属性值不低于 120Ah；研发费用属性值不高于 50w；最大功耗属性值不高于 30w；短路保护阈值属性值为 24A。	②
任务需求图和系统需求图	41	当任务需求中需求直接分配给系统需求中的需求时，具有分配关系的两个需求的内涵是一致的，表示不同层级间需求的垂直一致性。	④
供配电系统用例图和供配电系统上下文图	42	供配电系统用例图中的模块集合应该等价于供配电系统上下文模块定义图中模块集合，表示用例的构建考虑了所设计系统与所有上下文模块的交互活动。	③
供配电系统用例图和每个用例剖视的活动图	43	供配电系统用例图中所有用例剖视得到的活动图中所有泳道的类型集合，与所有用例关联的模块集合是等价的。表示描述每个用例的场景的活动图都完整的涵盖了与该用例关联的上下文模块之间的交互活动。	③
活动图和内部模块图	44	当活动图中动作（Action）中的栓与内部模块图中内部模块中的端口具有分配关系时，栓传输的事物类型与端口传输的事物类型是一致的，表示系统功能活动至逻辑组成信息传递的一致性。	⑤

续表 6.2

供配电系统模型	编号	约束规则	正确性内涵
外部接口交互图 和逻辑层内部接口交互视图	45	当外部接口交互图中的接口分配给逻辑层的内部组成上的接口时，具有分配关系的两个接口的特征是一致的。	④
逻辑层内部接口交互视图和单机层接口交互视图	46	当逻辑层内部接口交互视图中的接口分配给单机层接口交互视图上的接口时，具有分配关系的两个接口的特征是一致的。	④
分系统物理组成模型与分系统逻辑组成模型	47	分系统物理组成模型中的各个模块是对分系统逻辑组成中各个模块的物理实现，两者之间具有继承和细化的关系。	④
单机物理组成模型和单机逻辑组成模型	48	单机物理组成模型中的各个模块是对单机逻辑组成中各个模块的物理实现，两者之间具有继承和细化的关系。	④

6.4 模型正确性检查的实施

以 6.3 节中梳理的若干条约束规则为例，使用模型正确性检查语言描述这些约束规则，并通过模型正确性检查插件检查所构建供配电系统架构模型是否满足这些约束规则。选取的范式级别以及建模领域级别的约束规则包括：

- Rule1：表 6.1 中面向 SysML 需求图的、编号为 1~6 的约束规则；
- Rule2：表 6.1 中面向 SysML 用例图的、编号为 8~10 的约束规则；
- Rule3：表 6.1 中面向 SysML 模块定义图的、编号为 25~33 的约束规则；
- Rule4：表 6.1 中面向 SysML 内部模块图的、编号为 36~37 的约束规则。

此外，选取的目标建模对象级别的约束规则包括：

- Rule5：表 6.2 中编号为 42 和 43 的规则：供配电系统用例图中，所有模块集合应该等价于供配电系统上下文模块定义图中模块集合。对于每一个用例剖视得到的活动图中所有泳道的类型集合，与该用例关联的模块集合是等价的。
- Rule6：表 6.2 中编号为 44 的规则：当活动图中动作（Action）中的栓与内部模块图中内部模块中的端口具有分配关系时，栓传输的事物类型与端口传输

的事物类型是一致的，表示系统功能活动至逻辑组成信息传递的一致性。

(1) Rule1

Rule1 的作用对象是需求图中的需求，故在需求对象元模型 Object_Requirement 中定义了懒惰约束规则 rule1A 和 rule1B。该元模型具有若干属性：name1 代表“名称”、text1 代表“文本描述”、source1 代表“来源”、verify_Method1 代表“验证方法”、validate_Method1 代表“确认方法”，这些属性可作为参数被 rule1A 和 rule1B 使用。

```

1. Object Object_Requirement
2. initial Property name1="null" extends PropertyName annotation(localLabel
   ="名称");
3. initial Property text1="null" extends PropertyText annotation(localLabel
   ="文本描述");
4. initial Property source1="null" extends PropertySource annotation
   (localLabel ="来源");
5. initial Property verify_Method1 = "null" extends Property_Verify_Method
   annotation(localLabel ="验证方法");
6. initial Property validate_Method1 = "null" extends Property_validate
   _Method annotation(localLabel ="确认方法");
7. ....
8. Constraint(
9.     lazy check rule1A(){
10.         pre(
11.             Relationship[satisfy] cross;
12.             Graph[block_Definition_Diagram]!Object[block] obj
13.         )//没有 guard 部分，表示约束上下文为所有需求元模式的实例
14.         inv(
15.             name1.isMandatory(true); //必须有名称属性
16.             multiplicity(name1)==1; //仅有一个名称属性
17.             text1.isMandatory(true); //必须有文本描述属性
18.             source1.isMandatory(true); //必须有来源属性
19.             verify_Method1.isMandatory(true); //必须有验证方法属性
20.             validate_Method1.isMandatory(true); //必须有确认方法属性
21.             this.isConnectTo(obj, cross,"M"); //必须被模块满足
22.         )
23.     )
24.     lazy check rule1B(){
25.         guard(this.have(name1);) //表示约束上下文为具有名称属性的需求实例
26.         inv(name1.unique(true);) //名称属性的值是唯一的。

```



```

27.      )
28. );.....
29. end Object_Requirement

```

rule1A 的头部没有定义参数，即默认其约束上下文为需求对象元模型的所有实例，可使用图 6.22 中的模式图 P_A 表示。由于 rule1A 在需求对象元模型中定义，该模式图没有类型，故面向所有具有需求实例的图实例。在 rule1A 的 pre 中定义了两个参数：类型为满足关系元模型的 cross 以及类型为模块定义图元模型中模块对象元模型的 obj。在 inv 中使用属性强制性约束 isMandatory 表示每个需求实例的“名称”、“文本描述”、“来源”、“验证方法”以及“确认方法”属性是必须存在的；使用属性多重性约束函数 multiplicity 表示每个需求实例仅存在一个名称属性；使用结构强制性约束函数 isConnectedTo 描述每个需求实例都必须通过满足关系与模块实例相关联，从而表示每个需求都被模块满足了。rule1A 可以基于一阶逻辑表达为式 6.1。

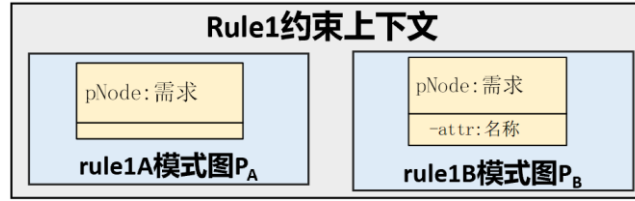


图 6.22 rule1A 和 rule1B 的约束上下文模式图

在 rule1B 的 guard 部分定义了二元关系：this.have(name1)，限制约束上下文为所有具有名称属性的需求实例，可使用图 6.22 中的模式图 P_B 表示。在 rule1B 的 inv 部分使用属性值唯一性约束函数 unique 表示每个需求的“名称”属性是唯一的。rule1B 可基于一阶逻辑表达为式 6.2。

$$\begin{aligned}
& \forall x (Object^{Object_Requirement}(x) \rightarrow \exists y_1, y_2, y_3, y_4, y_5, y_6 \exists z (Property^{name1}(y_1) \\
& \quad \wedge Property^{text1}(y_2) \wedge Property^{source1}(y_3) \wedge Property^{verify_Method1}(y_4) \\
& \quad \wedge Property^{validate_Method1}(y_5) \wedge \bigcup_1^5 Have(x, y_5) \\
& \quad \wedge Object^{block}(y_6) \wedge Relationship^{satisfy}(z) \wedge ConnectWith(z, x_1, y_6))) \quad (6.1)
\end{aligned}$$

$$\begin{aligned}
& \forall x_2 \forall y_5 (Object^{Object_Requirement}(x_2) \wedge Property^{name1}(y_5) \wedge \\
& \quad Have(x_2, y_5) \rightarrow unique(y_5)) \quad (6.2)
\end{aligned}$$

在 MetaGraph 中，通过 5.4 节构建的模型正确性检查插件，选择需求对象元模型 Object_Requirement 所在.karma 文件，执行检查。如图 6.23 所示，检查插件将查询到模型中所有符合模式图 P_A 和 P_B 的需求实例，并进行约束可满足性检查。然后输出错

误信息至 MetaGraph 的控制台区域。通过错误信息可以发现，在供电系统任务需求图中，所有需求都缺乏验证方法和确认方法属性，所有任务需求都没有被模块满足；在供电系统完整需求图中，所有需求都缺乏来源、验证方法和确认方法属性，所有需求都没有被模块满足。因此，后续需要对此两个需求图进行完善，增添其缺乏的属性信息，提升系统需求信息的完整性。并建立系统物理架构组成模块定义图中模块至需求的满足关系矩阵，用于确认供电系统的所有需求是否均被设计方案满足了。

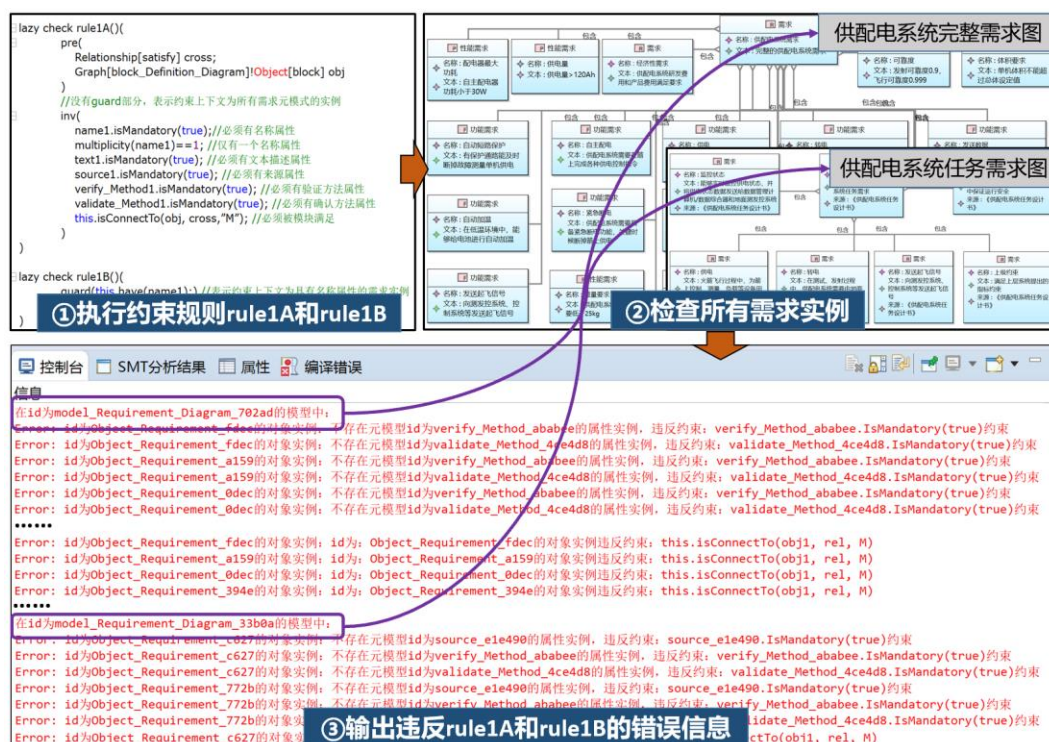


图 6.23 rule1A 和 rule1B 执行结果（错误信息仅展示部分）

(2) Rule2

Rule2 的作用对象为用例图中的用例和模块，故在用例图元模型 Use_Case_Diagram 中定义了 3 条懒惰约束规则 rule2A~rule2C。

```
1. partial Graph Use_Case_Diagram
2. ....
3. Constraint(
4.     lazy check rule2A(Object[block] obj1)(
5.         pre(//约束上下文为用例图中的所有模块实例
6.             Object[use_case] obj2;
7.             Relationship[association] rela;
```

```

8.      ) //每个模块都需要与用例通过关联关系建立联系
9.      inv(obj1.isConnectTo(obj2,rela,"M"));
10.   )
11.   lazy check rule2B(Object[abstractUse_Case] obj3)(
12.       pre(//约束上下文为用例图中的所有抽象用例实例
13.           Object[use_Case] obj4;
14.           Relationship[rela_extends] rela1;
15.       ) //每个抽象用例都需要与用例通过继承关系建立联系
16.       inv(obj3.isConnectTo(obj4, rela1,"M"));
17.   )
18.   lazy check rule2C(Object[use_Case] obj5)(
19.       //约束上下文为用例图中的所有用例实例
20.       pre(Graph[Activity_Diagram] g1;)
21.       inv(obj5.explode(g1,"M")); //每个用例都需要剖视至一个活动图
22.   )
23. ); .....
24. end Use_Case_Diagram

```

规则 rule2A 的头部指定了类型为模块对象元模型的参数 obj1, 表示该规则的约束上下文为用例图实例中的所有模块实例, 可使用图 6.24 中的 P_A 表示。在 pre 中定义了类型为用例对象元模型的参数 obj2 和类型为关联关系元模型的参数 rela, 并在 inv 中通过结构强制性约束 isConnectTo 描述每个模块实例必须通过关联关系实例与用例实例相连, 表示系统用例捕获到了与所有利益相关者具有交互的行为。rule2A 可基于一阶逻辑表达为式 6.3。

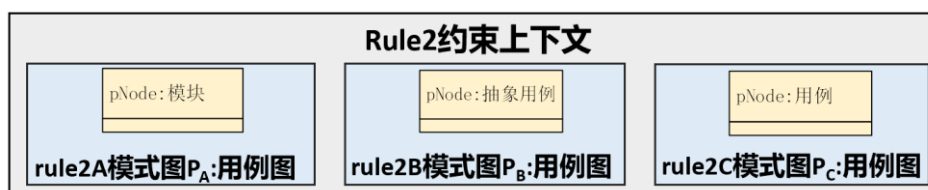


图 6.24 rule2A~rule2C 的约束上下文模式图

在规则 rule2B 的头部定义了类型为抽象用例对象元模型的参数 obj3, 表示该规则的约束上下文为用例图实例中的所有抽象用例实例, 可使用图 6.24 中的 P_B 表示。此外, 在 pre 中定义了类型为用例对象元模型的参数 obj4 和类型为继承关系元模型的参数 rela1。并在 inv 中通过结构强制性约束 isConnectTo 描述每个抽象用例实例必须通过继承关系实例与用例实例相连, 从而表示每个抽象用例都被其他用例实现了。rule2B

可基于一阶逻辑表达为式 6.4。

在 rule2C 的头部定义了类型为用例对象元模型的参数 obj5, 表示该规则的约束上下文为所有用例图实例中的用例实例, 可使用图 6.24 中的 P_C 表示。在 pre 部分定义了类型为活动图元模型的参数 g2, 并在 inv 部分通过剖视强制性约束: obj5.explode(g1, “M”)描述每个用例实例必须剖视至活动图实例, 从而描述该用例相关的场景。rule2C 可基于一阶逻辑表达为式 6.5。

$$\forall x_1 (Object^{block}(x_1) \rightarrow \exists y_1, z_1 (Object^{use_Case}(y_1) \wedge Relationship^{association}(z_1) \wedge ConnectWith(z_1, x_1, y_1))) \quad (6.3)$$

$$\forall x_2 (Object^{abstract_use_Case}(x_2) \rightarrow \exists y_2, z_1 (Object^{use_Case}(y_2) \wedge Relationship^{rela_extends}(z_1) \wedge ConnectWith(z_1, x_2, y_2))) \quad (6.4)$$

$$\forall x_3 (Object^{use_Case}(x_3) \rightarrow \exists y_3 (Graph^{Activity_Diagram}(y_3) \wedge Explode(x_3, y_3))) \quad (6.5)$$

基于模型正确性检查插件, 选择用例图元模型 Use_Case_Diagram 所在.karma 文件, 然后执行检查。用例图元模型的实例只有供配电系统用例图, 经正确性检查插件进行编译和执行后, 发现供配电系统用例图中满足规则 rule2A~rule2C。每一个用例实例均剖视到了一个活动图以详细描述该用例相关的场景, 每一个模块实例都关联到了至少一个用例实例, 确保所建模用例涵盖了与所有上下文模块的交互活动。因此如图 6.25 所示, MetaGraph 控制台区域无错误信息显示。

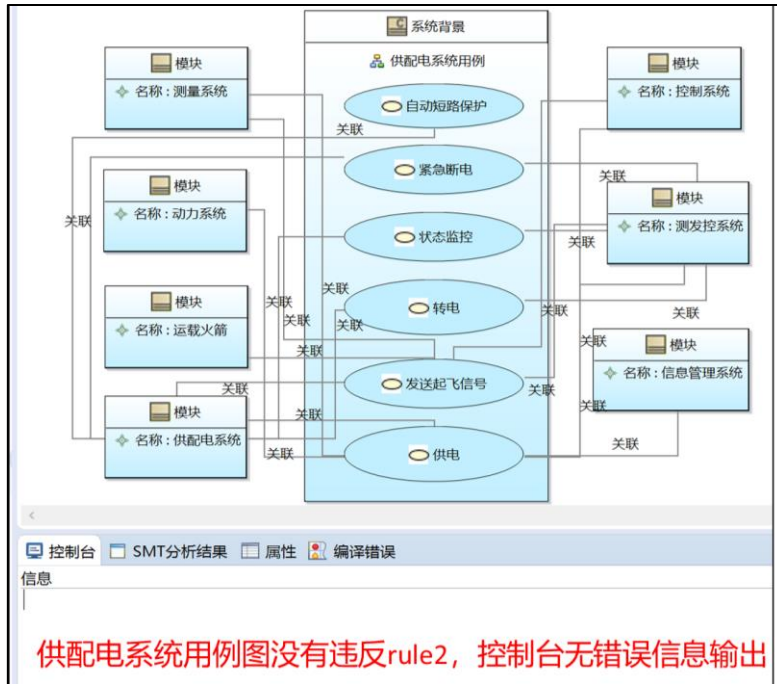


图 6.25 基于 rule2A~rule2C 执行模型正确性检查

(3) Rule3

Rule3 的作用范围是模块定义图中的模块和端口，如下所示，在模块定义图元模型 Block_Definition_Diagram 中定义了 5 条懒惰约束规则 rule3A~rule3E。

```

1. partial Graph Block_Definition_Diagram
2. Object block;
3. Object constraintBlock;
4. ....
5. Constraint(
6.     lazy check rule3A(Object[block]obj1, Object[block] obj2,
7.         Relationship[Composition] rel1)(
8.             guard(rel1.connectWith(obj1,obj2)); //约束上下文为所有由组合关系实例连接
9.             两端模块实例组成的模型结构
10.            inv(
11.                obj2.isCompositionOf(obj1); //模块与模块之间的组合约束
12.                rel1.isCyclical(false); //组成关系不成环约束
13.            )
14.            fix(executeComposite(obj1,obj2));)
15.     lazy check rule3B(Object[block]obj3, Object[block] obj4,
16.         Relationship[Aggregation] rel2)(
17.             guard(rel2.connectWith(obj3,obj4)); //约束上下文为所有由聚合关系实例连接
18.             两端模块实例组成的模型结构
19.            inv(
20.                obj4.isAggregationOf(obj3);
21.                rel2.isCyclical(false);
22.            ) //模块与模块之间的聚合约束
23.            fix(executeAggregate(obj3,obj4));)
24.     lazy check rule3C(Object[block]obj5, Object[block] obj6,
25.         Relationship[Generalization] rel3)(
26.             guard(rel3.connectWith(obj5,"n",obj6)); //约束上下文为所有由泛化关系实例
27.             连接始端 n 个模块实例和终端 1 一个模块实例组成的模型结构
28.            inv(
29.                obj6.isSubClassOf(obj5); //模块与模块之间的继承约束
30.                rel3.isCyclical(false); //继承关系不成环约束
31.                multiplicity(obj5,obj6,rel3,"source")>=0; //支持多继承
32.            )
33.            fix(executeExtends(obj5,obj6)); //自动修复

```



```

30.    )
31.    lazy check rule3D(Object[block] obj7, Point[proxyPort] po1)(
32.        pre(Property[type1] p3;)
33.        guard(
34.            //约束上下文为所有由模块实例和具有类型属性的代理端口实例构成的模型结构
35.            obj7.include(po1);
36.            po1.have(p3);
37.        ) //所有代理端口实例的类型属性值的类型是接口模块元模型
38.        inv(p3 instanceof Object[interfaceBlock];)
39.    )
40.    lazy check rule3E(Object[block] obj8, Point[fullPort] po2)(
41.        pre(Property[type2] p4;)
42.        guard(
43.            //约束上下文为所有由模块实例和具有类型属性的完整端口实例构成的模型结构
44.            obj8.include(po2);
45.            po2.have(p4);
46.        ) //所有完整端口实例的类型属性值的类型是模块元模型
47.        inv(p4 instanceof Object[block];)
48.    )
49. )
50. end Block_Defintion_Diagram

```

rule3A 的头部指定了两个类型为模块对象元模型的参数 obj1 和 obj2、一个类型为组合关系元模型的参数 rel1。在 rule3A 的 guard 部分使用三元关系 connectWith 表示该规则的约束上下文为所有由组合关系实例连接两端模块实例组成的模型结构，可使用图 6.26 中的模式图 P_A 表示。在 inv 中使用组合约束 isCompositionOf 表示 obj1 需具有 obj2 这个组成部分属性，即 obj1 有掌握 obj2 生死的权力；使用不成环约束表示组合关系连接的模型结构不成环。在 fix 中使用内置函数 executeComposite 表示当违反 inv 中定义的约束表达式时，将自动进行修复，使 obj1 具有值为 obj2 的组成部分属性。rule3A 可基于一阶逻辑表示为式 6.6。rule3B 和与 rule3A 类似，区别在于 rule3B 用于约束所有通过聚合关系实例连接的两个模块实例之间的聚合关系。rule3B 的约束上下文可使用图 6.26 中的模式图 P_B 表示，其本身可以一阶逻辑表示为式 6.7。

rule3C 的头部指定了类型为模块对象元模型的参数 obj5、obj6，以及类型为泛化关系元模型的参数 rel3。在 guard 中使用带有基数的三元关系 connectWith 表示该规则的约束上下文为所有由泛化关系实例连接始端 n 个模块实例和终端 1 一个模块实例组

成的模型结构，可使用图 6.26 中的模式图 P_C 表示。在 inv 中使用继承约束函数 $isSubClassOf$ 表示 $obj6$ 需继承 $obj5$ 的所有属性和端口；使用不成环约束函数 $isCyclical$ 表示由泛化关系实例连接模块实例组成的模型结构不能成环。在 fix 中使用内置函数 $executeExtends$ 表示当元素违反 inv 中定义的约束表达式时，将自动进行修复，使 $obj6$ 自动继承 $obj5$ 的所有属性和端口。 $rule3C$ 可基于一阶逻辑表示为式 6.8。

$rule3D$ 的头部定义了类型为模块对象元模型的参数 $obj7$ 和类型为代理端口点元模型的参数 $po1$ ，并在 pre 中定义了类型为类型属性元模型的参数 $p1$ 。在 $rule3D$ 的 $guard$ 部分使用二元关系 $include$ 和 $have$ ，限制该规则的约束上下文为所有由模块实例和具有类型属性的代理端口实例构成的模型结构，可使用图 6.26 的模式图 P_D 表示。在 inv 中通过 $instanceof$ 运算符表示所有代理端口实例的类型属性值的类型需为接口模块元模型。 $rule3D$ 可基于一阶逻辑表示为式 6.9。 $rule3E$ 与 $rule3D$ 类似，区别在于 $rule3E$ 用于约束所有完整端口实例的类型属性值的类型是模块元模型。 $rule3E$ 的约束上下文可使用图 6.26 中的模式图 P_E 表示， $rule3E$ 可基于一阶逻辑表达为式 6.10。

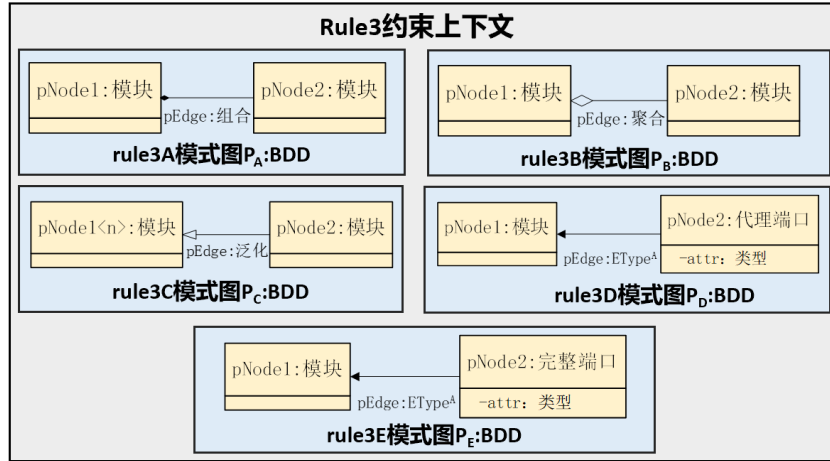


图 6.26 rule3A~rule3E 约束上下文模式图

$$\begin{aligned} & \forall x_1 \forall y_1 (Object^{block}(x_1) \wedge Object^{block}(y_1) \wedge Relationship^{Composition}(z_1) \\ & \wedge ConnectWith(z_1, x_1, y_1) \rightarrow Composite(x_1, y_1) \wedge \neg Composite(y_1, x_1)) \end{aligned} \quad (6.6)$$

$$\begin{aligned} & \forall x_2 \forall y_2 (Object^{block}(x_2) \wedge Object^{block}(y_2) \wedge Relationship^{Aggregation}(z_2) \\ & \wedge ConnectWith(z_2, x_2, y_2) \rightarrow Aggregate(x_2, y_2) \wedge \neg Aggregate(y_2, x_2)) \end{aligned} \quad (6.7)$$

$$\begin{aligned} & \forall x_1, \dots, x_n \forall y_5 (\bigcup_1^n Object^{block}(x_i) \wedge Object^{block}(y_5) \wedge \bigcup_1^n Relationship^{Generalization}(z_i) \\ & \wedge \bigcup_1^n ConnectWith(z_i, x_i, y_5) \rightarrow \bigcup_1^n (Inherit(x_i, y_5) \wedge \neg Inherit(y_5, x_i)) \wedge Ge(n, 1)) \end{aligned} \quad (6.8)$$

$$\begin{aligned} & \forall x_3 \forall p_1 (Object^{block}(x_3) \wedge Point^{proxyPort}(p_1) \wedge include(x_3, p_1) \\ & \wedge Property^{type1}(y_3) \wedge Have(p_1, y_3) \rightarrow Object^{interfaceBlock}(Vale(y_3))) \end{aligned} \quad (6.9)$$

$$\forall x_4 \forall p_2 (Object^{block}(x_4) \wedge Point^{fullPort}(p_2) \wedge include(x_4, p_2) \wedge Property^{type2}(y_4) \wedge Have(p_2, y_4) \rightarrow Object^{block}(Vaule(y_4))) \quad (6.10)$$

基于模型正确性检查插件，选择模块定义图元模型 Block_Defintion_Diagram 所在的.karma 文件，执行检查。首先将获取的 6.3.3 节所定义的所有模块定义图实例（例如供配电系统上下文模块定义图、供配电系统物理组成模块定义图等），然后基于图匹配引擎根据 rule3A~rule3E 的模式图，依次获取每个图实例中的查询结果。针对查询结果，进行约束可满足性检查，输出错误信息如图 6.27 所示。

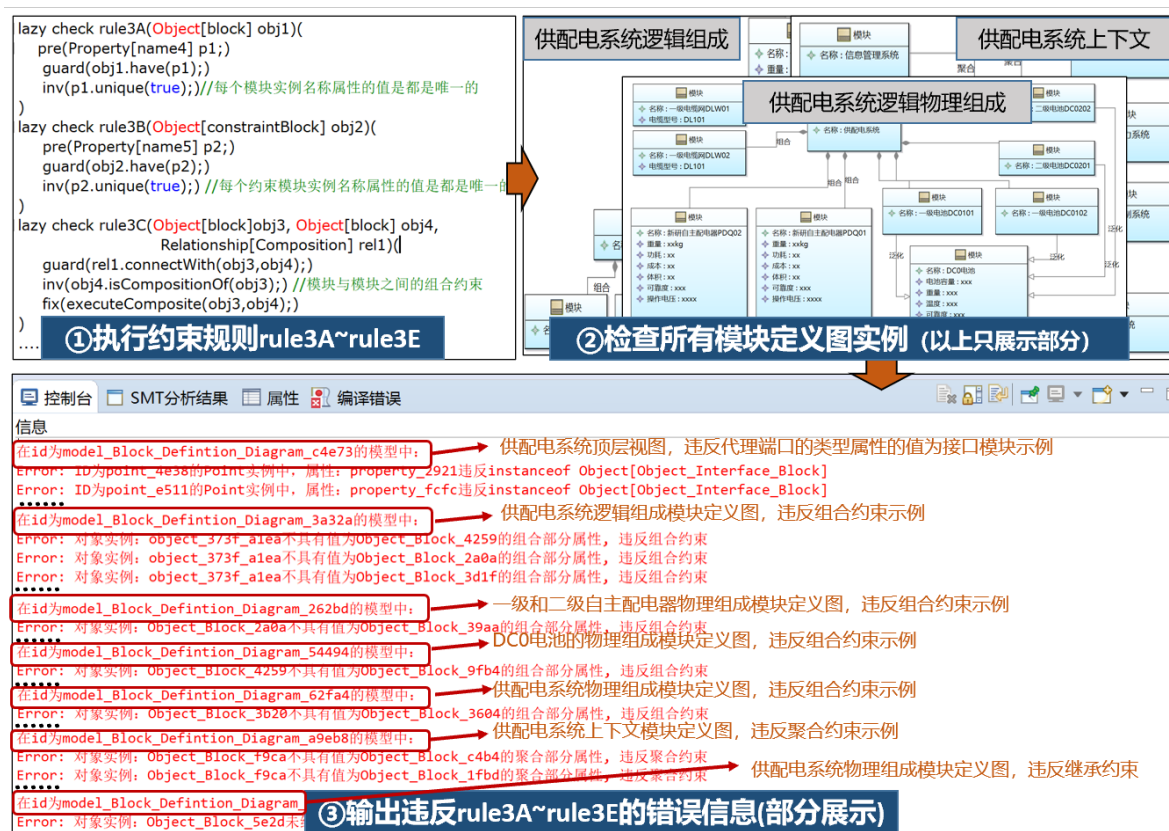
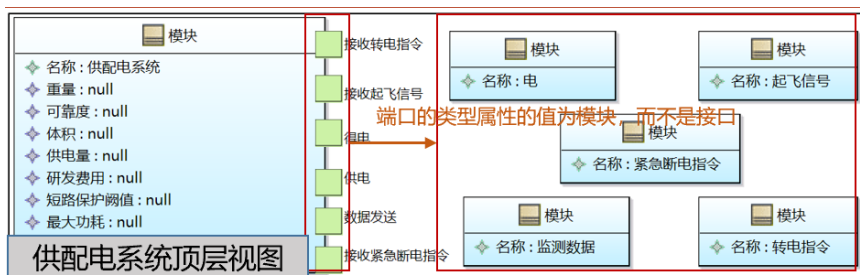


图 6.27 执行 rule3A~rule3E 后输出的违反约束的错误信息

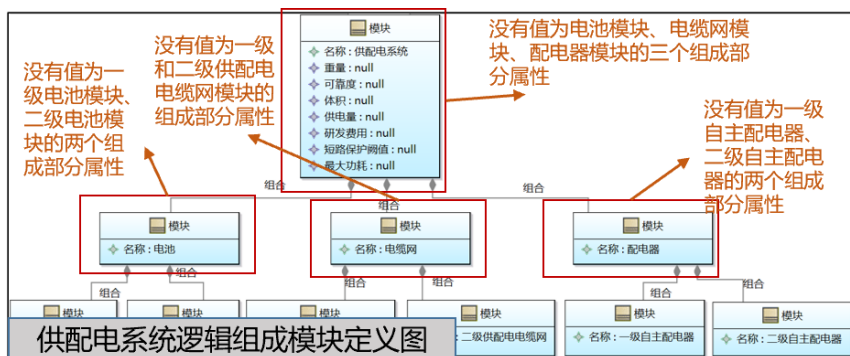
从错误信息中可以发现，供配电系统顶层视图违反规则 rule3D。如图 6.28 中的 A 图所示，名称为“供配电系统”的模块拥有的所有端口的类型为代理端口元模型，但是每个端口的类型属性的值均为模块，而非接口模块。供配电系统逻辑组成、DC0 电池的物理组成、一级和二级自主配电器物理组成以及供配电系统物理组成这四个模型定义图违反规则 rule3A。以供配电系统逻辑组成模块定义图为例，如图 6.28 的 B 图所示，名称为“供配电系统”的模块 x 与名称为“电池”、“电缆网”和“配电器”的模

块（分别为 a、b、c）通过组合关系相连，但是 x 不具有值为 a、b、c 的组成部分属性，违反组合约束。

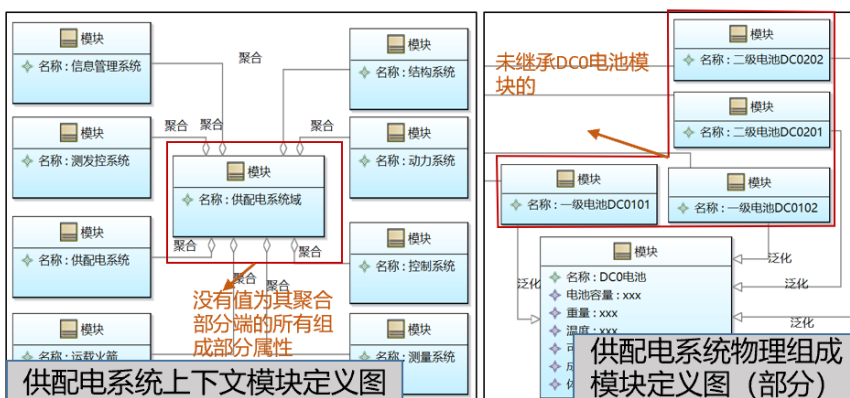
此外，供配电系统上下文模块定义图违反规则 rule3B。如图 6.28 的 C 图所示，名称为“供配电系统域”的模块实例 x 与其他所有模块实例之间具有聚合关系，然而 x 的属性实例中不具有值为其他模块实例的聚合部分属性。其次，供配电系统物理组成模型定义图违反规则 rule3C。如图 6.28 的 D 图所示，红框中的四个模块与名称为“DC0 电池”的模块 y 之间具有泛化关系，但此四个模块并未继承 y 的所有属性。违反规则 rule3A~3C 的本质问题在于即图形化展示的信息与实际模块拥有的属性信息不符。



A: 违反rule3D示例



B: 违反rule3A示例



C: 违反rule3B示例

D: 违反rule3C示例

图 6.28 不满足规则 rule3A~3D 的模型元素示例

此外，对于 rule3A-3C，若存在违反约束表达式的模型元素，将根据其中定义的 fix 函数进行自动修复。如图 6.29 所示，违反 rule3B 的“供配电系统域”模块实例经 executeAggregate 函数自动修复后，增添了 8 个聚合部分属性，值为与它通过聚合关系相连的所有模块。违反 rule3A 规则的“供配电系统”模块实例经 executeComposite 函数自动修复后，增添了 3 个组合部分属性，值为与它通过组合关系相连的所有模块。违反 rule3C 规则的“一级电池 DC0101”模块实例经 executeExtends 函数自动修复后，增添了六个继承自图 6.28 的 D 图中名称为“DC0 电池”的属性，这些属性使用“extended”进行标识。

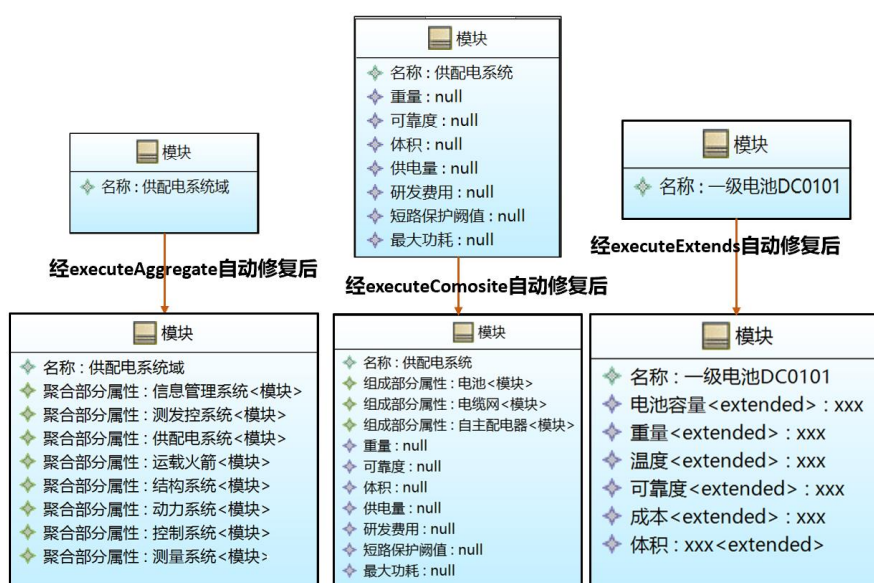


图 6.29 fix 自动修复功能示例

(4) Rule4

该规则的作用对象是内部模块图（简称为 IBD）中的端口和连接器，旨在保证端口（包括代理端口与完整端口）与端口之间、端口与连接器之间是兼容的，可以正确地进行事务的传输。以下在内部模块图元模型 Internal_Block_Diagram 中定义了懒惰规则 rule4A 和 rule4B。

```
1. partial Graph Internal_Block_Diagram
2. Relationship connector
3. ....
4. Constraint(
5.     lazy check rule4A(Point[proxyPort] po1, Point[proxyPort] po2)(
```

```

6.      //约束上下文为所有由连接器关系实例连接两端代理端口实例组成的模型结构
7.      guard(connector.ConnectWith(po1,po2));
8.      inv(//通过连接器连接的两个代理端口的类型是相同的、与连接器类型也相同
9.          po1.Property[type6] == po2.Property[type7];
10.         connector.Property[type8] == po1.Property[type6];
11.     )
12. )
13. lazy check rule4B(Point[fullPort] po3, Point[fullPort] po4)(
14.     //约束上下文为所有由连接器关系实例连接两端完整端口实例组成的模型结构
15.     guard(connector.ConnectWith(po3,po4));
16.     inv(//通过连接器连接的两个完整端口的类型是相同的、与连接器类型也相同
17.         po3.Property[type9] == po4.Property[type10];
18.         connector.Property[type8] == po3.Property[type9];
19.     )
20. )
21. );
22. end Internal_Block_Diagram

```

rule4A 的头部指定了两个类型为代理端口点元模型的参数 po1 和 po2，在 guard 部分使用三元关系 connectWith 表示该规则的约束上下文为所有由连接器关系实例连接两端代理端口实例组成的模型结构，可使用图 6.30 中的模式图 P_A 表示。在 inv 中使用两个自定义布尔表达式表示“通过连接器连接的两个代理端口的类型是相同的、它们的与连接器类型也相同”。rule4A 可基于一阶逻辑表示为式 6.11。rule4B 与 rule4A 类似，区别在于 rule4B 用于约束两个连接的完整端口之间以及端口与连接器之间的兼容关系。rule4B 的约束上下文可使用图 6.30 中的模式图 P_B 表示，其本身可以一阶逻辑表示为式 6.12。

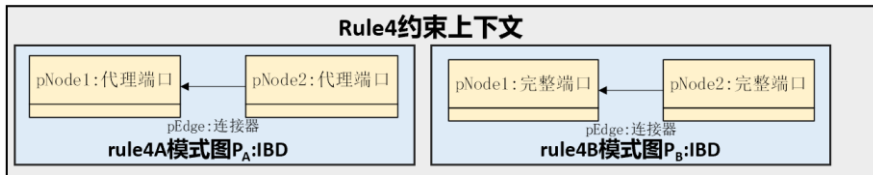


图 6.30 Rule4 约束上下文模式图

$$\begin{aligned}
 & \forall x_1, x_2, x_3 (\bigcup_1^2 \text{Point}^{\text{proxyPort}}(x_i) \wedge \text{Relationship}^{\text{connector}}(x_3) \wedge \text{ConnectWith}(x_3, x_1, x_2) \\
 & \rightarrow \exists p_1, p_2, p_3 (\bigcup_1^2 \text{Property}^{\text{type6}}(p_i) \wedge \text{Property}^{\text{type8}}(p_3) \wedge \bigcup_1^3 \text{Have}(x_i, p_i) \\
 & \wedge \text{Equals}(\text{Value}(p_1), \text{Value}(p_2)) \wedge \text{Equals}(\text{Value}(p_1), \text{Value}(p_3)))
 \end{aligned} \quad (6.11)$$

$$\begin{aligned}
& \forall x_4, x_5, x_6 (\bigcup_4^5 Point^{fullPort}(x_i) \wedge Relationship^{connector}(x_6) \wedge ConnectWith(x_6, x_4, x_5) \\
& \rightarrow \exists p_4, p_5, p_6 (\bigcup_4^5 Property^{type7}(p_i) \wedge Property^{type8}(p_6) \wedge \bigcup_4^6 Have(x_i, p_i) \\
& \wedge Equals(Value(p_4), Value(p_5)) \wedge Equals(Value(p_4), Value(p_6))) \quad (6.12)
\end{aligned}$$

然后采用模型正确性检查插件执行 rule4A 和 rule4B。首先依据图 6.30 中所定义的模式图 P_A 和 P_B 执行图模式匹配，获取到所有内部模块图实例中符合 P_A 和 P_B 的模型结构。其次基于所定义约束表达式进行约束可满足性检查，发现自主配电器物理连接内部模块图不满足 rule4B，输出错误信息至 MetaGraph 控制台如图 6.31 所示。从错误信息中，可以发现名称为“发送加温信号”和“接受加温信号”的两个端口的类型不相同，即两者无法支持传输相同的事务。倘若后续设计团队基于该模型进行单机的设计，则设计出来的单机之间无法进行正确的交互。而基于检查输出的错误信息，建模工程师可对模型进行纠错，避免端口不兼容的问题。

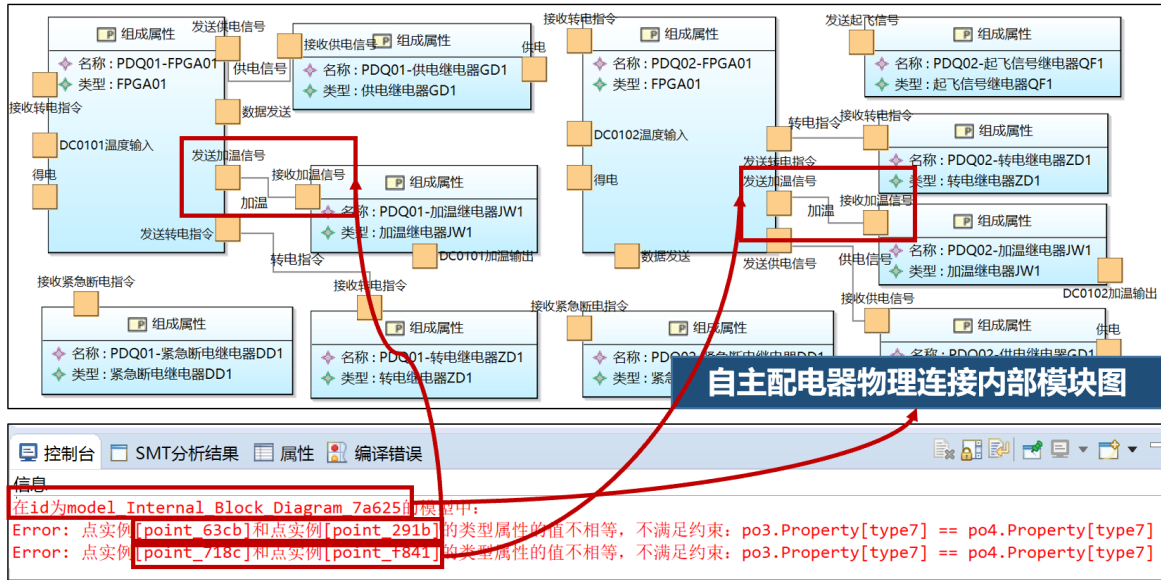


图 6.31 rule4A 和 rule4B 执行结果

(5) Rule5

该规则的作用对象涵盖用例图、用例剖视得到的活动图和供配电系统上下文模块定义图。以下在用例图元模型 Use_Case_Diagram 中定义了懒惰规则 rule5A 和 rule5B。

```

1. partial Graph Use_Case_Diagram
2. ....
3. Constraint(
4.     lazy check rule5A(Object[use_Case] obj1, Object[block] obj2,
       Relationship[association] rel, Graph[Activity_Diagram] g1)(

```

```

5.      guard(
6.          rel.connectWith(obj2,"n",obj1);
7.          obj1.explodeTo(g1);
8.      )
9.      inv(//对于每一个用例剖视得到的活动图中所有泳道的类型集合，与该用例关
联的模块集合是等价的
10.         obj2.isEquivalentTo(obj3.Property[type3]);
11.     )
12. )
13. lazy check rule5B()(
14.     pre(Graph[Block_Definition_Diagram] g1;)
15.     guard(
16.         this.localName == "供配电系统用例图";
17.         g1.localName == "供配电系统上下文模块定义图";
18.     ) //所有模块集合应该等价于供配电系统上下文模块定义图中模块集合
19.     inv(this.Object[block].isEquivalentTo(g1.Object[block]));)
20. )
21. );
22. end Use_Case_Diagram

```

rule5A 是一种普通跨图一致性约束，其头部定义了类型为用例对象元模型的参数 obj1、类型为模块对象元模型的参数 obj2、类型为关联关系元模型的参数 rel 以及类型为活动图元模型的参数 g1。在 guard 中通过三元关系 connectWith 和二元关系 explodeTo 限定该规则的约束上下文为用例图实例中所有符合图 6.32 中 P_A 所示的模型结构。其中，使用基数“n”表示在三元关系中，obj2 的个数可以有 1 至多个。在 rule5A 的 inv 部分通过等价约束表示每个用例关联的模块集合，与该用例剖视得到的活动图中所有泳道的类型集合是等价的。rule5A 可基于一阶逻辑表示为式 6.13。

rule5B 是一种普通跨图一致性约束，用于约束供配电系统用例图中，所有模块集合应该等价于供配电系统上下文模块定义图中模块集合。如图 6.32 所示，其约束上下文为供配电系统用例图。rule5B 可基于一阶逻辑描述为式 6.14。

$$\begin{aligned}
 & \forall x_1 (Object^{use_Case}(x_1) \wedge \bigcup_1^m (Object^{block}(y_i) \wedge Relationship^{association}(z_i) \\
 & \quad \wedge ConnectWith(z_i, y_i, x_1)) \wedge Graph^{Activity_Diagram}(g) \wedge Explode(x_1, g) \\
 & \quad \rightarrow \exists o_1, \dots, o_n (\bigcup_1^n (Object^{lane}(o_i) \wedge Include(g, o_i) \wedge Property^{type3}(p_i) \\
 & \quad \wedge Have(o_i, p_i)) \wedge EquivalentTo(Set(y_1, \dots, y_m), Set(Value(p_1), \dots, Value(p_n)))) \quad (6.13)
 \end{aligned}$$

$$\begin{aligned} & \bigcup_1^n Object^{block}(x_i) \wedge Graph^{Block_Definition_Diagram}(g) \wedge \bigcup_1^m Object^{block}(y_i) \\ & \wedge \bigcup_1^m Include(g, y_i) \rightarrow EquivalentTo(Set(x_1, \dots, x_n), Set(y_1, \dots, y_m)) \end{aligned} \quad (6.14)$$

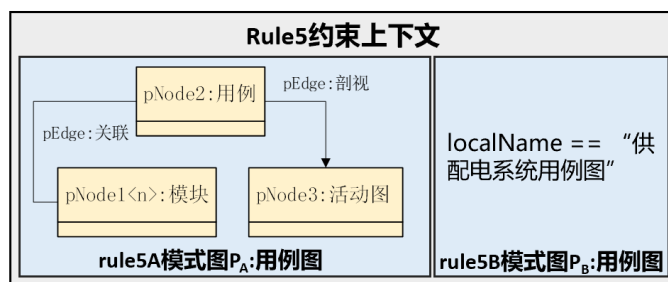


图 6.32 rule5 约束上下文模式图

然后采用模型正确性检查插件执行 rule5A 和 rule5B。首先根据模式图 P_A 和 P_B 进行模型查询，基于图模式匹配引擎获取符合 P_A 和 P_B 的所有匹配。然后基于一阶逻辑进行可满足性检查，输出错误信息至 MetaGraph 的控制台如图 6.33 所示。

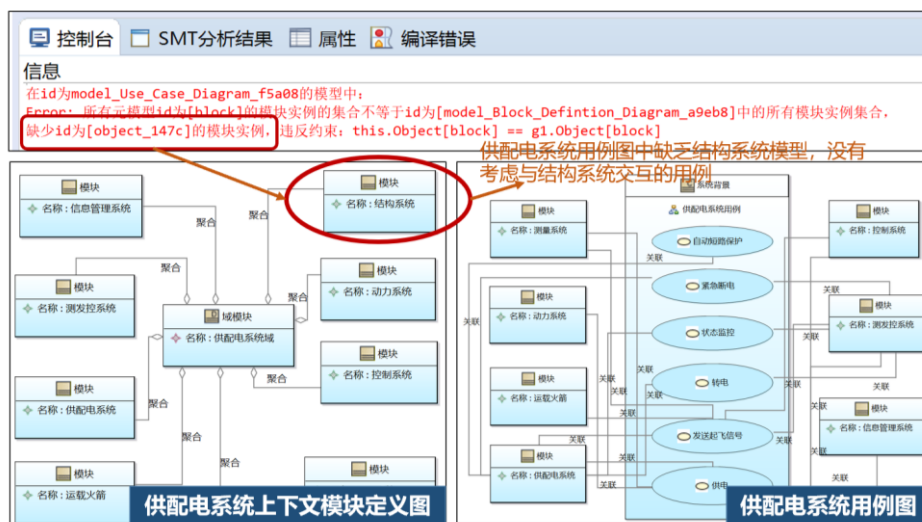
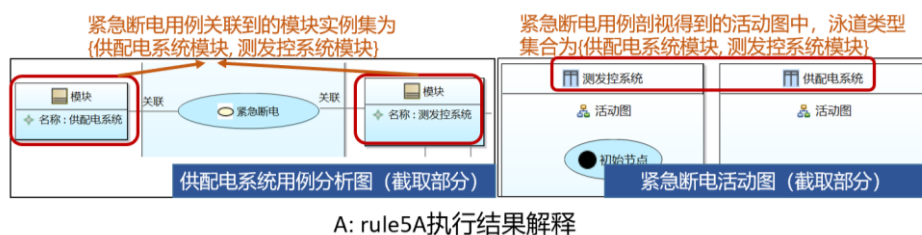


图 6.33 执行 rule5A 和 rule5B 后的结果示例

从错误信息中可以发现，没有模型违反 rule5A，即每个用例实例关联的模块实例集均与其剖视得到的活动图中所有泳道的类型集合相同。以图 6.33 中供配电系统用

例分析图中的紧急断电用例为例，其关联到的模块实例集为名称为“供配电系统”和“测发控系统”的模块，其剖视得到的活动图中，泳道的类型集合也为上述两个模块，即是等价的。因此可以认为描述每个用例场景的活动图都完整的涵盖了与该用例关联的上下文模块之间的交互活动。

此外，供配电系统用例图违反了规则 rule5B，缺乏名称为结构系统的模块实例。因此，可以认为用例图中没有涵盖与所有上下文系统具有交互的用例，即用例分析的不完整。后续建模工程师需要考虑供配电系统与结构系统之间的功能活动，并构建相应的用例进行描述。

(6) Rule6

Rule6 描述的是活动图中动作上的栓与内部模块图中内部模块上的端口之间约束，是一种系统功能视角和结构视角之间的水平一致性约束。以下在内部模块图元模型 Internal_Block_Diagram 中定义了懒惰规则 rule6，其头部定义了类型为活动图元模型中动作对象元模型的参数 obj1，类型为栓点元模型的参数 po1、类型为代理端口点元模型的参数 po2、类型为内部模块对象元模型的参数 obj2 和类型为分配关系元模型的参数 cross。在 guard 中通过跨图关联三元关系 mappingTo 和二元关系 containedIn 描述 obj1、po1、obj2、po2 和 cross 构成的模型结构，即约束上下文，可使用图 6.34 中的模式图 P6 进行表示。在 rule6 的 inv 中使用自定义布尔表达式表示栓传输的事物类型与代理端口传输的事物类型是一致的。rule6 可以基于一阶逻辑表示为式 6.15

```

23. partial Graph Internal_Block_Diagram
24. ....
25. Constraint(
26.     lazy check rule6(Graph[Activity_Diagram]!Object[Action]
        obj1,Point[Pin] po1,Point[proxyPort]
        po2,Object[Part]obj1,Relationship[allocate] cross)(
27.         guard(
28.             po1.containedIn(obj1);
29.             po2.containedIn(obj2);
30.             po1.mappingTo(po2, cross);
31.         )
32.         inv(//栓传输的事物类型与代理端口传输的事物类型是一致的
33.             po1.Property[type5] == po2.Property[type6];
34.         )

```

```

35.      )
36. );
37. ....
38. end Internal_Block_Diagram

```

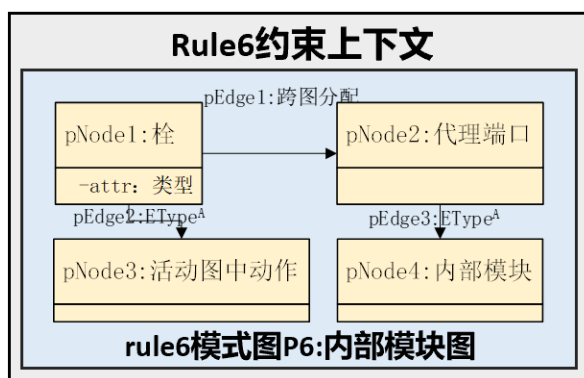


图 6.34 rule6 约束上下文模式图

$$\begin{aligned}
& \forall x \forall y (Point^{Pin}(x) \wedge Point^{ProxyPort}(y) \wedge Reference^{allocate}(x, y) \wedge Object^{Action}(z_1) \\
& \quad \wedge Object^{Part}(z_2) \rightarrow Property^{type5}(p_1) \wedge Include(z_1, x) \wedge Include(z_2, y) \\
& \quad \wedge Property^{type6}(p_2) \wedge Have(x, p_1) \wedge Have(y, p_2) \wedge Equals(Value(p_1), Value(p_2))) \quad (6.15)
\end{aligned}$$

然后通过模型正确性检查插件执行 rule6，实施检查。首先根据模式图 P6 进行模型查询，定位到所有的内部模块图实例（如 6.3.3 节中的外部接口交互内部模块图、供配电系统逻辑连接内部模块图等）。然后基于图模式匹配引擎获取每个图实例中符合 P6 的所有匹配。然后基于一阶逻辑进行可满足性检查，输出错误信息如图 6.35 所示。发现在供配电系统逻辑连接内部模块图中，名称为一级加温输出 1，一级加温输入 1，一级加温输出 2，一级加温输入 2、二级加温输出 1、二级加温输入 1，二级加温输出 2，二级加温输入 2 的代理端口点实例违反约束。以一级加温输出 1 点实例 x 为例，其类型属性的值为“加温指令<接口模块>”。与 x 具有分配关系的点实例在电池温度监测活动图中，即名称为“一级加温电流 output1”的栓点实例 y 。 y 的类型属性的值为“加温电流<接口模块>”，与 x 的类型属性的值不一致。由于逻辑元素的连接关系来源于逻辑元素之间的功能活动交互，因此，建模工程师后续需依据电池温度监测活动图中栓实例的类型，修改供配电系统逻辑连接内部模块图中违反约束的代理端口实例，以确保描述系统功能活动的视图与描述系统结构的视图之间的一致性。

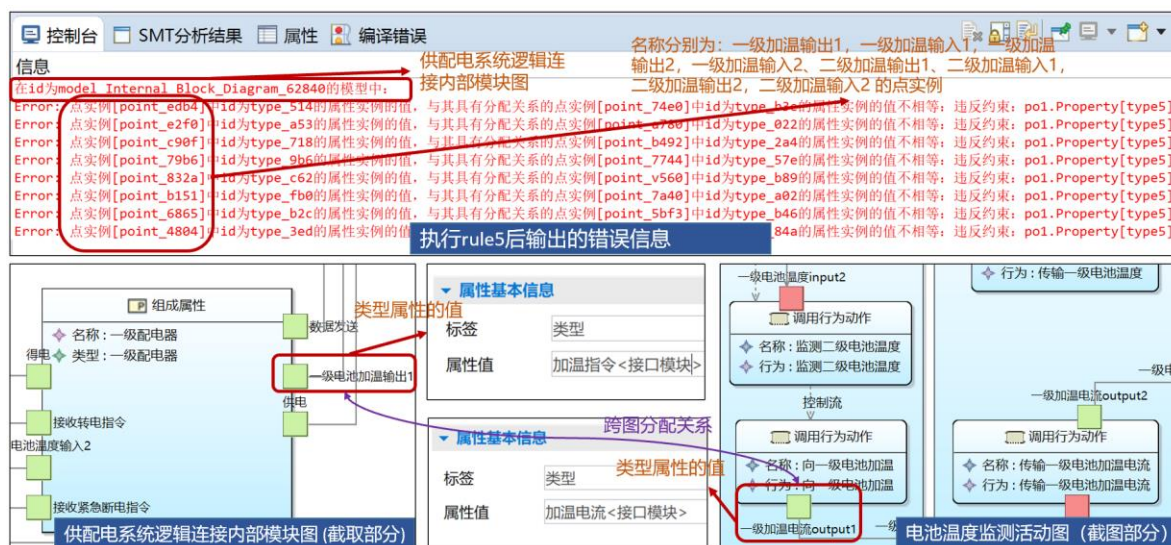


图 6.35 执行 rule6 之后输出的错误信息

6.5 案例总结与分析

本案例以运载火箭供配电系统为对象, 使用集成了模型正确性检查插件的 MetaGraph 多架构建模平台和 KARMA 语言, 完成了供配电系统架构模型的构建, 模型正确性检查约束规则的构建以及基于约束规则的正确性检查执行。从定量分析的角度来看, 在 6.3.3 节使用 KARMA 语言构建了 56 个图模型 (篇幅原因, 文中未展示全部模型), 涵盖系统的需求分析-功能分析-逻辑设计-物理设计四个过程以及分系统-单机两个层次。此外, 根据第三章所提面向模型正确性检查的约束规则体系, 在 6.3.2 节梳理了 39 条范式级别和建模领域级别的约束规则, 在 6.3.4 节梳理了 9 条目标建模对象级别的约束规则。然后, 在 6.4 节以 15 条约束规则为例阐述所提出模型正确性检查方法、语言和插件的可用性。此 15 条约束规则的类型、所在的级别以及规则执行后的效果总结为如表 6.3 所示。

从定性分析的角度来看, 本文所提出的模型正确性检查方法和语言的可用性得到了验证。首先在 6.4 节, 采用模型正确性检查语言描述了 15 条约束规则, 并采用所提出的基于图模式匹配和一阶逻辑的正确性检查方法对供配电系统架构模型进行检查, 在输出的报错信息中发现了若干个模型不满足所定义约束规则, 即模型没有被正确地构建, 建模工程师可根据报错信息定义到有问题的、不符合规范的模型元素, 并进行模型的修改和完善, 从而提供系统架构模型的质量。

表 6.3 应用的约束规则类型、描述的正确性内涵及执行后的效果

名称	级别	涉及的约束类型	执行效果
rule1A	F、D	属性强制性约束、属性多重性约束、结构强制性约束	发现存在模型违反规则 rule1A，即在供配电系统任务需求图和完整需求图中，所有的需求都缺乏验证方法和确认方法，所有需求都没有被模块满足。在完整需求图中，所有需求都缺乏来源。
rule1B	F	属性值唯一性约束	
rule2A	F	结构强制性约束	发现没有模型违反规则 rule2A 和 rule2B。即用例图中每一个用例均剖视到了一个活动图以描述该用例相关的场景，每一个模块都关联到了至少一个用例，确保所构建用例涵盖了与所有模块的交互活动。
rule2B	F	结构强制性约束	
rule2C	F	剖视强制性约束	
rule3A	F	组合约束,不成环约束	发现存在模型违反 rule3A~rule3C，例如供配电系统逻辑组成模块定义图违反组合约束，供配电系统上下文模块定义图违反聚合约束，物理组成模型定义图违反继承约束。供配电系统顶层视图违反规则 rule3D，图中每个端口的类型属性的值均为模块，而非接口模块，不符合建模惯例的要求。
rule3B	F	聚合约束,不成环约束	
rule3C	F、D	继承约束,对象多重性约束,不成环约束	
rule3D	F	属性值约束	
rule3E	F	属性值约束	
rule4A	F	混合约束	发现自主配电器物理连接内部模块图不满足 rule4B，该图模型中存在端口不兼容性问题，即通过连接器连接的两个端口无法支持传输相同的事务。
rule4B	F	混合约束	
rule5A	T	集比较约束中的等价性约束	供配电系统用例图违反规则 rule5B，该图模型中的模块集合与供配电系统上下文中的模块集合不等价。可认为该用例图没有涵盖与供配电系统上下文中所有系统具有交互的用例，即用例分析的不完整。
rule5B	T	集比较约束中的等价性约束	
rule6	T	混合约束	供配电系统逻辑连接内部模块图违反 rule6。该模型描述的是逻辑视角的信息，存在部分元素 x 由功能分析阶段的模型元素 y 分配得出。但 x 的属性信息与 y 的属性信息不一致，不符合水平一致性。

最后，从可推广性角度来看，本文所提出方法具备支持多领域架构模型构建和正确性检查的能力。首先 GOPRR-E 元建模方法具备多领域架构建模的能力，支持多种

领域建模语言（如 SysML、UML、AADL）的形式化表达，能有效地支持多种复杂系统（如航天航空、汽车、船舶）的早期架构设计。此外，所提出的模型正确性检查方法和语言面向 GOPRR-E 元建模方法所构建模型，且梳理了元模型构建阶段和模型构建阶段中、面向范式级别-建模领域级别-目标建模对象级别的约束规则。这些约束规则面向的是建模语言的元模型且该元模型可重用于其他复杂系统的设计过程中，因此，所定义的约束规则也能被重用于其他系统架构模型的正确性检查过程，达到一次定义、多次使用的效果，从而提升设计效率。

6.6 本章小结

本章应用所提出的约束规则体系、模型正确性检查方法和语言解决了运载火箭供配电系统设计过程中的架构模型正确性检查问题。首先基于 KARMA 语言构建 SysML 语言元模型，并梳理范式级别和建模领域级别的约束规则。然后基于 RFLP 方法构建了运载火箭供配电系统的需求-功能-逻辑-物理架构模型，并梳理了目标建模对象级别的约束规则。然后以梳理的若干条约束规则为例，使用所提出正确性检查语言对这些约束规则进行描述。基于模型正确性检查插件执行约束规则，检查供配电系统架构模型中不符合约束规则的模型元素，输出错误信息以帮助建模工程师对模型进行修正。通过此案例验证了所提出模型正确性检查方法的有效性。

总结与展望

工作总结

在基于模型的系统架构设计过程中，模型正确性检查方法可通过约束规则描述模型需要满足的系统给定设计规范，自动地发现不满足规范的错误模型。通过模型正确性检查能有效地提高模型质量，实现“尽早发现错误、尽早改正”的效果，从而提升设计效率并降低成本。本文针对目前基于模型的复杂系统设计过程中，缺乏面向模型正确性检查的约束规则体系、现有模型正确性检查方法难以统一实现多领域系统架构模型构建和正确性检查的问题，提出了融合了 GOPPRR-E 元建模方法、图模式匹配和一阶逻辑的多领域系统架构模型正确性检查方法。论文的主要工作内容总结如下：

- (1) 构建了面向模型正确性检查的约束规则体系。该部分首先界定了模型正确性的内涵，然后明确了约束规则类型以支持正确性内涵的描述。最后结合 MOF 元建模框架和 RFLP 的系统架构建模过程，形成了涵盖范式级别-建模领域级别-目标建模对象级别的约束规则体系，为不同建模阶段需要定义的约束规则提供了指导。该方法与系统架构设计中的元建模过程和建模过程紧密结合，具备可实施性。
- (2) 形成了面向多领域系统架构模型的正确性检查方法。该部分在理论层面提出了系统架构模型的正确性检查方法。以 GOPPRR-E 元建模方法作为多领域系统架构模型统一描述的基础，并构建面向 GOPPRR-E 方法所构建模型的正确性检查的流程。然后提出基于图模式匹配方法解决正确性检查流程中的模型查询问题，基于一阶逻辑解决正确性检查流程中的约束可满足性问题。该方法以 GOPPRR 六种元素为底层数据进行模型形式化表达和正确性检查，能够有效地支持若干领域架构模型的检查。
- (3) 设计并实现了模型正确性检查语言。该部分在实施层面实现了所提出的模型正确性检查方法。首先在基于 GOPPRR-E 方法的 KAMRA 语言的基础上扩展，设计并定义了模型正确性检查的抽象语法和具体语法规则，为约束规则的定义提供媒介。然后基于 ANTLR、AGG 图匹配引擎和 Z3 约束求解器构建

模型正确性检查插件，以支持模型查询和约束可满足性检查。本部分研究以 KARMA 语言为基础，实现了面向多领域架构模型的模型正确性检查语言，支持建模工程师基于一种语言和平台实现多领域架构模型的构建和正确性检查，降低了系统设计过程中的资源消耗。

- (4) 基于案例验证上述研究工作的有效性。以运载火箭供配电系统为对象，采用所提约束规则体系、模型正确性检查方法和语言实现供配电系统架构模型的正确性检查。通过检查发现了若干个不符合约束规则的错误模型，验证了所提方法和语言的有效性。

基于工作总结，本文的**创新点**包括：

(1) 提出了**基于图模式匹配和一阶逻辑的模型正确性检查**方法。提出了一种面向多领域系统架构模型的通用模型正确性检查方法。该方法使用 GOPRR-E 元建模方法的实现多种领域建模语言所构建系统架构模型的统一描述。在此基础上，提出了涵盖模型查询和约束可满足性检查的模型正确性检查流程。然后基于图模式匹配方法实现模型查询，基于一阶逻辑理论实现约束可满足性检查。所提出的方法为目前模型正确性检查方法不具有通用性的问题提供了解决思路，能有效应用于复杂系统架构设计过程。

(2) 提出了**基于 KARMA 的模型正确性检查语言**。在 KARMA 语言的基础上，设计了模型正确性检查语言的语法语义，实现基于 KARMA 的多领域架构模型描述以及正确性检查的一体化表达，为目前主流建模语言不支持模型正确性检查的问题提供了解决思路。此外，构建了面向 KARMA 模型的正确性检查插件，实现多领域系统架构模型构建以及正确性检查的平台一体化，能有效地提升建模工程师的集成设计能力。建模工程师无需耗费额外的精力将系统架构模型转换至其他正确性检查工具，提高系统设计的效率。

工作展望

本文的研究围绕模型正确性检查的约束规则体系、检查方法和语言展开，仍存在一些不足和可改进的方面，现做出工作展望如下：

- 模型正确性检查的约束规则的冲突检测支持不足。目前的检查方法以建模工

程师定义的约束规则为准则进行检查，但对约束规则的冲突检测支持不足。约束规则可以在图/对象/点/关系/角色/属性元模型中定义，不同元模型中定义的约束规则之间可能存在矛盾，使得无法得到一个符合所有规则的模型。后续可以考虑建立完善的冲突检测机制，考虑不同元模型中约束规则的优先级。

- 在约束规则的构建以及错误信息的显示方面，用户友好性不足。目前约束规则的构建需要用户熟悉模型正确性检查的所有语法语义并采用文本的方式添加。相比于采用图形化的方式进行约束规则的构建，文本式对用户的要求更高。此外，目前的报错信息在控制台显示，没有在图形化模型中直接定位到有问题的元素，友好性不足。在后续的工作中，可以考虑构建图形化的约束规则添加框，并将违反约束规则的错误信息定位在图形化模型中。
- 模型正确性检查插件对建模过程中的实时检查支持不足。目前开发的插件中，能够支持建模后手动触发检查，而由于编译性能原因，对在建模过程中的实时地模型正确性检查支持不足。后续可以优化代码结构，从而支持实时检查，提高用户友好度。

参考文献

- [1] NASA systems engineering handbook[M]. Diane Publishing, 2010.
- [2] 周潇雅,杨亮,张茜,孙树森,肖进.基于 ARCADIA MBSE 在运载火箭能源子系统的应用[J].计算机测量与控制,2020,28(11):211-215+221.DOI:10.16526/j.cnki.11-4762/tp.2020.11.043.
- [3] Huldt T, Stenius I. State - of - practice survey of model - based systems engineering[J]. Systems engineering, 2019, 22(2): 134-145.
- [4] 何巍,胡久辉,赵婷,彭越,唐俊杰.基于模型的运载火箭总体设计方法初探[J].导弹与航天运载技术,2021(01):12-17+32.
- [5] Alenazi M, Niu N, Savolainen J. SysML modeling mistakes and their impacts on requirements[C]//2019 IEEE 27th International Requirements Engineering Conference Workshops (REW). IEEE, 2019: 14-23.
- [6] Unhelkar B. Verification and validation for quality of UML 2.0 models[M]. John Wiley & Sons, 2005.
- [7] Clements P C, Northrop L M. Software Architecture: An Executive Overview[R]. Carnegie-Mellon Univ Pittsburgh PA Software Engineering Inst, 1996.
- [8] Estefan J A. Survey of model-based systems engineering (MBSE) methodologies[J]. Incose MBSE Focus Group, 2007, 25(8): 1-12.
- [9] Delligatti L. SysML distilled: A brief guide to the systems modeling language[M]. Addison-Wesley, 2013.
- [10] T. Clark, P. Sammut, J.S. Willans, Applied metamodeling: A foundation for language driven development (third edition), 2015, CoRR abs/1505.00149
- [11] Booch G, Rumbaugh J, Jacobson I. UML: unified modeling language[J]. Versão, 1997.
- [12] Hause M. The SysML modelling language[C].Fifteenth European Systems Engineering Conference. 2006, 9: 1-12.
- [13] Chinosi M, Trombetta A. BPMN: An introduction to the standard[J]. Computer Standards & Interfaces, 2012, 34(1): 124-134.
- [14] Hause M. The Unified Profile for DoDAF/MODAF (UPDM) enabling systems of systems on many levels[C]//2010 IEEE international systems conference. IEEE, 2010: 426-431.
- [15] Feiler P H, Gluch D P, Hudak J J. The architecture analysis & design language (AADL): An

- introduction[R]. Carnegie-Mellon Univ Pittsburgh PA Software Engineering Inst, 2006.
- [16] Fritzson P, Engelson V. Modelica-a unified object-oriented language for system modeling and simulation[C]//ECOOP. 1998, 98: 67-90.
- [17] Lu J, Wang G, Ma J, et al. General Modeling Language to Support Model - based Systems Engineering Formalisms (Part 1)[C].INCOSE International Symposium. 2020, 30(1): 323-338.
- [18] Paige R F, Ostroff J S, Brooke P J. Principles for modeling language design[J]. Information and Software Technology, 2000, 42(10): 665-675.
- [19] Scheer A W. ARIS—business process modeling[M]. Springer Science & Business Media, 2000.
- [20] Steinberg D, Budinsky F, Merks E, et al. EMF: eclipse modeling framework[M]. Pearson Education, 2008.
- [21] Ledecz A, Maroti M, Bakay A, et al. The generic modeling environment[C]//Workshop on Intelligent Signal Processing, Budapest, Hungary. 2001, 17(01): 2001.
- [22] Tolvanen J P, Pohjonen R, Kelly S. Advanced tooling for domain-specific modeling: MetaEdit+[C]//The 7th OOPSLA Workshop on Domain-Specific Modeling, Finland. sn, 2007: 104.
- [23] Cook S, Jones G, Kent S, et al. Domain-specific development with visual studio dsl tools[M]. Pearson Education, 2007.
- [24] Biafore, B.: Visio 2007 Bible. John Wiley & Sons Inc. 2007.
- [25] H.-P. Hoffmann, Systems engineering best practices with the Rational Workbench for Systems and Software Engineering, IBM Corporation, Deskbook Release 3.1.1 edition, 2010.
- [26] Lykins H, Friedenthal S, Meilich A. 4.4.4 Adapting UML for an Object Oriented Systems Engineering Method (OOSEM)[J]. INCOSE International Symposium, 2000,10(1).
- [27] Morkevicius A, Aleksandraviciene A, Mazeika D, et al. MBSE Grid: A Simplified SysML - Based Approach for Modeling Complex Systems[C]. INCOSE International Symposium. 2017, 27(1): 136-150.
- [28] Šilingas D, Butleris R. Towards implementing a framework for modeling software requirements in MagicDraw UML[J]. Information Technology and Control, 2009, 38(2).
- [29] Roques P. MBSE with the ARCADIA Method and the Capella Tool[C]. 8th European Congress on Embedded Real Time Software and Systems. 2016.
- [30] Gery E, Harel D, Palachi E. Rhapsody: A complete life-cycle model-based development system[C].

- International Conference on Integrated Formal Methods. Springer, Berlin, Heidelberg, 2002: 1-10.
- [31] Matthes F, Buckl S, Leitel J, et al. Enterprise architecture management tool survey 2008[M]. München: Techn. Univ. München, 2008.
- [32] 蒋浴芹. M-Design-多域产品系统建模平台的研究与实现[D].浙江大学,2011.
- [33] 杉石科技. Modelook[Z]. <http://www.sysmagic.com.cn/Home/ProductModelook>.2023
- [34] Lindland O I, Sindre G, Solvberg A. Understanding quality in conceptual modeling[J]. IEEE software, 1994, 11(2): 42-49.
- [35] J. Krogstie, A. Sølvsberg, Information Systems Engineering: Conceptual Modeling in a Quality perspective, Kompendiumforlaget, Norway, 2000.
- [36] Unhelkar B. Verification and validation for quality of UML 2.0 models[M]. John Wiley & Sons, 2005.
- [37] Mohagheghi P, Dehlen V, Neple T. Definitions and approaches to model quality in model-based software development – A review of literature[J]. Information and software technology, 2009, 51(12): 1646-1669.
- [38] Kahraman G, Bilgen S. A framework for qualitative assessment of domain-specific languages[J]. Software & Systems Modeling, 2015, 14: 1505-1526.
- [39] Berenbach B. The evaluation of large, complex UML analysis and design models[C]//Proceedings. 26th International Conference on Software Engineering. IEEE, 2004: 232-241.
- [40] Sourrouille J L, Caplat G. Constraint checking in UML modeling[C]//Proceedings of the 14th international conference on Software engineering and knowledge engineering. 2002: 217-224.
- [41] Feldmann S, Herzig S J I, Kernschmidt K, et al. A comparison of inconsistency management approaches using a mechatronic manufacturing system design case study[C]//2015 IEEE International Conference on Automation Science and Engineering (CASE). IEEE, 2015: 158-165.
- [42] 陈钢,于林宇,裘宗燕等.基于逻辑的形式化验证方法:进展及应用[J].北京大学学报(自然科学版),2016,52(02):363-373.DOI:10.13209/j.0479-8023.2015.131.
- [43] Yang B, Bryant R E, O' Hallaron D R, et al. A performance study of BDD-based model checking[C]//Formal Methods in Computer-Aided Design: Second International Conference, FMCAD' 98 Palo Alto, CA, USA, November 4 – 6, 1998 Proceedings 2. Springer Berlin Heidelberg, 1998: 255-289.
- [44] Soeken M, Wille R, Kuhlmann M, et al. Verifying UML/OCL models using Boolean

- satisfiability[C]//2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010). IEEE, 2010: 1341-1344.
- [45] Soeken M, Wille R, Drechsler R. Verifying dynamic aspects of UML models[C]//2011 Design, Automation & Test in Europe. IEEE, 2011: 1-6.
- [46] Abrial J R. The B-book: assigning programs to meanings. Cambridge: Cambridge University Press, 2005.
- [47] Bousse E, Mentré D, Combemale B, et al. Aligning SysML with the B method to provide V&V for systems engineering[C]//Proceedings of the Workshop on Model-driven Engineering, Verification and Validation. 2012: 11-16.
- [48] Behrmann G, David A, Larsen K G. A tutorial on uppaal[J]. Formal Methods for the Design of Real-Time Systems: International School on Formal Methods for the Design of Computer, Communication, and Software Systems, Bertinora, Italy, September 13-18, 2004, Revised Lectures, 2004: 200-236.
- [49] Abdulhameed A, Hammad A, Mountassir H, et al. An approach combining simulation and verification for SysML using SystemC and Uppaal[C]//CAL 2014, 8ème conférence francophone sur les architectures logicielles. 2014: 9 pages.
- [50] Liu W Q, Easterbrook S, Mylopoulos J. Rule-based detection of inconsistency in UML models[C]//Workshop on Consistency Problems in UML-Based Software Development. 2002, 5.
- [51] Herzig S J I, Qamar A, Paredis C J J. An approach to identifying inconsistencies in model-based systems engineering[J]. Procedia Computer Science, 2014, 28: 354-362.
- [52] Wei B, Sun J. Leveraging SPARQL Queries for UML Consistency Checking[J]. International Journal of Software Engineering and Knowledge Engineering, 2021, 31(04): 635-654.
- [53] Shaikh A, Wiil U K. A feedback technique for unsatisfiable UML/OCL class diagrams[J]. Software: Practice and Experience, 2014, 44(11): 1379-1393.
- [54] Kolovos D S, Paige R F, Polack F A C. Eclipse development tools for epsilon[C]//Eclipse summit Europe, eclipse modeling symposium. 2006, 20062: 200.
- [55] Ehrig K, Küster J M, Taentzer G. Generating instance models from meta models[J]. 2008.
- [56] OMG. Meta Object Facility (MOF) Specification. Object Management Group, 2000.
- [57] Kern H, Hummel A, Kühne S. Towards a comparative analysis of meta-metamodels[C]//Proceedings of the compilation of the co-located workshops on DSM'11, TMC'11, AGERE! 2011, AOOPES'11,

- NEAT'11, & VMIL'11. 2011: 7-12.
- [58] Thacker B H, Doebbling S W, Hemez F M, et al. Concepts of model verification and validation[J]. 2004.
- [59] Feldmann S, Herzig S J I, Kernschmidt K, et al. A comparison of inconsistency management approaches using a mechatronic manufacturing system design case study[C]//2015 IEEE International Conference on Automation Science and Engineering (CASE). IEEE, 2015: 158-165.
- [60] Clarke E M. Model checking[C]//Foundations of Software Technology and Theoretical Computer Science: 17th Conference Kharagpur, India, December 18 – 20, 1997 Proceedings 17. Springer Berlin Heidelberg, 1997: 54-56.
- [61] Hehenberger P, Egyed A, Zeman K. Consistency checking of mechatronic design models[C].International Design Engineering Technical Conferences and Computers and Information in Engineering Conference. 2010, 44113: 1141-1148.
- [62] Nuseibeh B, Easterbrook S, Russo A. Leveraging inconsistency in software development[J]. Computer, 2000, 33(4): 24-29.
- [63] Heidenreich F, Johannes J, Karol S, et al. Integrating OCL and textual modelling languages[C]//Models in Software Engineering: Workshops and Symposia at MODELS 2010, Oslo, Norway, October 2-8, 2010, Reports and Revised Selected Papers 13. Springer Berlin Heidelberg, 2011: 349-363.
- [64] Warner J B, Kleppe A G. The object constraint language: getting your models ready for MDA[M]. Addison-Wesley Professional, 2003.
- [65] Kleppe A, Rensink A. On a graph-based semantics for UML class and object diagrams[J]. Electronic Communications of the EASST, 2008, 10: 1-16.
- [66] Halpin T A. Object-Role Modeling (ORM/NIAM)[J]. Handbook on architectures of information systems, 2006, 81: 103.
- [67] Grunske L, Geiger L, Zündorf A, et al. Using graph transformation for practical model-driven software engineering[J]. Model-driven Software Development, 2005: 91-117.
- [68] Champin P A, Solnon C. Measuring the similarity of labeled graphs[C]//Case-Based Reasoning Research and Development: 5th International Conference on Case-Based Reasoning, ICCBR 2003 Trondheim, Norway, June 23 – 26, 2003 Proceedings 5. Springer Berlin Heidelberg, 2003: 80-95.

- [69] Ehrig Hartmut, Ehrig Karsten. Overview of Formal Concepts for Model Transformations Based on Typed Attributed Graph Transformation[J]. Electronic Notes in Theoretical Computer Science, 2006, 152: 3-22.
- [70] Larrosa J, Valiente G. Constraint satisfaction algorithms for graph pattern matching[J]. Mathematical structures in computer science, 2002, 12(4): 403-422.
- [71] Gallagher B. Matching Structure and Semantics: A Survey on Graph-Based Pattern Matching[C]//AAAI Fall Symposium: Capturing and Using Patterns for Evidence Detection. 2006, 45.
- [72] Smullyan R M. First-order logic[M]. Courier Corporation, 1995.
- [73] Pospesel H. Introduction to logic: Propositional logic[J]. 1974.
- [74] Krajicek J. Bounded arithmetic, propositional logic and complexity theory[M]. Cambridge University Press, 1995.
- [75] 王伟静,陈荟慧.谓词逻辑中前束范式的求解方法研究[J].现代计算机(专业版),2012,No.388(04):14-15+22.
- [76] Parr, T. (2013). The definitive ANTLR 4 reference.
- [77] De Moura L, Bjørner N. Z3: An efficient SMT solver[C]//International conference on Tools and Algorithms for the Construction and Analysis of Systems. Springer, Berlin, Heidelberg, 2008: 337-340.
- [78] Taentzer G. AGG: A graph transformation environment for modeling and validation of software[C]//International workshop on applications of graph transformations with industrial relevance. Springer, Berlin, Heidelberg, 2004: 446-453.
- [79] Bardohl R, Ermel C, Weinhold I. AGG and GenGED: Graph Transformation-Based Specification and Analysis Techniques for Visual Languages[J]. Electronic Notes in Theoretical Computer Science, 2002, 72(2): 14-24.
- [80] 杨兆瑞,于翔,王坚,鲁金直,兰小平,姚春波.多架构体系建模与仿真联合平台[J].计算机系统应用,2021,30(12):63-72.DOI:10.15888/j.cnki.csa.008226.
- [81] 辛高波,王猛,丁秀峰等.运载火箭冗余分级智能供配电技术研究[J].计算机测量与控制,2018,26(10):137-140+154.DOI:10.16526/j.cnki.11-4762/tp.2018.10.030.

攻读学位期间发表论文与研究成果清单

参与科研项目：

- (1) 2020-2023，科技部复杂产品设计制造服务一体化建模理论。
- (2) 2020-2023，装备预先研究项目“多系统建模与设计空间智能决策技术”。
- (3) 2021-2022，国防基础科研项目“面向 MBSE 的雷达系统总体布局研究”。

发表论文：

- (1) Integration of modeling and verification for system model based on KARMA language[C]//Proceedings of the 18th ACM SIGPLAN International Workshop on Domain-Specific Modeling. 2021: 41-50.（第一作者，EI 检索）
- (2) Code Generation Approach Supporting Complex System Modeling based on Graph Pattern Matching[J]. IFAC-PapersOnLine, 2022, 55(10): 3004-3009.（第一作者，EI 检索，论文 4.3 部分）
- (3) A Model Query Language for Domain-Specific Models[C]//2020 5th International Conference on Mechanical, Control and Computer Engineering (ICMCCE). IEEE, 2020: 1207-1213.（第三作者，EI 检索）

软件著作权：

- (1) 第二作者（导师一作）取得软著“多域一体化建模语言代码生成器软件”。
- (2) 第二作者（导师一作）取得软著“多域一体化建模语言解释器软件”。

相关专利：

- (1) 第二作者（导师一作）申请专利：一种支持复杂模型结构转换的代码生成方法，专利号：202110822605.9。

相关竞赛：

- (1) 2021 年 9 月，北京理工大学第十七届“世纪杯”学生创业竞赛银奖。

- (2) 2021 年 9 月，第六届中国国际“互联网+”大学生创新创业大赛国赛入围。
- (3) 2021 年 8 月，北京市“互联网+”大学生创新创业大赛一等奖。
- (4) 2021 年 6 月，北京市优秀创业团队一等奖。
- (5) 2022 年 6 月，第十三届“挑战杯”中国大学生创业计划竞赛北京理工大学校级金奖。

致谢

临近答辩前夕，才逐渐感受到我的研究生生涯到了尾声，致谢预示着结束，也预示着新的开始。

盛行千里，不忘师恩。首先感谢我的导师：姚丽亚和王国新老师。姚丽亚老师以温柔的态度关心着我的日常生活，在我遇到问题时给予了我很大的帮助。在伴随着三年疫情的研究生生涯中，姚丽亚老师无时无刻不关心着我的身体健康和人身安全。王国新老师是我科研生活中的良师益友，从我三年前加入知识工程课题组开始，王国新老师就以他治学严谨、踏实肯干、认真负责的作风春风化雨般影响着我的科研和工作态度。此外，王国新老师悉心指导我的毕业设计过程，为我从开题到定稿的整个过程提出了许多宝贵的意见，帮我攻克难关。在生活和就业工作方面，王老师也给了我方向指引和莫大的帮助。其次，感谢鲁金直博士对我科研上的帮助，他虽身在国外，但却一次次通过线上会议对我科研中遇到的技术问题和论文写作问题进行指导，帮助我提升独立解决问题的能力。另外，知识工程组的阎艳老师、郝佳老师、明振军老师、王儒老师、黄思翰老师也为我的科研和生活提供了指导和帮助，十分感谢！

山水一程，有幸遇见。进入知识工程组这个大家庭，加入系统工程小组，让我研究生三年的生活充满了欢声笑语。首先感谢我的师兄师姐们，郭江敏师兄的帮助让我从一个懵懂的本科生快速成长为一个具有一定科研思维的研究生。同时感谢马君达、余士严、栗寒师兄，靳益利、陈婧琦师姐对我困惑时的开导和就业时的帮助，我也愿意将你们的帮助传递给我的师弟师妹们。吴绶玄、李子航是与我同时加入课题组的同学，三年前的暑假我们相见合肥，三年间我们互相鼓励、共同奋斗，一起赴上海进行半年的实践，这其中太多的故事值得回忆。感谢你们的陪伴，也由衷地祝福我们都有美好的未来。同时感谢我可爱的师弟师妹们：董梦如、陈锐、刘智清和李嘉伟，谢谢你们帮助我仔细检查毕业论文和答辩 PPT 中的格式问题。另外，知识工程组的其他同学们，如万斯来师兄、黄禹师兄、李传浩师兄、朱志成师兄、聂士琪同学、曹贝宁同学等，大家都以热忱善良的心对我的生活提供了帮助，谢谢你们！

愿岁并谢，与友长兮。感谢一路走来的小伙伴们：谢宇静、司聪敏、周姝男、冯银花、彭茜、汪静、黄颂林。你们在不同阶段陪伴我走过了高中、本科和研究生的学习生涯。在我难过、生气、郁闷、低落时倾听我、开解我、鼓励我、支持我，给予我的生活许多快乐，

让我的生活充满阳光和希望。能遇见你们是我觉得特别幸运的事情，生活一直在继续，希望我们的友谊长存！此外，还需要感谢陪伴我三年的舍友们，她们是：蒲雪宁、张馥琳和王滨珏。你们为我营造了一个温暖、和气、健康、包容、充满欢乐的宿舍环境，也经常倾听我在科研和生活中遇到的困难，排解我的情绪，让我在集体宿舍也能找到倚靠的港湾。

父母之恩，铭记于心。感谢父母对我一如既往的关心、鼓励和支持，包容我的脾气、尊重我的选择，让我衣食无忧、平安健康地成长。你们多年来的默默付出，是我奋斗的源动力。感谢其他家人，爷爷经常送我上学，语重心长地教导我，奶奶为我的身体健康和饮食付出了很多努力，你们的关心藏在我生活的点点滴滴中。

最后，感谢北京理工大学给我提供的良好的学习生活环境，让我汲取知识、锤炼品格、开拓视野。在我以后的人生路中我将谨记“德以明理、学以精工”的校训，做一个有价值的人。感谢三年求学生涯期间坚守在工作岗位的各位老师、职工和同学，为我们的学校生活保驾护航；感谢各位老师和专家在百忙之中评审本论文，并提出宝贵的意见。向各位辛苦的老师致敬！

二零二三年五月
于北京理工大学 1 号教学楼